Vibe Coding

שיעור 4 - Context Engineering

הקשר לאורך פרויקט שלם

ההודעה הנוכחית | קבצים | היסטוריה | קובץ ההנחיה | הוראות הכלי

ד״ר ולד לנדא · אוניברסיטת אריאל

Vibe Coding

שיעור 4 - מעבר מפרומפט בודד לפרויקט שלם

במפגש 3 סגרנו החלטות פתוחות בתוך **פרומפט אחד**, ולמדנו למדוד את איכות הפרומפט כהסתברות הצלחה $P_{\text{succ}}(x)$.

מפגש 3
פרומפט אחד, שיחה אחת

Vibe Coding

שיעור 4 - מעבר מפרומפט בודד לפרויקט שלם

במפגש 3 סגרנו החלטות פתוחות בתוך **פרומפט אחד**, ולמדנו למדוד את איכות הפרומפט כהסתברות הצלחה $P_{\text{succ}}(x)$.

בפרויקט אמיתי כותבים **עשרות בקשות**, על פני ימים ובכמה שיחות. לפעמים כמה אנשים וכמה כלים עובדים על אותו קוד.

שיחה 1
יום א׳: add_task

שיחה 2
יום ב׳: delete_task

שיחה 3
יום ד׳: mark_done

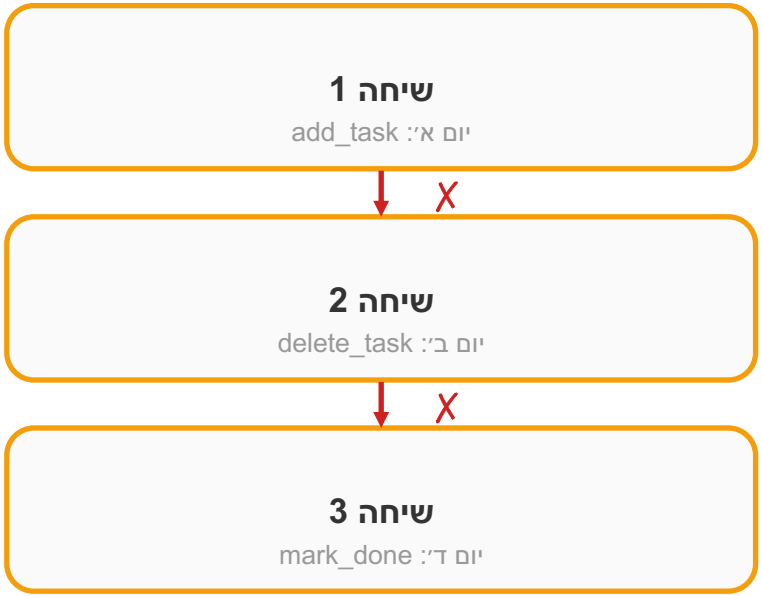
Vibe Coding

שיעור 4 - מעבר מפרומפט בודד לפרויקט שלם

במפגש 3 סגרנו החלטות פתוחות בתוך **פרומפט אחד**, ולמדנו למדוד את איכות הפרומפט כהסתברות הצלחה $P_{\text{succ}}(x)$.

בפרויקט אמיתי כותבים **עשרות בקשות**, על פני ימים ובכמה שיחות. לפעמים כמה אנשים וכמה כלים עובדים על אותו קוד.

במפגש 2 ראינו ש**מודל אין זיכרון בין שיחות.** גם בתוך שיחה הוא לא "זוכר", אלא קורא מחדש את מה שנשאר בחלון.



Vibe Coding

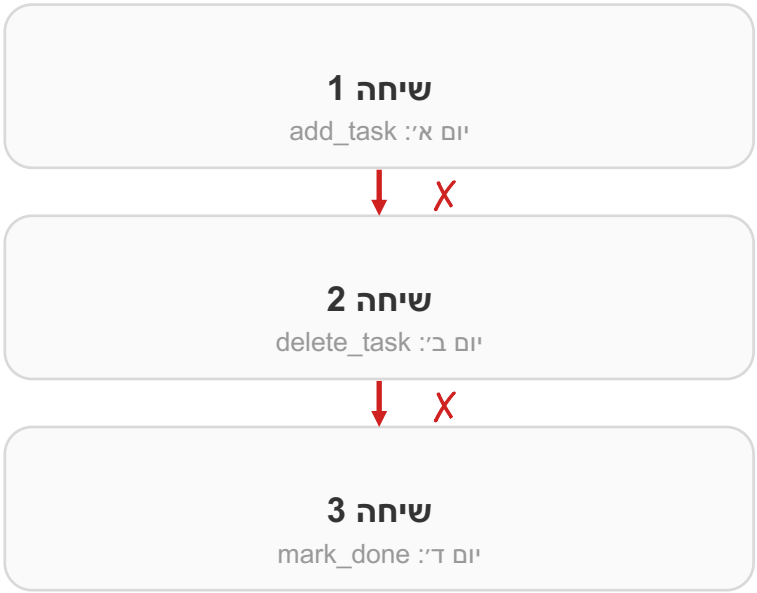
שיעור 4 - מעבר מפרומפט בודד לפרויקט שלם

במפגש 3 סגרנו החלטות פתוחות בתוך **פרומפט אחד**, ולמדנו למדוד את איכות הפרומפט כהסתברות הצלחה $P_{\text{succ}}(x)$.

בפרויקט אמיתי כותבים **עשרות בקשות**, על פני ימים ובכמה שיחות. לפעמים כמה אנשים וכמה כלים עובדים על אותו קוד.

במפגש 2 ראינו של**מודל אין זיכרון בין שיחות.** גם בתוך שיחה הוא לא "זוכר", אלא קורא מחדש את מה שנשאר בחלון.

שאלה: אז מה קורה למוסכמות של הפרויקט (למשל "type hints בכל פונקציה") כשפותחים שיחה חדשה?



Vibe Coding

שיעור 4 - מעבר מפרומפט בודד לפרויקט שלם

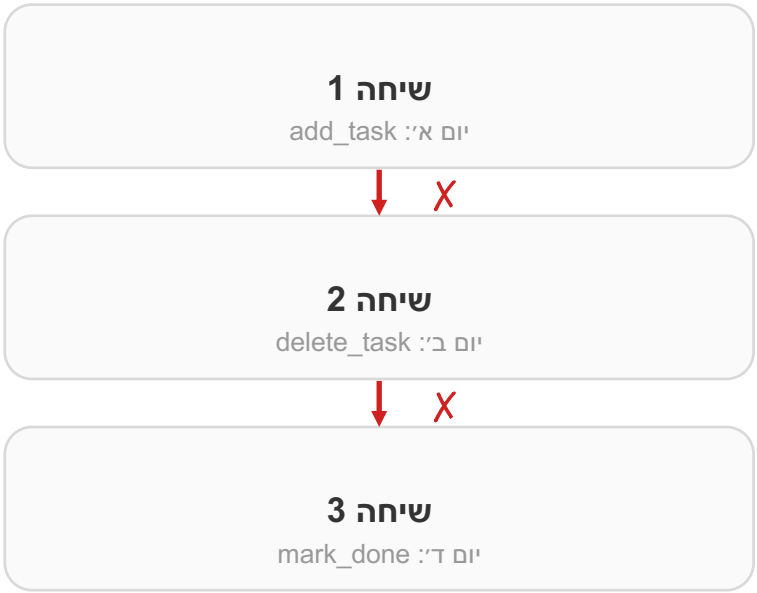
במפגש 3 סגרנו החלטות פתוחות בתוך **פרומפט אחד**, ולמדנו למדוד את איכות הפרומפט כהסתברות הצלחה $P_{\text{succ}}(x)$.

בפרויקט אמיתי כותבים **עשרות בקשות**, על פני ימים ובכמה שיחות. לפעמים כמה אנשים וכמה כלים עובדים על אותו קוד.

במפגש 2 ראינו של**מודל אין זיכרון בין שיחות.** גם בתוך שיחה הוא לא "זוכר", אלא קורא מחדש את מה שנשאר בחלון.

שאלה: אז מה קורה למוסכמות של הפרויקט (למשל "type hints בכל פונקציה") כשפותחים שיחה חדשה?

תשובה: הן **חוזרות להיות החלטות פתוחות:** המוסכמה לא נמצאת בחלון של השיחה החדשה, והמודל ינחש אותה מחדש.



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - מה זה Context Engineering?

Context Engineering הוא ההחלטה **מה נכנס לחלון ההקשר בכל בקשה**: אילו הוראות קבועות, אילו קבצים, כמה היסטוריה, ובאיזו צורה.

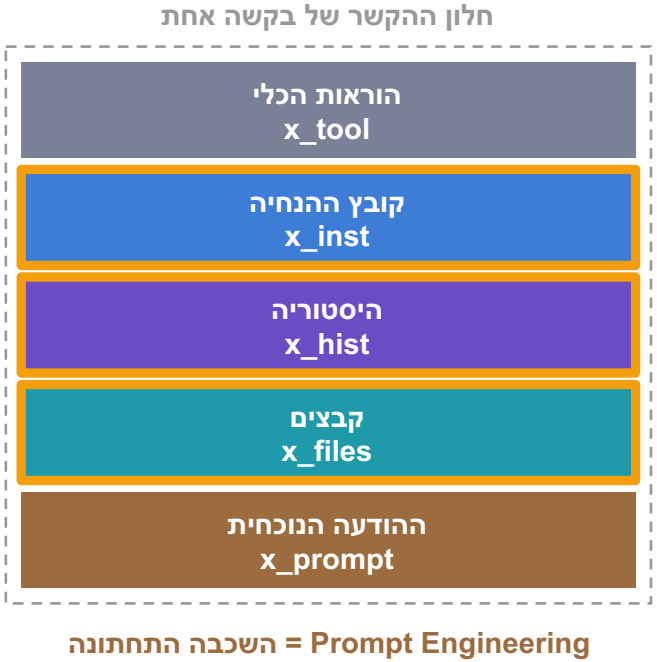


Vibe Coding

שיעור 4 - מה זה Context Engineering?

Context Engineering הוא ההחלטה **מה נכנס לחלון ההקשר בכל בקשה**: אילו הוראות קבועות, אילו קבצים, כמה היסטוריה, ובאיזו צורה.

אם Prompt Engineering (מפגש 3) הוא בחירת **ההודעה** שלכם, Context Engineering הוא בחירה של **כל השאר**: שלוש השכבות המודגשות באיור.



ד״ר ולד לנדא

Vibe Coding

שיעור 4 - מפת המפגש

השאלה של המפגש: **מה המודל רואה בכל בקשה, ואיך שולטים בזה?**

1. **חלק א׳:** מפרקים את הקלט לחמישה חלקים. שלושה מהם בשליטתכם: קובץ ההנחיה, הקבצים וההיסטוריה.
2. **חלק ב׳:** הבעיה. כשאין מקום קבוע למוסכמות, הקוד לא נשאר עקבי, ונחשב עד כמה.
3. **חלקים ג׳-ה׳:** הפתרון, קובץ הנחיית פרויקט: מה כותבים בו, אילו כללים שווים את המקום, ואיך שומרים אותו מעודכן.
4. **חלק ו׳:** אילו קבצים מצרפים לבקשה, ובאיזו רמת פירוט.
5. **חלק ז׳:** ההיסטוריה: מה קורה כששיחה מתארכת, ומתי לפתוח שיחה חדשה.
6. **חלק ח׳:** מבנה תיקיות ושמות שעוזרים לבחור את ההקשר הנכון.



דוגמה מלווה לכל השיעור: **אפליקציית Todo** בפייתון.

ד״ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - מה המודל רואה בכל בקשה

<u>למה זה חשוב:</u>

"המודל לא ידע שאנחנו עובדים עם Python 3.11" היא תלונה נפוצה, אבל היא מנוסחת לא נכון.

חלון ההקשר של בקשה אחת



Vibe Coding

שיעור 4 - חלק א׳ - מה המודל רואה בכל בקשה

<u>למה זה חשוב:</u>

"המודל לא ידע שאנחנו עובדים עם Python 3.11" היא תלונה נפוצה, אבל היא מנוסחת לא נכון.

שאלה: איפה המודל היה אמור לראות שהפרויקט כתוב ב-Python 3.11?



Vibe Coding

שיעור 4 - חלק א׳ - מה המודל רואה בכל בקשה

<u>למה זה חשוב:</u>

"המודל לא ידע שאנחנו עובדים עם Python 3.11" היא תלונה נפוצה, אבל היא מנוסחת לא נכון.

שאלה: איפה המודל היה אמור לראות שהפרויקט כתוב ב-Python 3.11?

תשובה: רק **בקלט של הבקשה הזו.** למודל אין ידע על הפרויקט מעבר למה שנמצא בקלט שלו ברגע הבקשה, כולל דברים שאתם לא כתבתם ולא רואים.



ד״ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - מה המודל רואה בכל בקשה

<u>למה זה חשוב:</u>

"המודל לא ידע שאנחנו עובדים עם Python 3.11" היא תלונה נפוצה, אבל היא מנוסחת לא נכון.

שאלה: איפה המודל היה אמור לראות שהפרויקט כתוב ב-Python 3.11?

תשובה: רק **בקלט של הבקשה הזו.** למודל אין ידע על הפרויקט מעבר למה שנמצא בקלט שלו ברגע הבקשה, כולל דברים שאתם לא כתבתם ולא רואים.

<u>ההסבר:</u>

כשאתם שולחים הודעה, הכלי בונה את הקלט למודל מחמישה חלקים, ושולח את כולם יחד:

חלון ההקשר של בקשה אחת

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - מה המודל רואה בכל בקשה

<u>למה זה חשוב:</u>

"המודל לא ידע שאנחנו עובדים עם Python 3.11" היא תלונה נפוצה, אבל היא מנוסחת לא נכון.

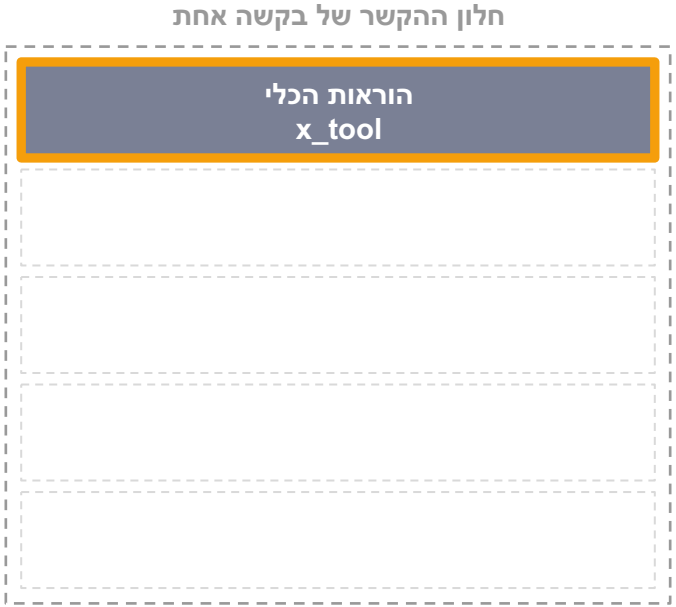
שאלה: איפה המודל היה אמור לראות שהפרויקט כתוב ב-Python 3.11?

תשובה: רק **בקלט של הבקשה הזו.** למודל אין ידע על הפרויקט מעבר למה שנמצא בקלט שלו ברגע הבקשה, כולל דברים שאתם לא כתבתם ולא רואים.

<u>ההסבר:</u>

כשאתם שולחים הודעה, הכלי בונה את הקלט למודל מחמישה חלקים, ושולח את כולם יחד:

1. **הוראות הכלי:** הנחיית מערכת שכתב הספק. אתם לא כותבים אותה, ובדרך כלל גם לא רואים.



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - מה המודל רואה בכל בקשה

למה זה חשוב:

"המודל לא ידע שאנחנו עובדים עם Python 3.11" היא תלונה נפוצה, אבל היא מנוסחת לא נכון.

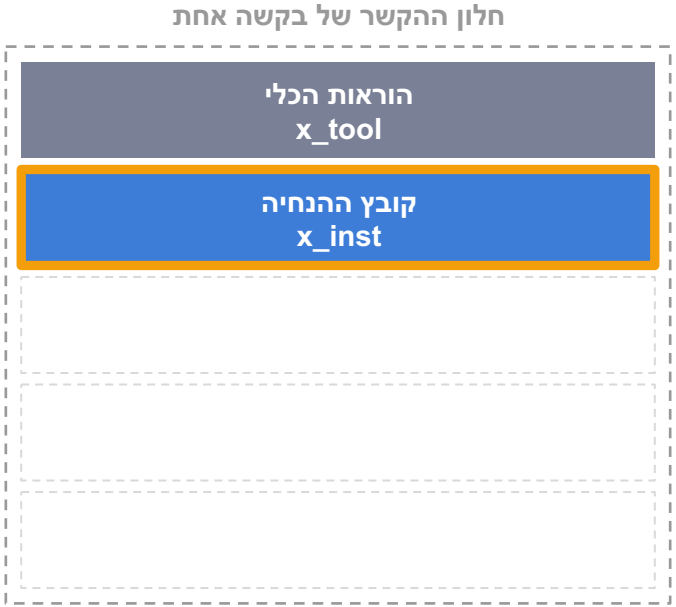
שאלה: איפה המודל היה אמור לראות שהפרויקט כתוב ב-Python 3.11?

תשובה: רק **בקלט של הבקשה הזו.** למודל אין ידע על הפרויקט מעבר למה שנמצא בקלט שלו ברגע הבקשה, כולל דברים שאתם לא כתבתם ולא רואים.

ההסבר:

כשאתם שולחים הודעה, הכלי בונה את הקלט למודל מחמישה חלקים, ושולח את כולם יחד:

1. **הוראות הכלי:** הנחיית מערכת שכתב הספק. אתם לא כותבים אותה, ובדרך כלל גם לא רואים.
2. **קובץ ההנחיה** של הפרויקט, אם הכלי טוען כזה (חלק ג׳).



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - מה המודל רואה בכל בקשה

<u>למה זה חשוב:</u>

"המודל לא ידע שאנחנו עובדים עם Python 3.11" היא תלונה נפוצה, אבל היא מנוסחת לא נכון.

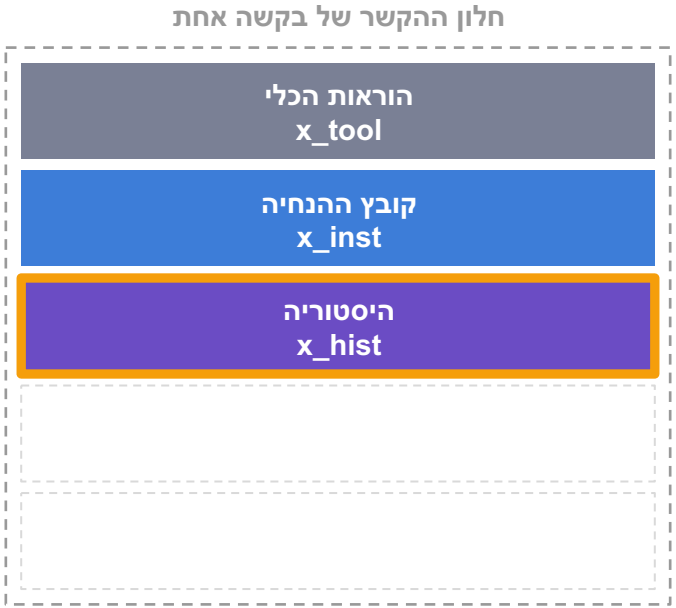
שאלה: איפה המודל היה אמור לראות שהפרויקט כתוב ב-Python 3.11?

תשובה: רק **בקלט של הבקשה הזו.** למודל אין ידע על הפרויקט מעבר למה שנמצא בקלט שלו ברגע הבקשה, כולל דברים שאתם לא כתבתם ולא רואים.

<u>ההסבר:</u>

כשאתם שולחים הודעה, הכלי בונה את הקלט למודל מחמישה חלקים, ושולח את כולם יחד:

1. **הוראות הכלי:** הנחיית מערכת שכתב הספק. אתם לא כותבים אותה, ובדרך כלל גם לא רואים.
2. **קובץ ההנחיה** של הפרויקט, אם הכלי טוען כזה (חלק ג׳).
3. **ההיסטוריה:** ההודעות הקודמות בשיחה, או מה שנשאר מהן (חלק ז׳).



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - מה המודל רואה בכל בקשה

<u>למה זה חשוב:</u>

"המודל לא ידע שאנחנו עובדים עם Python 3.11" היא תלונה נפוצה, אבל היא מנוסחת לא נכון.

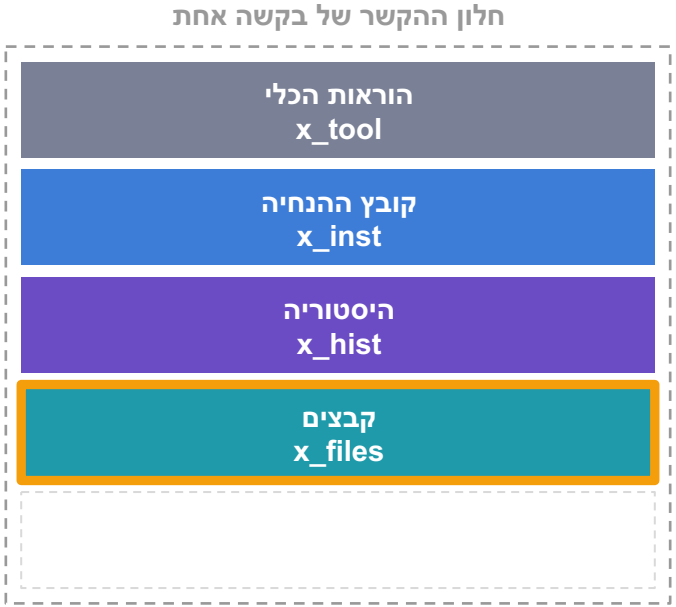
שאלה: איפה המודל היה אמור לראות שהפרויקט כתוב ב-Python 3.11?

תשובה: רק **בקלט של הבקשה הזו.** למודל אין ידע על הפרויקט מעבר למה שנמצא בקלט שלו ברגע הבקשה, כולל דברים שאתם לא כתבתם ולא רואים.

<u>ההסבר:</u>

כשאתם שולחים הודעה, הכלי בונה את הקלט למודל מחמישה חלקים, ושולח את כולם יחד:

1. **הוראות הכלי:** הנחיית מערכת שכתב הספק. אתם לא כותבים אותה, ובדרך כלל גם לא רואים.
2. **קובץ ההנחיה** של הפרויקט, אם הכלי טוען כזה (חלק ג׳).
3. **ההיסטוריה:** ההודעות הקודמות בשיחה, או מה שנשאר מהן (חלק ז׳).
4. **קבצים** שצירפתם, או שהכלי בחר בעצמו (חלק ו׳).



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - מה המודל רואה בכל בקשה



<u>למה זה חשוב:</u>

"המודל לא ידע שאנחנו עובדים עם Python 3.11" היא תלונה נפוצה, אבל היא מנוסחת לא נכון.

שאלה: איפה המודל היה אמור לראות שהפרויקט כתוב ב-Python 3.11?

תשובה: רק **בקלט של הבקשה הזו.** למודל אין ידע על הפרויקט מעבר למה שנמצא בקלט שלו ברגע הבקשה, כולל דברים שאתם לא כתבתם ולא רואים.

<u>ההסבר:</u>

כשאתם שולחים הודעה, הכלי בונה את הקלט למודל מחמישה חלקים, ושולח את כולם יחד:

1. **הוראות הכלי:** הנחיית מערכת שכתב הספק. אתם לא כותבים אותה, ובדרך כלל גם לא רואים.
2. **קובץ ההנחיה** של הפרויקט, אם הכלי טוען כזה (חלק ג׳).
3. **ההיסטוריה:** ההודעות הקודמות בשיחה, או מה שנשאר מהן (חלק ז׳).
4. **קבצים** שצירפתם, או שהכלי בחר בעצמו (חלק ו׳).
5. **ההודעה הנוכחית:** מה שכתבתם עכשיו.

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - ניסוח פורמלי

ניסוח פורמלי - הקלט כשרשור של חלקים:

<u>הגדרות:</u>

נסמן ב-x את הקלט שהמודל מקבל בכל בקשה, כמו במפגשים 2 ו-3.
בפועל x הוא **שרשור** (צירוף ברצף, חלק אחרי חלק) של חמשת החלקים מהשקופית הקודמת:

$$x = \left(x_{\text{tool}},\ x_{\text{inst}},\ x_{\text{hist}},\ x_{\text{files}},\ x_{\text{prompt}}\right)$$



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - ניסוח פורמלי

ניסוח פורמלי - הקלט כשרשור של חלקים:

<u>הגדרות:</u>

נסמן ב-x את הקלט שהמודל מקבל בכל בקשה, כמו במפגשים 2 ו-3.
בפועל x הוא **שרשור** (צירוף ברצף, חלק אחרי חלק) של חמשת החלקים מהשקופית הקודמת:

$$x = \left(x_{\text{tool}},\ x_{\text{inst}},\ x_{\text{hist}},\ x_{\text{files}},\ x_{\text{prompt}}\right)$$

כאן x_{tool} הן הוראות הכלי, x_{inst} קובץ ההנחיה, x_{hist} ההיסטוריה, x_{files} הקבצים המצורפים, ו-x_{prompt} ההודעה הנוכחית.



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - ניסוח פורמלי: תקציב החלון

<u>תקציב החלון:</u>

במפגש 2 ראינו שהקלט **והתשובה** חייבים להיכנס יחד לחלון ההקשר, שאורכו L טוקנים. נסמן ב-n את מספר הטוקנים של כל חלק (למשל n_{tool} להוראות הכלי), וב-n_{answer} את אורך התשובה:

$$\underbrace{n_{\text{tool}} + n_{\text{inst}}}_{n_{\text{system}}} + n_{\text{history}} + n_{\text{files}} + n_{\text{prompt}} + n_{\text{answer}} \leq L$$



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - ניסוח פורמלי: תקציב החלון

<u>תקציב החלון:</u>

במפגש 2 ראינו שהקלט **והתשובה** חייבים להיכנס יחד לחלון ההקשר, שאורכו L טוקנים. נסמן ב-n את מספר הטוקנים של כל חלק (למשל n_{tool} להוראות הכלי), וב-n_{answer} את אורך התשובה:

$$\underbrace{n_{\text{tool}} + n_{\text{inst}}}_{n_{\text{system}}} + n_{\text{history}} + n_{\text{files}} + n_{\text{prompt}} + n_{\text{answer}} \leq L$$

במפגש 2 שני האיברים הראשונים הופיעו יחד, כ-n_{system}. כאן מפרידים ביניהם, כי את x_{tool} קובע הספק ואת x_{inst} כותבים אתם.



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק א׳ - מסקנות

<u>מסקנות:</u>

1. **כל מה שהמודל "יודע" על הפרויקט נמצא באחד מחמשת החלקים.** ידע כללי (איך עובדת pandas) מגיע מהמשקלים θ של המודל, כלומר ממה שלמד באימון (מפגש 2); ידע על **הפרויקט שלכם** מגיע רק מ-x.



ד"ר ולד לנדא

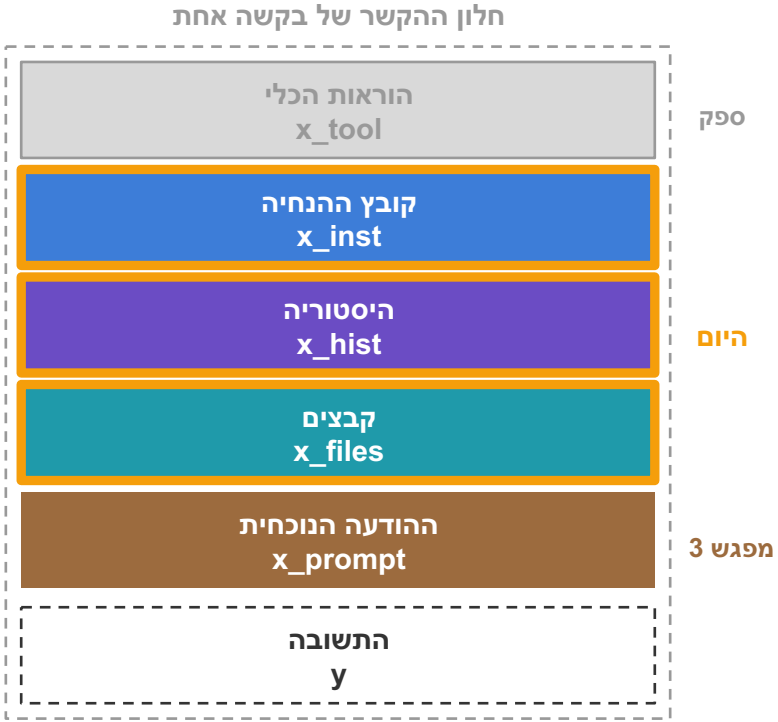
Vibe Coding

שיעור 4 - חלק א׳ - מסקנות

<u>מסקנות:</u>

1. **כל מה שהמודל "יודע" על הפרויקט נמצא באחד מחמשת החלקים.** ידע כללי (איך עובדת pandas) מגיע מהמשקלים θ של המודל, כלומר ממה שלמד באימון (מפגש 2); ידע על **הפרויקט שלכם** מגיע רק מ-x.

2. **ההודעה היא רק חלק אחד מחמישה.** במפגש 3 שלטנו ב-x_{prompt}. היום שולטים גם בשלושה נוספים: x_{inst}, x_{hist} ו-x_{files} (את x_{tool} קובע הספק).



Vibe Coding

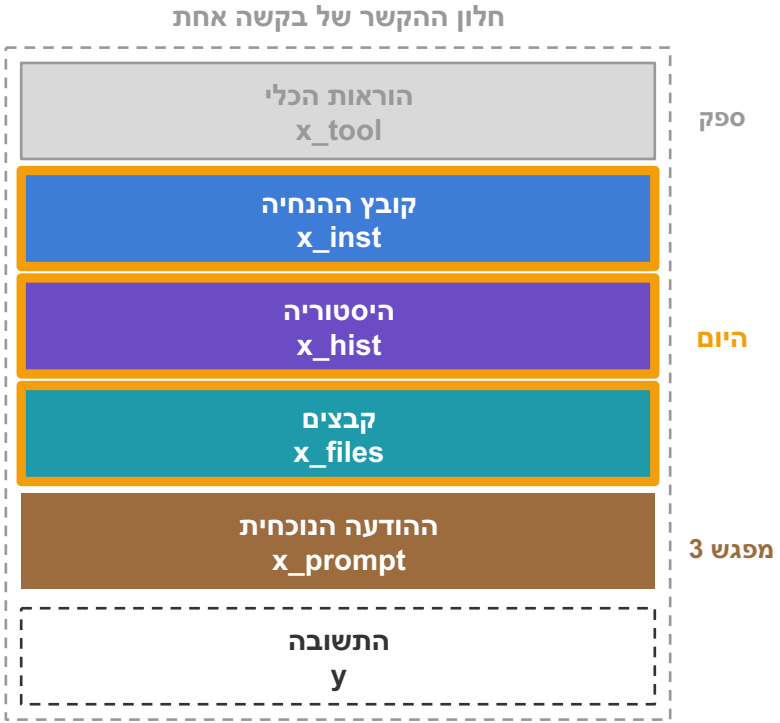
שיעור 4 - חלק א׳ - מסקנות

<u>מסקנות:</u>

1. **כל מה שהמודל "יודע" על הפרויקט נמצא באחד מחמשת החלקים.** ידע כללי (איך עובדת pandas) מגיע מהמשקלים θ של המודל, כלומר ממה שלמד באימון (מפגש 2); ידע על **הפרויקט שלכם** מגיע רק מ-x.

2. **ההודעה היא רק חלק אחד מחמישה.** במפגש 3 שלטנו ב-x_{prompt}. היום שולטים גם בשלושה נוספים: x_{inst}, x_{hist} ו-x_{files} (את x_{tool} קובע הספק).

3. **ההסתברות P_{succ} תלויה בכל הקלט.** אותה הודעה יכולה להצליח בשיחה אחת ולהיכשל באחרת. לכן במפגש 3 כל הרצה הייתה בשיחה חדשה, כדי שההיסטוריה תהיה ריקה וזהה בכל ההרצות.



Vibe Coding

שיעור 4 - חלק א׳ - מכאן לחלק הבא

<u>מכאן לחלק הבא:</u>

חלק א׳ הראה שכל מה שהמודל יודע על הפרויקט נמצא בקלט של הבקשה. בשיחה חדשה ההיסטוריה ריקה, ואם אין קובץ הנחיה, גם x_{inst} ריק. **בחלק ב׳ נראה מה המחיר של זה** כשעובדים על פרויקט שלם, עם הרבה בקשות.



ד״ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ב׳ - הבעיה: עקביות לאורך בקשות רבות

<u>למה זה חשוב:</u>

בפרויקט לא מספיק שכל תשובה תהיה נכונה: כל התשובות צריכות **להתאים זו לזו**. אותו סגנון שמות, אותו מבנה נתונים, אותן ספריות.

Vibe Coding

שיעור 4 - חלק ב׳ - הבעיה: עקביות לאורך בקשות רבות

למה זה חשוב:

בפרויקט לא מספיק שכל תשובה תהיה נכונה: כל התשובות צריכות **להתאים זו לזו**. אותו סגנון שמות, אותו מבנה נתונים, אותן ספריות.

ההסבר:

שלוש בקשות לאותו פרויקט, **כל אחת בשיחה חדשה.** בבקשה 1 כתבתם: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה".

בקשה 1: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה"

```
def add_task(title: str) -> None:
```

✓

Vibe Coding

שיעור 4 - חלק ב׳ - הבעיה: עקביות לאורך בקשות רבות

<u>למה זה חשוב:</u>

בפרויקט לא מספיק שכל תשובה תהיה נכונה: כל התשובות צריכות **להתאים זו לזו**. אותו סגנון שמות, אותו מבנה נתונים, אותן ספריות.

<u>ההסבר:</u>

שלוש בקשות לאותו פרויקט, **כל אחת בשיחה חדשה.** בבקשה 1 כתבתם: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה".

בבקשה 2 שוב כתבתם את ההקדמה: "שוב בפייתון עם type hints, כתוב פונקציה שמוחקת מטלה". בשתי התשובות יש type hints.

בקשה 1: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה"

```
def add_task(title: str) -> None:
```

✓

בקשה 2: "שוב בפייתון עם type hints, כתוב פונקציה שמוחקת מטלה"

```
def delete_task(index: int) -> None:
```

✓

Vibe Coding

שיעור 4 - חלק ב׳ - הבעיה: עקביות לאורך בקשות רבות

<u>למה זה חשוב:</u>

בפרויקט לא מספיק שכל תשובה תהיה נכונה: כל התשובות צריכות **להתאים זו לזו**. אותו סגנון שמות, אותו מבנה נתונים, אותן ספריות.

<u>ההסבר:</u>

שלוש בקשות לאותו פרויקט, **כל אחת בשיחה חדשה.** בבקשה 1 כתבתם: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה".

בבקשה 2 שוב כתבתם את ההקדמה: "שוב בפייתון עם type hints, כתוב פונקציה שמוחקת מטלה". בשתי התשובות יש type hints.

שאלה: בבקשה 3 כתבתם רק "כתוב פונקציה שמסמנת מטלה כהושלמה", בלי ההקדמה. מה יקרה?

בקשה 1: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה"

```
def add_task(title: str) -> None:
```

✓

בקשה 2: "שוב בפייתון עם type hints, כתוב פונקציה שמוחקת מטלה"

```
def delete_task(index: int) -> None:
```

✓

בקשה 3: "כתוב פונקציה שמסמנת מטלה כהושלמה"

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ב׳ - הבעיה: עקביות לאורך בקשות רבות

בקשה 1: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה"

```
def add_task(title: str) -> None:
```

✓

בקשה 2: "שוב בפייתון עם type hints, כתוב פונקציה שמוחקת מטלה"

```
def delete_task(index: int) -> None:
```

✓

בקשה 3: "כתוב פונקציה שמסמנת מטלה כהושלמה"

```
def done(t):
```

✗

<u>למה זה חשוב:</u>

בפרויקט לא מספיק שכל תשובה תהיה נכונה: כל התשובות צריכות **להתאים זו לזו**. אותו סגנון שמות, אותו מבנה נתונים, אותן ספריות.

<u>ההסבר:</u>

שלוש בקשות לאותו פרויקט, **כל אחת בשיחה חדשה.** בבקשה 1 כתבתם: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה".

בבקשה 2 שוב כתבתם את ההקדמה: "שוב בפייתון עם type hints, כתוב פונקציה שמוחקת מטלה". בשתי התשובות יש type hints.

שאלה: בבקשה 3 כתבתם רק "כתוב פונקציה שמסמנת מטלה כהושלמה", בלי ההקדמה. מה יקרה?

תשובה: המודל חזר לברירת המחדל שלו: **בלי type hints, ושם בסגנון אחר** (done במקום mark_done).

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ב׳ - הבעיה: עקביות לאורך בקשות רבות

בקשה 1: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה"

```
def add_task(title: str) -> None:
```

✓

בקשה 2: "שוב בפייתון עם type hints, כתוב פונקציה שמוחקת מטלה"

```
def delete_task(index: int) -> None:
```

✓

בקשה 3: "כתוב פונקציה שמסמנת מטלה כהושלמה"

```
def done(t):
```

✗

<u>למה זה חשוב:</u>

בפרויקט לא מספיק שכל תשובה תהיה נכונה: כל התשובות צריכות **להתאים זו לזו**. אותו סגנון שמות, אותו מבנה נתונים, אותן ספריות.

<u>ההסבר:</u>

שלוש בקשות לאותו פרויקט, **כל אחת בשיחה חדשה.** בבקשה 1 כתבתם: "בפייתון, עם type hints, כתוב פונקציה שמוסיפה מטלה".

בבקשה 2 שוב כתבתם את ההקדמה: "שוב בפייתון עם type hints, כתוב פונקציה שמוחקת מטלה". בשתי התשובות יש type hints.

שאלה: בבקשה 3 כתבתם רק "כתוב פונקציה שמסמנת מטלה כהושלמה", בלי ההקדמה. מה יקרה?

תשובה: המודל חזר לברירת המחדל שלו: **בלי type hints, ושם בסגנון אחר** (done במקום mark_done).

אין כאן באג במודל. בשיחה החדשה המוסכמה פשוט לא הייתה בחלון.

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי

ניסוח פורמלי - עקביות לאורך Q בקשות:

<u>הגדרות:</u>

- **המוסכמה** ℓ - החלטה אחת שחלה על **כל הקוד בפרויקט**, למשל "type hints בכל פונקציה".

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי

ניסוח פורמלי - עקביות לאורך Q בקשות:

<u>הגדרות:</u>

- **המוסכמה** ℓ - החלטה אחת שחלה על **כל הקוד בפרויקט**, למשל "type hints בכל פונקציה".
- **מספר הבקשות** Q - מספר הבקשות בפרויקט שהמוסכמה רלוונטית להן (כאן: כל בקשה שכותבת פונקציה). כל בקשה נשלחת **בשיחה חדשה**.

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי

ניסוח פורמלי - עקביות לאורך Q בקשות:

<u>הגדרות:</u>

- **המוסכמה** ℓ - החלטה אחת שחלה על **כל הקוד בפרויקט**, למשל "type hints בכל פונקציה".
- **מספר הבקשות** Q - מספר הבקשות בפרויקט שהמוסכמה רלוונטית להן (כאן: כל בקשה שכותבת פונקציה). כל בקשה נשלחת **בשיחה חדשה**.
- **ההסתברות** g_ℓ - ההסתברות שתשובה אחת מקיימת את המוסכמה. במפגש 3 זו הייתה ההסתברות להחלטה ℓ בפרומפט אחד; כאן זו אותה הסתברות, בכל אחת מ-Q הבקשות.

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי

ניסוח פורמלי - עקביות לאורך Q בקשות:

<u>הגדרות:</u>

- **המוסכמה** ℓ - החלטה אחת שחלה על **כל הקוד בפרויקט**, למשל "type hints בכל פונקציה".
- **מספר הבקשות** Q - מספר הבקשות בפרויקט שהמוסכמה רלוונטית להן (כאן: כל בקשה שכותבת פונקציה). כל בקשה נשלחת **בשיחה חדשה**.
- **ההסתברות** g_ℓ - ההסתברות שתשובה אחת מקיימת את המוסכמה. במפגש 3 זו הייתה ההסתברות להחלטה ℓ בפרומפט אחד; כאן זו אותה הסתברות, בכל אחת מ-Q הבקשות.
- **חריגה** - תשובה שלא מקיימת את המוסכמה.

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי

ניסוח פורמלי - עקביות לאורך Q בקשות:

<u>הגדרות:</u>

- **המוסכמה** ℓ - החלטה אחת שחלה על **כל הקוד בפרויקט**, למשל "type hints בכל פונקציה".
- **מספר הבקשות** Q - מספר הבקשות בפרויקט שהמוסכמה רלוונטית להן (כאן: כל בקשה שכותבת פונקציה). כל בקשה נשלחת **בשיחה חדשה**.
- **ההסתברות** g_ℓ - ההסתברות שתשובה אחת מקיימת את המוסכמה. במפגש 3 זו הייתה ההסתברות להחלטה ℓ בפרומפט אחד; כאן זו אותה הסתברות, בכל אחת מ-Q הבקשות.
- **חריגה** - תשובה שלא מקיימת את המוסכמה.

<u>הנחה:</u>

בשיחה חדשה המודל לא רואה את התשובות הקודמות, ולכן כל תשובה היא "הגרלה" נפרדת: התשובות **בלתי-תלויות** זו בזו.

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי: נוסחאות

נוסחאות:

נתונים: מוסכמה ℓ, Q בקשות בלתי-תלויות, והסתברות g_ℓ בכל אחת (ההגדרות מהשקופית הקודמת). ההסתברות שכל Q התשובות מקיימות את המוסכמה:

$$\Pr[\text{all } Q \text{ answers follow convention } \ell] = g_\ell^Q$$

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי: נוסחאות

<u>נוסחאות:</u>

נתונים: מוסכמה ℓ, Q בקשות בלתי-תלויות, והסתברות g_ℓ בכל אחת (ההגדרות מהשקופית הקודמת). ההסתברות שכל Q התשובות מקיימות את המוסכמה:

$$\Pr[\text{all } Q \text{ answers follow convention } \ell] = g_\ell^Q$$

זה מודל המכפלה ממפגש 3: ההסתברות שכמה אירועים בלתי-תלויים קורים כולם היא מכפלת ההסתברויות, כאן Q פעמים g_ℓ.

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי: נוסחאות

<u>נוסחאות:</u>

נתונים: מוסכמה ℓ, Q בקשות בלתי-תלויות, והסתברות g_ℓ בכל אחת (ההגדרות מהשקופית הקודמת). ההסתברות שכל Q התשובות מקיימות את המוסכמה:

$$\Pr[\text{all } Q \text{ answers follow convention } \ell] = g_\ell^Q$$

זה מודל המכפלה ממפגש 3: ההסתברות שכמה אירועים בלתי-תלויים קורים כולם היא מכפלת ההסתברויות, כאן Q פעמים g_ℓ.

תוחלת מספר החריגות (מספר התשובות שלא מקיימות את המוסכמה):

$$\mathbb{E}[\text{number of violations}] = Q\,(1 - g_\ell)$$

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי: נוסחאות

<u>נוסחאות:</u>

נתונים: מוסכמה ℓ, Q בקשות בלתי-תלויות, והסתברות g_ℓ בכל אחת (ההגדרות מהשקופית הקודמת). ההסתברות שכל Q התשובות מקיימות את המוסכמה:

$$\Pr[\text{all } Q \text{ answers follow convention } \ell] = g_\ell^Q$$

זה מודל המכפלה ממפגש 3: ההסתברות שכמה אירועים בלתי-תלויים קורים כולם היא מכפלת ההסתברויות, כאן Q פעמים g_ℓ.

תוחלת מספר החריגות (מספר התשובות שלא מקיימות את המוסכמה):

$$\mathbb{E}[\text{number of violations}] = Q\,(1 - g_\ell)$$

מאיפה הנוסחה הזו? נגזור אותה בשקופית הבאה, מתוך ההתפלגות של מספר החריגות.

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי: מאיפה התוחלת?

<u>איזו התפלגות יש למספר החריגות?</u>

לכל בקשה i נגדיר משתנה מקרי X_i: הוא שווה 1 אם התשובה חורגת מהמוסכמה, ו-0 אם היא מקיימת אותה. הסיכוי שתשובה תחרוג משלים את g_ℓ ל-1, ולכן כל X_i הוא משתנה **ברנולי** עם הסיכוי הזה:

$$X_i \sim \text{Bernoulli}(1 - g_\ell)$$

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי: מאיפה התוחלת?

<u>איזו התפלגות יש למספר החריגות?</u>

לכל בקשה i נגדיר משתנה מקרי X_i: הוא שווה 1 אם התשובה חורגת מהמוסכמה, ו-0 אם היא מקיימת אותה. הסיכוי שתשובה תחרוג משלים את g_ℓ ל-1, ולכן כל X_i הוא משתנה **ברנולי** עם הסיכוי הזה:

$$X_i \sim \text{Bernoulli}(1 - g_\ell)$$

מספר החריגות V הוא סכום כל המשתנים X_i. סכום של Q משתני ברנולי **בלתי-תלויים** עם אותה הסתברות הוא משתנה **בינומי**:

$$V = \sum_{i=1}^{Q} X_i \ \sim \ \text{Bin}(Q,\, 1 - g_\ell)$$

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי: מאיפה התוחלת?

<u>איזו התפלגות יש למספר החריגות?</u>

לכל בקשה i נגדיר משתנה מקרי X_i: הוא שווה 1 אם התשובה חורגת מהמוסכמה, ו-0 אם היא מקיימת אותה. הסיכוי שתשובה תחרוג משלים את g_ℓ ל-1, ולכן כל X_i הוא משתנה **ברנולי** עם הסיכוי הזה:

$$X_i \sim \text{Bernoulli}(1 - g_\ell)$$

מספר החריגות V הוא סכום כל המשתנים X_i. סכום של Q משתני ברנולי **בלתי-תלויים** עם אותה הסתברות הוא משתנה **בינומי**:

$$V = \sum_{i=1}^{Q} X_i \ \sim \ \text{Bin}(Q,\, 1 - g_\ell)$$

התוחלת של X_i אחד היא פשוט ההסתברות לחריגה:

$$\mathbb{E}[X_i] = 1 \cdot (1 - g_\ell) + 0 \cdot g_\ell = 1 - g_\ell$$

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי: מאיפה התוחלת?

<u>איזו התפלגות יש למספר החריגות?</u>

לכל בקשה i נגדיר משתנה מקרי X_i: הוא שווה 1 אם התשובה חורגת מהמוסכמה, ו-0 אם היא מקיימת אותה. הסיכוי שתשובה תחרוג משלים את g_ℓ ל-1, ולכן כל X_i הוא משתנה **ברנולי** עם הסיכוי הזה:

$$X_i \sim \text{Bernoulli}(1 - g_\ell)$$

מספר החריגות V הוא סכום כל המשתנים X_i. סכום של Q משתני ברנולי **בלתי-תלויים** עם אותה הסתברות הוא משתנה **בינומי**:

$$V = \sum_{i=1}^{Q} X_i \ \sim \ \text{Bin}(Q,\, 1 - g_\ell)$$

התוחלת של X_i אחד היא פשוט ההסתברות לחריגה:

$$\mathbb{E}[X_i] = 1 \cdot (1 - g_\ell) + 0 \cdot g_\ell = 1 - g_\ell$$

ותוחלת של סכום היא סכום התוחלות, ולכן:

$$\mathbb{E}[V] = \sum_{i=1}^{Q} \mathbb{E}[X_i] = Q\,(1 - g_\ell)$$

Vibe Coding

שיעור 4 - חלק ב׳ - ניסוח פורמלי: מאיפה התוחלת?

<u>איזו התפלגות יש למספר החריגות?</u>

לכל בקשה i נגדיר משתנה מקרי X_i: הוא שווה 1 אם התשובה חורגת מהמוסכמה, ו-0 אם היא מקיימת אותה. הסיכוי שתשובה תחרוג משלים את g_ℓ ל-1, ולכן כל X_i הוא משתנה **ברנולי** עם הסיכוי הזה:

$$X_i \sim \text{Bernoulli}(1 - g_\ell)$$

מספר החריגות V הוא סכום כל המשתנים X_i. סכום של Q משתני ברנולי **בלתי-תלויים** עם אותה הסתברות הוא משתנה **בינומי**:

$$V = \sum_{i=1}^{Q} X_i \ \sim \ \text{Bin}(Q,\, 1 - g_\ell)$$

התוחלת של X_i אחד היא פשוט ההסתברות לחריגה:

$$\mathbb{E}[X_i] = 1 \cdot (1 - g_\ell) + 0 \cdot g_\ell = 1 - g_\ell$$

ותוחלת של סכום היא סכום התוחלות, ולכן:

$$\mathbb{E}[V] = \sum_{i=1}^{Q} \mathbb{E}[X_i] = Q\,(1 - g_\ell)$$

זו התוחלת הידועה של התפלגות בינומית: מספר הניסיונות כפול הסיכוי להצלחה בכל ניסיון. ובאותו מודל, הסיכוי שאין אף חריגה הוא בדיוק הנוסחה מהשקופית הקודמת:

$$\Pr[V = 0] = g_\ell^{\,Q}$$

Vibe Coding

שיעור 4 - חלק ב׳ - שלוש דרכים לטפל במוסכמה

<u>שלוש דרכים לטפל במוסכמה אחת (ערכי g להמחשה; החישוב אמיתי):</u>

אותה מוסכמה ואותן 10 בקשות. כל דרך קובעת מה ה-g בכל בקשה,
והוא נבנה משני ערכי בסיס:

Vibe Coding

שיעור 4 - חלק ב׳ - שלוש דרכים לטפל במוסכמה

<u>שלוש דרכים לטפל במוסכמה אחת (ערכי g להמחשה; החישוב אמיתי):</u>

אותה מוסכמה ואותן 10 בקשות. כל דרך קובעת מה ה-g בכל בקשה, והוא נבנה משני ערכי בסיס:

- g^{open}**:** ההסתברות לקיים את המוסכמה כשהיא **לא כתובה** בשום מקום בקלט, והמודל מנחש.

Vibe Coding

שיעור 4 - חלק ב׳ - שלוש דרכים לטפל במוסכמה

<u>שלוש דרכים לטפל במוסכמה אחת (ערכי g להמחשה; החישוב אמיתי):</u>

אותה מוסכמה ואותן 10 בקשות. כל דרך קובעת מה ה-g בכל בקשה, והוא נבנה משני ערכי בסיס:

- g^{open}**:** ההסתברות לקיים את המוסכמה כשהיא **לא כתובה** בשום מקום בקלט, והמודל מנחש.
- g^{closed}**:** ההסתברות לקיים אותה כשהיא **כתובה בקלט**, והמודל מציית. כאן $g^{\text{closed}} = 0.95$.

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ב׳ - שלוש דרכים לטפל במוסכמה

<u>שלוש דרכים לטפל במוסכמה אחת (ערכי g להמחשה; החישוב אמיתי):</u>

אותה מוסכמה ואותן 10 בקשות. כל דרך קובעת מה ה-g בכל בקשה, והוא נבנה משני ערכי בסיס:

- g^{open}**:** ההסתברות לקיים את המוסכמה כשהיא **לא כתובה** בשום מקום בקלט, והמודל מנחש.
- g^{closed}**:** ההסתברות לקיים אותה כשהיא **כתובה בקלט**, והמודל מציית. כאן $g^{\text{closed}} = 0.95$.

דרך ראשונה - לא כותבים אותה בכלל: ההחלטה פתוחה בכל בקשה, ולכן $g = g^{\text{open}} = 0.7$. בטבלה משמאל: g^{10} = 0.028, ו-3 חריגות צפויות.

דרך	g	g^{10}	חריגות צפויות
לא כותבים	0.7	0.028	3.0

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ב׳ - שלוש דרכים לטפל במוסכמה

<u>שלוש דרכים לטפל במוסכמה אחת (ערכי g להמחשה; החישוב אמיתי):</u>

אותה מוסכמה ואותן 10 בקשות. כל דרך קובעת מה ה-g בכל בקשה, והוא נבנה משני ערכי בסיס:

- g^{open}**:** ההסתברות לקיים את המוסכמה כשהיא **לא כתובה** בשום מקום בקלט, והמודל מנחש.
- g^{closed}**:** ההסתברות לקיים אותה כשהיא **כתובה בקלט**, והמודל מציית. כאן $g^{\text{closed}} = 0.95$.

דרך ראשונה - לא כותבים אותה בכלל: ההחלטה פתוחה בכל בקשה, ולכן $g = g^{\text{open}} = 0.7$. בטבלה משמאל: g^{10} = 0.028, ו-3 חריגות צפויות.

דרך שנייה - כותבים אותה ידנית בכל פרומפט, אבל לפעמים שוכחים. נסמן ב-ε את ההסתברות לשכוח, כאן $\varepsilon = 0.2$. לפי נוסחת ההסתברות השלמה, g הוא ממוצע של שני המקרים, משוקלל בהסתברויות שלהם:

$$g_\ell = (1-\varepsilon)\, g_\ell^{\text{closed}} + \varepsilon\, g_\ell^{\text{open}}$$
$$= 0.8 \cdot 0.95 + 0.2 \cdot 0.7 = 0.9$$

דרך	g	g^{10}	חריגות צפויות
לא כותבים	0.7	0.028	3.0
בפרומפט, ידנית	0.9	0.349	1.0

Vibe Coding

שיעור 4 - חלק ב׳ - שלוש דרכים: קובץ הנחיה

שאלה: ומה אם כותבים את המוסכמה **פעם אחת, בקובץ הנחיה**:
קובץ שהכלי מכניס לקלט בכל בקשה (חלק ג׳)?

דרך	g	g^{10}	חריגות צפויות
לא כותבים	0.7	0.028	3.0
בפרומפט, ידנית	0.9	0.349	1.0

Vibe Coding

שיעור 4 - חלק ב׳ - שלוש דרכים: קובץ הנחיה

שאלה: ומה אם כותבים את המוסכמה **פעם אחת, בקובץ הנחיה**:
קובץ שהכלי מכניס לקלט בכל בקשה (חלק ג׳)?

תשובה: דרך שלישית: אין שכחה, כלומר $\varepsilon = 0$. לכן g שווה ל-g^{closed}, כלומר 0.95.

דרך	g	g^{10}	חריגות צפויות
לא כותבים	0.7	0.028	3.0
בפרומפט, ידנית	0.9	0.349	1.0
קובץ הנחיה	0.95	0.599	0.5

Vibe Coding

שיעור 4 - חלק ב׳ - שלוש דרכים: קובץ הנחיה

שאלה: ומה אם כותבים את המוסכמה **פעם אחת, בקובץ הנחיה**: קובץ שהכלי מכניס לקלט בכל בקשה (חלק ג׳)?

תשובה: דרך שלישית: אין שכחה, כלומר $\varepsilon = 0$. לכן g שווה ל-g^{closed}, כלומר 0.95.

בטבלה משמאל, לכל דרך: ה-g בכל בקשה, ההסתברות g^{10} שכל 10 התשובות עקביות, ותוחלת מספר החריגות. שתי העמודות האחרונות מחושבות בנוסחאות מהניסוח הפורמלי.

דרך	g	g^{10}	חריגות צפויות
לא כותבים	0.7	0.028	3.0
בפרומפט, ידנית	0.9	0.349	1.0
קובץ הנחיה	0.95	0.599	0.5

Vibe Coding

שיעור 4 - חלק ב׳ - שלוש דרכים: קובץ הנחיה

שאלה: ומה אם כותבים את המוסכמה **פעם אחת, בקובץ הנחיה**: קובץ שהכלי מכניס לקלט בכל בקשה (חלק ג׳)?

תשובה: דרך שלישית: אין שכחה, כלומר $\varepsilon = 0$. לכן g שווה ל-g^{closed}, כלומר 0.95.

בטבלה משמאל, לכל דרך: ה-g בכל בקשה, ההסתברות g^{10} שכל 10 התשובות עקביות, ותוחלת מספר החריגות. שתי העמודות האחרונות מחושבות בנוסחאות מהניסוח הפורמלי.

דרך	g	g^{10}	חריגות צפויות
לא כותבים	0.7	0.028	3.0
בפרומפט, ידנית	0.9	0.349	1.0
קובץ הנחיה	0.95	0.599	0.5

המעבר מ"לא כותבים" לקובץ הנחיה מעלה את הסיכוי לקוד עקבי **פי 20 בערך** (בערך 3% לעומת 60%), ומוריד את החריגות הצפויות **מ-3 לחצי.**

Vibe Coding

שיעור 4 - חלק ב׳ - כמה מוסכמות

<u>כמה מוסכמות בלתי-תלויות:</u>

בפרויקט אמיתי יש יותר ממוסכמה אחת. נסמן ב-D את מספר המוסכמות, ונניח שהן בלתי-תלויות: עמידה באחת לא משנה את הסיכוי לעמוד באחרת.

Vibe Coding

שיעור 4 - חלק ב׳ - כמה מוסכמות

<u>כמה מוסכמות בלתי-תלויות:</u>

בפרויקט אמיתי יש יותר ממוסכמה אחת. נסמן ב-D את מספר המוסכמות, ונניח שהן בלתי-תלויות: עמידה באחת לא משנה את הסיכוי לעמוד באחרת.

ההסתברות שכל Q התשובות מקיימות את **כל** D המוסכמות היא מכפלה כפולה, על המוסכמות ועל הבקשות:

$$\Pr[\text{all } Q \text{ answers follow all } D \text{ conventions}] = \prod_{\ell=1}^{D} g_\ell^{\,Q} = \left(\prod_{\ell=1}^{D} g_\ell\right)^{Q}$$

Vibe Coding

שיעור 4 - חלק ב׳ - כמה מוסכמות

<u>כמה מוסכמות בלתי-תלויות:</u>

בפרויקט אמיתי יש יותר ממוסכמה אחת. נסמן ב-D את מספר המוסכמות, ונניח שהן בלתי-תלויות: עמידה באחת לא משנה את הסיכוי לעמוד באחרת.

ההסתברות שכל Q התשובות מקיימות את **כל** D המוסכמות היא מכפלה כפולה, על המוסכמות ועל הבקשות:

$$\Pr[\text{all } Q \text{ answers follow all } D \text{ conventions}] = \prod_{\ell=1}^{D} g_\ell^{\,Q} = \left(\prod_{\ell=1}^{D} g_\ell\right)^{Q}$$

<u>דוגמה:</u>

שתי מוסכמות: type hints ($g^{\text{open}} = 0.7$) ו-docstrings ($g^{\text{open}} = 0.8$). עם קובץ הנחיה, $g^{\text{closed}} = 0.95$ לשתיהן. בפרויקט 10 בקשות:

$$\text{without file:} \quad \left(\prod_{\ell=1}^{2} g_\ell^{\text{open}}\right)^{Q} = (0.7 \cdot 0.8)^{10} \approx 0.003$$

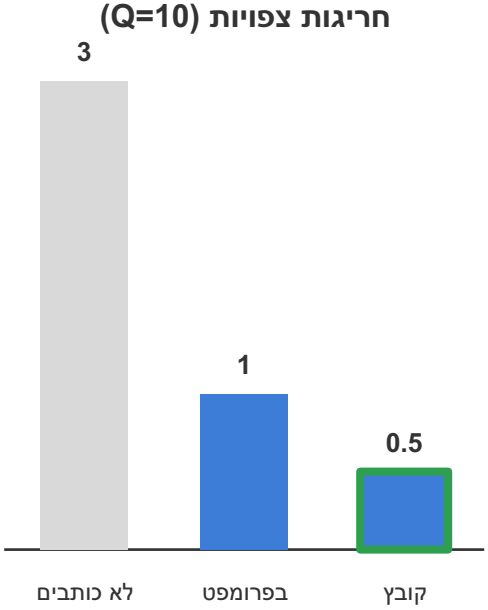
$$\text{with file:} \quad \left(\prod_{\ell=1}^{2} g_\ell^{\text{closed}}\right)^{Q} = (0.95 \cdot 0.95)^{10} \approx 0.358$$

Vibe Coding

שיעור 4 - חלק ב׳ - מסקנות והסתייגות

<u>מסקנות:</u>

1. **עבור בקשה בודדת המוסכמה נשמרת לרוב, אבל בפרויקט שלם כמעט בטוח שתהיה חריגה.** סיכוי של 0.7 לקיים את המוסכמה בכל בקשה נשמע סביר. אבל אחרי 10 בקשות, הסיכוי שכל הקוד עקבי הוא 3% בלבד. כלומר, בסיכוי של 97% לפחות פונקציה אחת תחרוג. ועם שתי מוסכמות, הסיכוי לקוד עקבי הוא 0.3% בלבד.



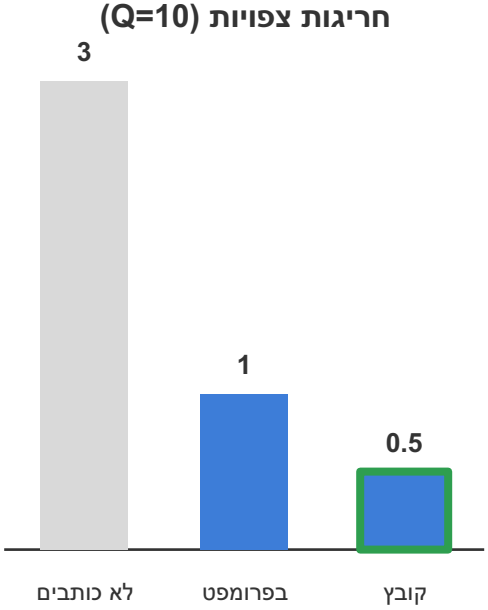
ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ב׳ - מסקנות והסתייגות

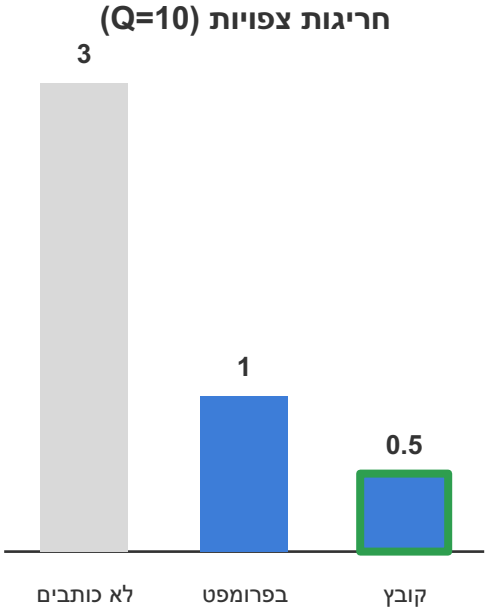
<u>מסקנות:</u>

1. **עבור בקשה בודדת המוסכמה נשמרת לרוב, אבל בפרויקט שלם כמעט בטוח שתהיה חריגה.** סיכוי של 0.7 לקיים את המוסכמה בכל בקשה נשמע סביר. אבל אחרי 10 בקשות, הסיכוי שכל הקוד עקבי הוא 3% בלבד. כלומר, בסיכוי של 97% לפחות פונקציה אחת תחרוג. ועם שתי מוסכמות, הסיכוי לקוד עקבי הוא 0.3% בלבד.
2. **קובץ הנחיה מבטל את ε,** ההסתברות לשכוח לכתוב את המוסכמה. כתיבה ידנית בכל פרומפט תלויה בזיכרון שלכם; קובץ שהכלי טוען אוטומטית לא.



Vibe Coding

שיעור 4 - חלק ב׳ - מסקנות והסתייגות

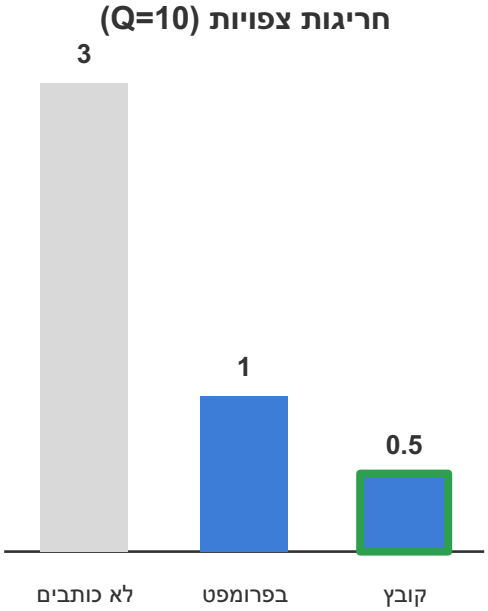


<u>מסקנות:</u>

1. **עבור בקשה בודדת המוסכמה נשמרת לרוב, אבל בפרויקט שלם כמעט בטוח שתהיה חריגה.** סיכוי של 0.7 לקיים את המוסכמה בכל בקשה נשמע סביר. אבל אחרי 10 בקשות, הסיכוי שכל הקוד עקבי הוא 3% בלבד. כלומר, בסיכוי של 97% לפחות פונקציה אחת תחרוג. ועם שתי מוסכמות, הסיכוי לקוד עקבי הוא 0.3% בלבד.
2. **קובץ הנחיה מבטל את ε,** ההסתברות לשכוח לכתוב את המוסכמה. כתיבה ידנית בכל פרומפט תלויה בזיכרון שלכם; קובץ שהכלי טוען אוטומטית לא.
3. **גם עם קובץ, המודל לא מציית תמיד** (g^{closed} קטן מאחד). עם $g^{\text{closed}} = 0.95$ ו-10 בקשות, הסיכוי שכל התשובות עקביות הוא בערך 0.6, ולכן בסיכוי של 40% בערך תהיה לפחות חריגה אחת. הקובץ מקטין את מספר החריגות, אבל לא פוטר מבדיקה (טסטים, מפגש 8).

Vibe Coding

שיעור 4 - חלק ב׳ - מסקנות והסתייגות



<u>מסקנות:</u>

1. **עבור בקשה בודדת המוסכמה נשמרת לרוב, אבל בפרויקט שלם כמעט בטוח שתהיה חריגה.** סיכוי של 0.7 לקיים את המוסכמה בכל בקשה נשמע סביר. אבל אחרי 10 בקשות, הסיכוי שכל הקוד עקבי הוא 3% בלבד. כלומר, בסיכוי של 97% לפחות פונקציה אחת תחרוג. ועם שתי מוסכמות, הסיכוי לקוד עקבי הוא 0.3% בלבד.
2. **קובץ הנחיה מבטל את ε,** ההסתברות לשכוח לכתוב את המוסכמה. כתיבה ידנית בכל פרומפט תלויה בזיכרון שלכם; קובץ שהכלי טוען אוטומטית לא.
3. **גם עם קובץ, המודל לא מציית תמיד** (g^{closed} קטן מאחד). עם $g^{\text{closed}} = 0.95$ ו-10 בקשות, הסיכוי שכל התשובות עקביות הוא בערך 0.6, ולכן בסיכוי של 40% בערך תהיה לפחות חריגה אחת. הקובץ מקטין את מספר החריגות, אבל לא פוטר מבדיקה (טסטים, מפגש 8).

<u>הסתייגות:</u>

בתוך שיחה אחת הבקשות **לא** בלתי-תלויות: המודל רואה את התשובות הקודמות וממשיך את הסגנון שלהן (few-shot מתוך ההיסטוריה). ההנחה מתאימה לשיחות חדשות, ושם בדיוק הבעיה חמורה.

Vibe Coding

שיעור 4 - חלק ב׳ - מכאן לחלק הבא

<u>מכאן לחלק הבא:</u>

המסקנה מחלק ב׳: מוסכמות צריכות **מקום קבוע**, שנכנס לקלט של כל בקשה בלי שתצטרכו לזכור. בחלק ג׳ נבנה את המקום הזה: **קובץ הנחיית פרויקט.**

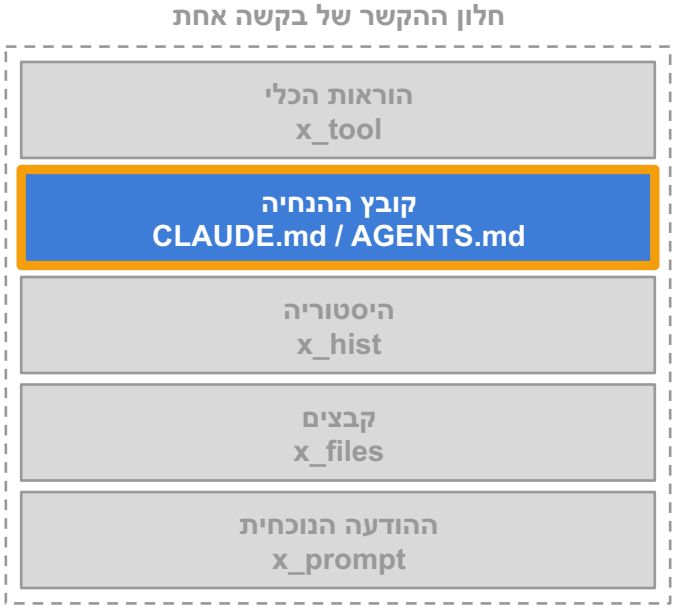


Vibe Coding

שיעור 4 - חלק ג׳ - קובץ הנחיית פרויקט

<u>למה זה חשוב:</u>

זה הכלי המרכזי של המפגש: מקום אחד וקבוע, שבו כותבים **פעם אחת** את מה שחל על כל הבקשות. בפרויקט הגמר תעבדו עם עשרות בקשות וכמה חברי צוות, והקובץ גורם לכל השיחות "לדבר באותה שפה".



ד"ר ולד לנדא

Vibe Coding

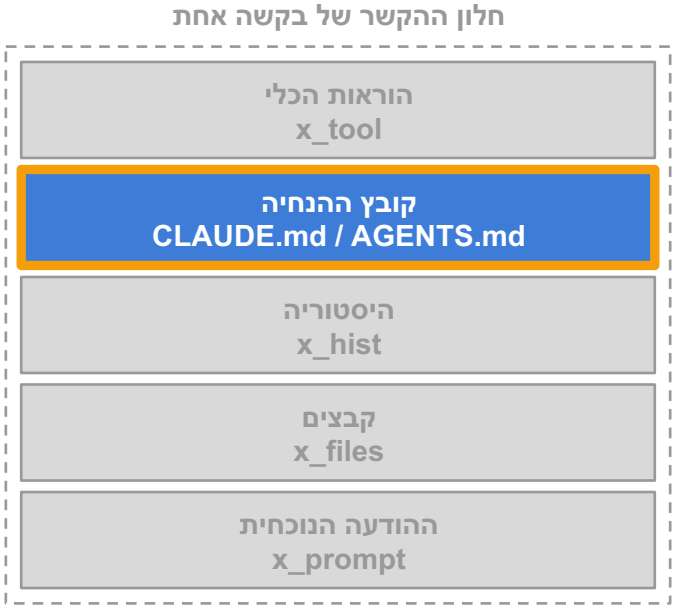
שיעור 4 - חלק ג׳ - קובץ הנחיית פרויקט

למה זה חשוב:

זה הכלי המרכזי של המפגש: מקום אחד וקבוע, שבו כותבים **פעם אחת** את מה שחל על כל הבקשות. בפרויקט הגמר תעבדו עם עשרות בקשות וכמה חברי צוות, והקובץ גורם לכל השיחות "לדבר באותה שפה".

ההסבר:

קובץ הנחיית פרויקט הוא קובץ טקסט בשפה טבעית (בדרך כלל Markdown), שהכלי מכניס לחלון ההקשר **בתחילת כל שיחה**.
זהו x_{inst} מחלק א׳ (השכבה המודגשת באיור): הוא סוגר החלטות ברמת הפרויקט, פעם אחת, לכל הבקשות.



Vibe Coding

שיעור 4 - חלק ג׳ - ששת המרכיבים

1. **סקירה:** מה האפליקציה, למי ובאיזה קנה מידה. בלי "CLI למשתמש בודד", המודל עלול להניח אתר עם התחברות ובסיס נתונים.

```
# CLAUDE.md

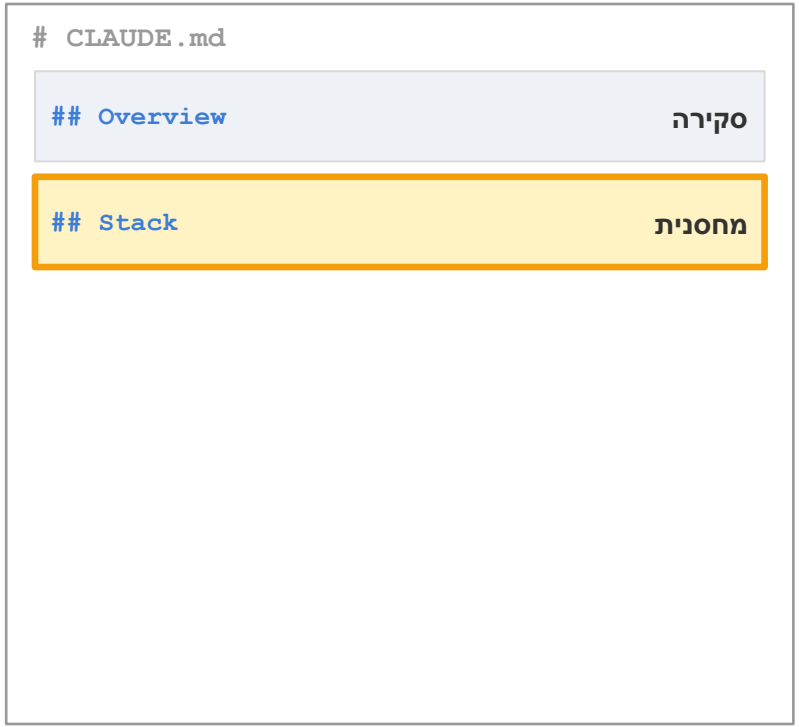
## Overview                    סקירה
```

Vibe Coding

שיעור 4 - חלק ג׳ - ששת המרכיבים

1. **סקירה:** מה האפליקציה, למי ובאיזה קנה מידה. בלי "CLI למשתמש בודד", המודל עלול להניח אתר עם התחברות ובסיס נתונים.

2. **מחסנית (Stack):** שפה, גרסה וספריות מותרות. הגרסה קובעת איזה תחביר זמין (match נוסף ב-Python 3.10).



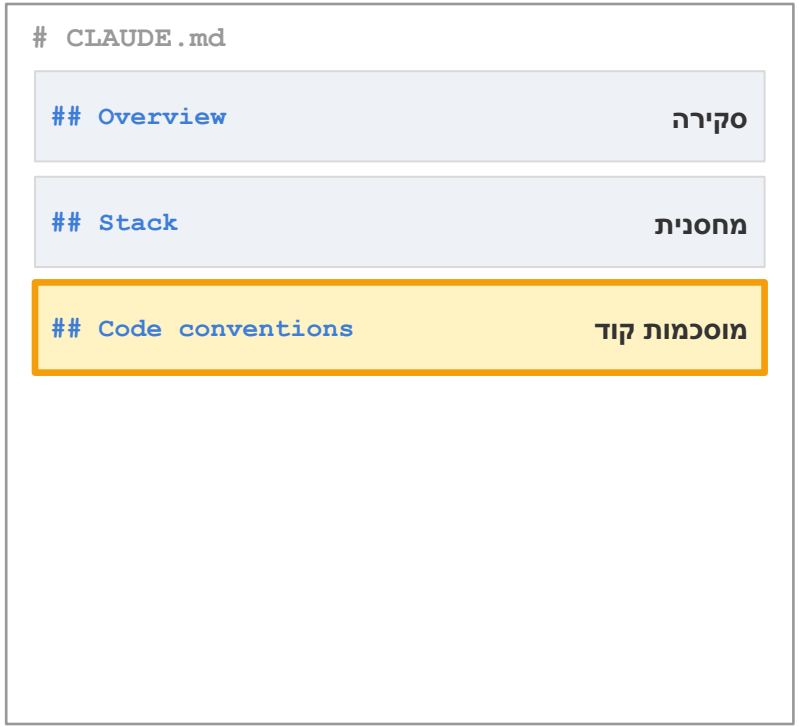
Vibe Coding

שיעור 4 - חלק ג׳ - ששת המרכיבים

1. **סקירה:** מה האפליקציה, למי ובאיזה קנה מידה. בלי "CLI למשתמש בודד", המודל עלול להניח אתר עם התחברות ובסיס נתונים.

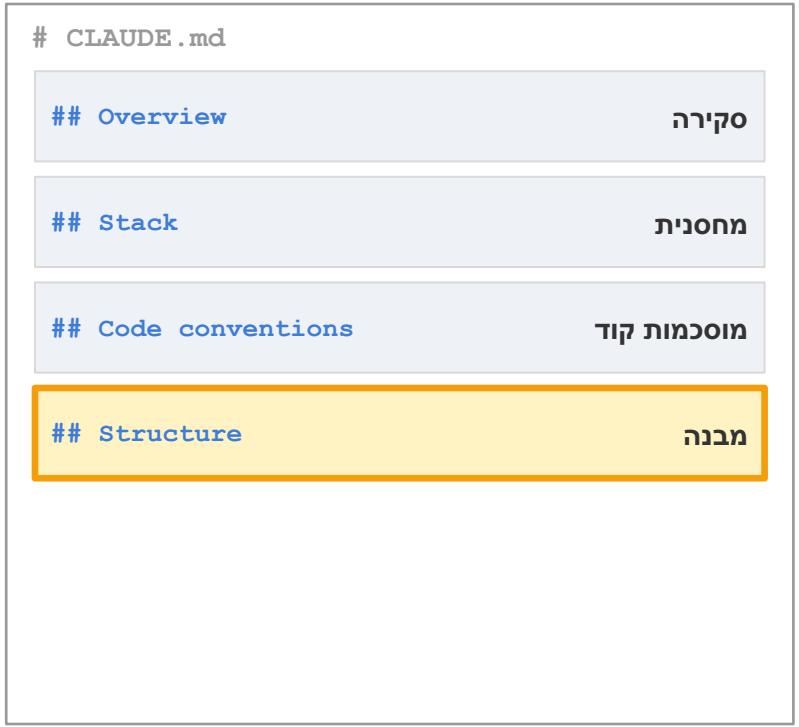
2. **מחסנית (Stack):** שפה, גרסה וספריות מותרות. הגרסה קובעת איזה תחביר זמין (match נוסף ב-Python 3.10).

3. **מוסכמות קוד:** שמות, טיפוסים, תיעוד ושגיאות. דוגמת ניגוד קצרה ("add_task, לא task_add") היא few-shot בתוך כלל.



Vibe Coding

שיעור 4 - חלק ג׳ - ששת המרכיבים



1. **סקירה:** מה האפליקציה, למי ובאיזה קנה מידה. בלי "CLI למשתמש בודד", המודל עלול להניח אתר עם התחברות ובסיס נתונים.

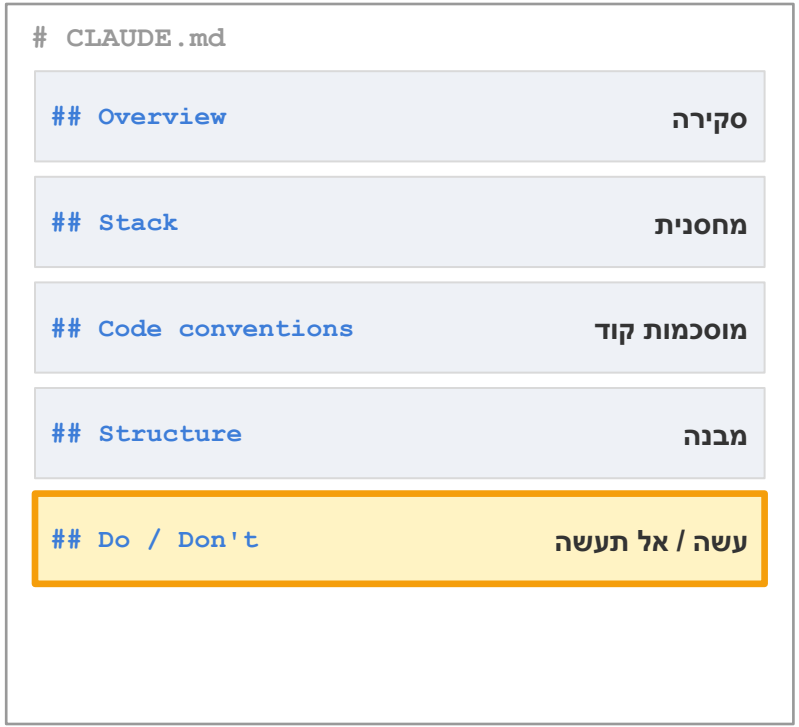
2. **מחסנית (Stack):** שפה, גרסה וספריות מותרות. הגרסה קובעת איזה תחביר זמין (match נוסף ב-Python 3.10).

3. **מוסכמות קוד:** שמות, טיפוסים, תיעוד ושגיאות. דוגמת ניגוד קצרה ("add_task, לא task_add") היא few-shot בתוך כלל.

4. **מבנה:** איפה כל דבר נמצא, ולאן מוסיפים קוד חדש. "הוסף מחיקה" הולך ל-tasks.py, והטסט ל-tests/.

Vibe Coding

שיעור 4 - חלק ג׳ - ששת המרכיבים



1. **סקירה:** מה האפליקציה, למי ובאיזה קנה מידה. בלי "CLI למשתמש בודד", המודל עלול להניח אתר עם התחברות ובסיס נתונים.

2. **מחסנית (Stack):** שפה, גרסה וספריות מותרות. הגרסה קובעת איזה תחביר זמין (match נוסף ב-Python 3.10).

3. **מוסכמות קוד:** שמות, טיפוסים, תיעוד ושגיאות. דוגמת ניגוד קצרה ("add_task, לא task_add") היא few-shot בתוך כלל.

4. **מבנה:** איפה כל דבר נמצא, ולאן מוסיפים קוד חדש. "הוסף מחיקה" הולך ל-tasks.py, והטסט ל-tests/.

5. **עשה / אל תעשה:** כללים שאי אפשר להסיק מההקשר, כמו "אל תוסיף תלויות בלי לשאול".

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ג׳ - ששת המרכיבים

1. **סקירה:** מה האפליקציה, למי ובאיזה קנה מידה. בלי "CLI למשתמש בודד", המודל עלול להניח אתר עם התחברות ובסיס נתונים.

2. **מחסנית (Stack):** שפה, גרסה וספריות מותרות. הגרסה קובעת איזה תחביר זמין (match נוסף ב-Python 3.10).

3. **מוסכמות קוד:** שמות, טיפוסים, תיעוד ושגיאות. דוגמת ניגוד קצרה ("add_task, לא task_add") היא few-shot בתוך כלל.

4. **מבנה:** איפה כל דבר נמצא, ולאן מוסיפים קוד חדש. "הוסף מחיקה" הולך ל-tasks.py, והטסט ל-tests/.

5. **עשה / אל תעשה:** כללים שאי אפשר להסיק מההקשר, כמו "אל תוסיף תלויות בלי לשאול".

6. **בדיקות:** איך מוודאים שהקוד עובד. בכלי agent, "הרץ את כל הטסטים לפני שמסיימים" גורם לו לבדוק את עצמו.

```
# CLAUDE.md

## Overview          סקירה

## Stack             מחסנית

## Code conventions  מוסכמות קוד

## Structure         מבנה

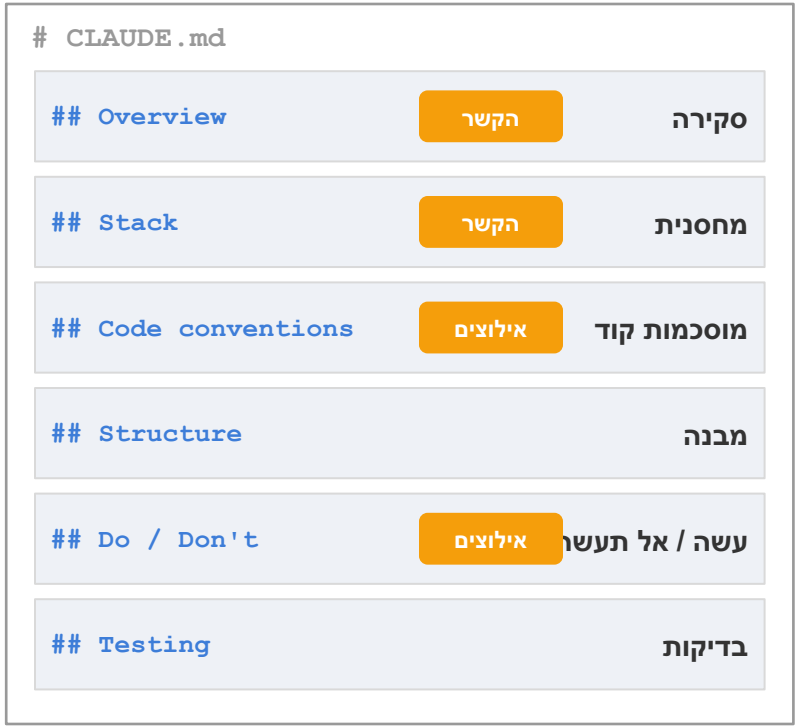
## Do / Don't        עשה / אל תעשה

## Testing           בדיקות
```

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ג׳ - ששת המרכיבים



1. **סקירה:** מה האפליקציה, למי ובאיזה קנה מידה. בלי "CLI למשתמש בודד", המודל עלול להניח אתר עם התחברות ובסיס נתונים.

2. **מחסנית (Stack):** שפה, גרסה וספריות מותרות. הגרסה קובעת איזה תחביר זמין (match נוסף ב-Python 3.10).
3. **מוסכמות קוד:** שמות, טיפוסים, תיעוד ושגיאות. דוגמת ניגוד קצרה ("add_task, לא task_add") היא few-shot בתוך כלל.

4. **מבנה:** איפה כל דבר נמצא, ולאן מוסיפים קוד חדש. "הוסף מחיקה" הולך ל-tasks.py, והטסט ל-tests/.
5. **עשה / אל תעשה:** כללים שאי אפשר להסיק מההקשר, כמו "אל תוסיף תלויות בלי לשאול".
6. **בדיקות:** איך מוודאים שהקוד עובד. בכלי agent, "הרץ את כל הטסטים לפני שמסיימים" גורם לו לבדוק את עצמו.

אלה אבני היסוד ממפגש 3, ברמת הפרויקט: סקירה ומחסנית הן **הקשר**; מוסכמות ועשה/אל תעשה הן **אילוצים** (התגיות באיור). בפרויקט קטן לא חייבים את כל השישה.

Vibe Coding

שיעור 4 - חלק ג׳ - דוגמה: קובץ מלא

דוגמה - הקובץ של פרויקט ה-Todo:

משמאל קובץ הנחיה מלא לאפליקציית ה-Todo, אפליקציית שורת פקודה בפייתון לניהול רשימת מטלות. כל אחד מששת המרכיבים הוא כותרת ## בקובץ (מקוצר מעט לשקף).

```
# Project: Todo App
## Overview
CLI app for personal tasks. Single user,
no network, no external database.
## Stack
Python 3.11. Standard library only
(json, pathlib, argparse are allowed).
## Code conventions
- Full type hints on every function.
- snake_case, verb first (add_task, not task_add).
- Every public function: one-line docstring.
## Structure
- tasks.py: core logic
- storage.py: save/load tasks as a JSON file
- dates.py: date parsing and formatting
- main.py: entry point and CLI
- tests/: one test file per module
## Do / Don't
- Do: handle an empty list and invalid input.
- Don't: add dependencies without asking first.
- Don't: change the save-file format
  without updating this file.
## Testing
Every new function gets a test in tests/.
Run all tests (pytest) before finishing.
```

Vibe Coding

שיעור 4 - חלק ג׳ - דוגמה: קובץ מלא

```
# Project: Todo App
## Overview
CLI app for personal tasks. Single user,
no network, no external database.
## Stack
Python 3.11. Standard library only
(json, pathlib, argparse are allowed).
## Code conventions
- Full type hints on every function.
- snake_case, verb first (add_task, not task_add).
- Every public function: one-line docstring.
## Structure
- tasks.py: core logic
- storage.py: save/load tasks as a JSON file
- dates.py: date parsing and formatting
- main.py: entry point and CLI
- tests/: one test file per module
## Do / Don't
- Do: handle an empty list and invalid input.
- Don't: add dependencies without asking first.
- Don't: change the save-file format
  without updating this file.
## Testing
Every new function gets a test in tests/.
Run all tests (pytest) before finishing.
```

<u>דוגמה - הקובץ של פרויקט ה-Todo:</u>

משמאל קובץ הנחיה מלא לאפליקציית ה-Todo, אפליקציית שורת פקודה בפייתון לניהול רשימת מטלות. כל אחד משש המרכיבים הוא כותרת ## בקובץ (מקוצר מעט לשקף).

שאלה: כמה טוקנים עולה הקובץ הזה, בכל בקשה?

Vibe Coding

שיעור 4 - חלק ג׳ - דוגמה: קובץ מלא

```
# Project: Todo App
## Overview
CLI app for personal tasks. Single user,
no network, no external database.
## Stack
Python 3.11. Standard library only
(json, pathlib, argparse are allowed).
## Code conventions
- Full type hints on every function.
- snake_case, verb first (add_task, not task_add).
- Every public function: one-line docstring.
## Structure
- tasks.py: core logic
- storage.py: save/load tasks as a JSON file
- dates.py: date parsing and formatting
- main.py: entry point and CLI
- tests/: one test file per module
## Do / Don't
- Do: handle an empty list and invalid input.
- Don't: add dependencies without asking first.
- Don't: change the save-file format
  without updating this file.
## Testing
Every new function gets a test in tests/.
Run all tests (pytest) before finishing.
```

<u>דוגמה - הקובץ של פרויקט ה-Todo:</u>

משמאל קובץ הנחיה מלא לאפליקציית ה-Todo, אפליקציית שורת פקודה בפייתון לניהול רשימת מטלות. כל אחד מששת המרכיבים הוא כותרת ## בקובץ (מקוצר מעט לשקף).

שאלה: כמה טוקנים עולה הקובץ הזה, בכל בקשה?

תשובה: נמדד בטוקנייזר o200k_base, כמו במפגש 2: 157 מילים ו-226 טוקנים. אותו קובץ בעברית: 348 טוקנים, פי 1.54.

Vibe Coding

שיעור 4 - חלק ג׳ - דוגמה: קובץ מלא

```
# Project: Todo App
## Overview
CLI app for personal tasks. Single user,
no network, no external database.
## Stack
Python 3.11. Standard library only
(json, pathlib, argparse are allowed).
## Code conventions
- Full type hints on every function.
- snake_case, verb first (add_task, not task_add).
- Every public function: one-line docstring.
## Structure
- tasks.py: core logic
- storage.py: save/load tasks as a JSON file
- dates.py: date parsing and formatting
- main.py: entry point and CLI
- tests/: one test file per module
## Do / Don't
- Do: handle an empty list and invalid input.
- Don't: add dependencies without asking first.
- Don't: change the save-file format
  without updating this file.
## Testing
Every new function gets a test in tests/.
Run all tests (pytest) before finishing.
```

<u>דוגמה - הקובץ של פרויקט ה-Todo:</u>

משמאל קובץ הנחיה מלא לאפליקציית ה-Todo, אפליקציית שורת פקודה בפייתון לניהול רשימת מטלות. כל אחד מששת המרכיבים הוא כותרת ## בקובץ (מקוצר מעט לשקף).

שאלה: כמה טוקנים עולה הקובץ הזה, בכל בקשה?

תשובה: נמדד בטוקנייזר o200k_base, כמו במפגש 2: 157 מילים ו-226 טוקנים. אותו קובץ בעברית: 348 טוקנים, פי 1.54.

היחס קטן מזה של משפט עברי טהור (פי 2.14, מפגש 2), כי חלק גדול מהקובץ הוא שמות קבצים ופקודות באנגלית. לכן, כמו הקוד עצמו, **כותבים את הקובץ באנגלית.**

Vibe Coding

שיעור 4 - חלק ג׳ - דוגמה: קובץ מלא

```
# Project: Todo App
## Overview
CLI app for personal tasks. Single user,
no network, no external database.
## Stack
Python 3.11. Standard library only
(json, pathlib, argparse are allowed).
## Code conventions
- Full type hints on every function.
- snake_case, verb first (add_task, not task_add).
- Every public function: one-line docstring.
## Structure
- tasks.py: core logic
- storage.py: save/load tasks as a JSON file
- dates.py: date parsing and formatting
- main.py: entry point and CLI
- tests/: one test file per module
## Do / Don't
- Do: handle an empty list and invalid input.
- Don't: add dependencies without asking first.
- Don't: change the save-file format
  without updating this file.
## Testing
Every new function gets a test in tests/.
Run all tests (pytest) before finishing.
```

<u>דוגמה - הקובץ של פרויקט ה-Todo:</u>

משמאל קובץ הנחיה מלא לאפליקציית ה-Todo, אפליקציית שורת פקודה בפייתון לניהול רשימת מטלות. כל אחד משש המרכיבים הוא כותרת ## בקובץ (מקוצר מעט לשקף).

שאלה: כמה טוקנים עולה הקובץ הזה, בכל בקשה?

תשובה: נמדד בטוקנייזר o200k_base, כמו במפגש 2: 157 מילים ו-226 טוקנים. אותו קובץ בעברית: 348 טוקנים, פי 1.54.

היחס קטן מזה של משפט עברי טהור (פי 2.14, מפגש 2), כי חלק גדול מהקובץ הוא שמות קבצים ופקודות באנגלית. לכן, כמו הקוד עצמו, **כותבים את הקובץ באנגלית.**

שימו לב לכלל ה"מטא" בסוף הקובץ (מודגש): **אל תשנו את פורמט השמירה בלי לעדכן את הקובץ הזה.** נחזור אליו בחלק ה׳.

Vibe Coding

שיעור 4 - חלק ג׳ - Before / After: אותו פרומפט

Before / After:

הפרומפט בשתי השיחות: **"הוסף פונקציה שמוחקת מטלה לפי אינדקס"**. משמאל למעלה: הפלט בלי קובץ הנחיה (פלט מייצג).

בלי קובץ הנחיה (פלט מייצג)

```
def delete(i):
    del tasks[i]
```

Vibe Coding

שיעור 4 - חלק ג׳ - Before / After: אותו פרומפט

Before / After:

הפרומפט בשתי השיחות: **"הוסף פונקציה שמוחקת מטלה לפי אינדקס"**. משמאל למעלה: הפלט בלי קובץ הנחיה (פלט מייצג).

שאלה: מה לא בסדר בגרסה הזו, מעבר לסגנון?

בלי קובץ הנחיה (פלט מייצג)

```
def delete(i):
    del tasks[i]
```

Vibe Coding

שיעור 4 - חלק ג׳ - Before / After: אותו פרומפט

Before / After:

הפרומפט בשתי השיחות: **"הוסף פונקציה שמוחקת מטלה לפי אינדקס"**. משמאל למעלה: הפלט בלי קובץ הנחיה (פלט מייצג).

שאלה: מה לא בסדר בגרסה הזו, מעבר לסגנון?

תשובה: יש בה באג: המשתנה tasks גלובלי ולא מוגדר. **המודל ניחש מבנה נתונים שלא קיים.**

בלי קובץ הנחיה (פלט מייצג)

```
def delete(i):
    del tasks[i]
```

✗

Vibe Coding

שיעור 4 - חלק ג׳ - Before / After: אותו פרומפט

Before / After:

הפרומפט בשתי השיחות: **"הוסף פונקציה שמוחקת מטלה לפי אינדקס"**. משמאל למעלה: הפלט בלי קובץ הנחיה (פלט מייצג).

שאלה: מה לא בסדר בגרסה הזו, מעבר לסגנון?

תשובה: יש בה באג: המשתנה tasks גלובלי ולא מוגדר. **המודל ניחש מבנה נתונים שלא קיים.**

משמאל למטה: אותו פרומפט בדיוק, בשיחה **עם** קובץ ההנחיה (פלט מייצג).

בלי קובץ הנחיה (פלט מייצג)

```
def delete(i):
    del tasks[i]
```

✗

עם קובץ ההנחיה (פלט מייצג)

```
def delete_task(tasks: list[dict], index: int) -> None:
    """Delete the task at the given index."""
    if not 0 <= index < len(tasks):
        raise IndexError(f"no task at index {index}")
    del tasks[index]
```

Vibe Coding

שיעור 4 - חלק ג׳ - Before / After: אותו פרומפט

Before / After:

הפרומפט בשתי השיחות: **"הוסף פונקציה שמוחקת מטלה לפי אינדקס"**. משמאל למעלה: הפלט בלי קובץ הנחיה (פלט מייצג).

שאלה: מה לא בסדר בגרסה הזו, מעבר לסגנון?

תשובה: יש בה באג: המשתנה tasks גלובלי ולא מוגדר. **המודל ניחש מבנה נתונים שלא קיים.**

משמאל למטה: אותו פרומפט בדיוק, בשיחה **עם** קובץ ההנחיה (פלט מייצג).

הקובץ סגר **ארבע החלטות**, ואף אחת מהן לא הופיעה בפרומפט: שם (פועל בהתחלה), type hints, docstring, וטיפול בקלט לא תקין.

בלי קובץ הנחיה (פלט מייצג)

```python
def delete(i):
    del tasks[i]
```

✗

עם קובץ ההנחיה (פלט מייצג)

```python
def delete_task(tasks: list[dict], index: int) -> None:
    """Delete the task at the given index."""
    if not 0 <= index < len(tasks):
        raise IndexError(f"no task at index {index}")
    del tasks[index]
```

שם + type hints **docstring** **טיפול בקלט לא תקין**

Vibe Coding

שיעור 4 - חלק ג׳ - מכאן לחלק הבא

<u>מכאן לחלק הבא:</u>

עכשיו יש לנו קובץ, ונשארה השאלה **מה לכתוב בו.** כל כלל נוסף תופס טוקנים בכל בקשה, ואפשר להמשיך להוסיף כללים בלי סוף. בחלק ד׳ נחשב **כמה כלל אחד שווה**, וכך נחליט אילו כללים נכנסים לקובץ.



ד"ר ולד לנדא

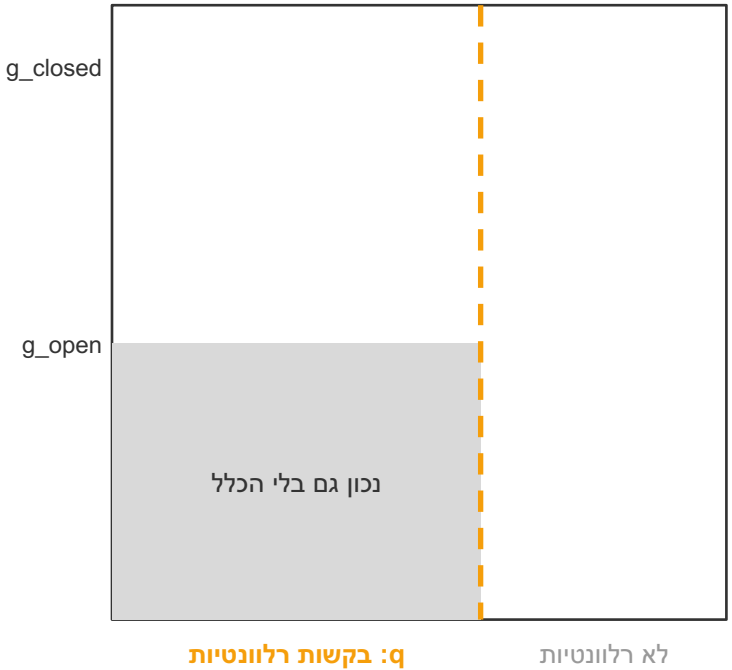
Vibe Coding

שיעור 4 - חלק ד׳ - ניסוח פורמלי: הרווח הצפוי מכלל

ניסוח פורמלי - הרווח הצפוי מכלל:

<u>הגדרות:</u>

כלל הוא שורה בקובץ ההנחיה שסוגרת החלטה ℓ. ההחלטה רלוונטית רק לחלק מהבקשות: נסמן ב-q_ℓ את שבר הבקשות שבהן היא רלוונטית (למשל, "argparse לשורת הפקודה" רלוונטי רק לבקשות שנוגעות בשורת הפקודה).



Vibe Coding

שיעור 4 - חלק ד׳ - ניסוח פורמלי: הרווח הצפוי מכלל

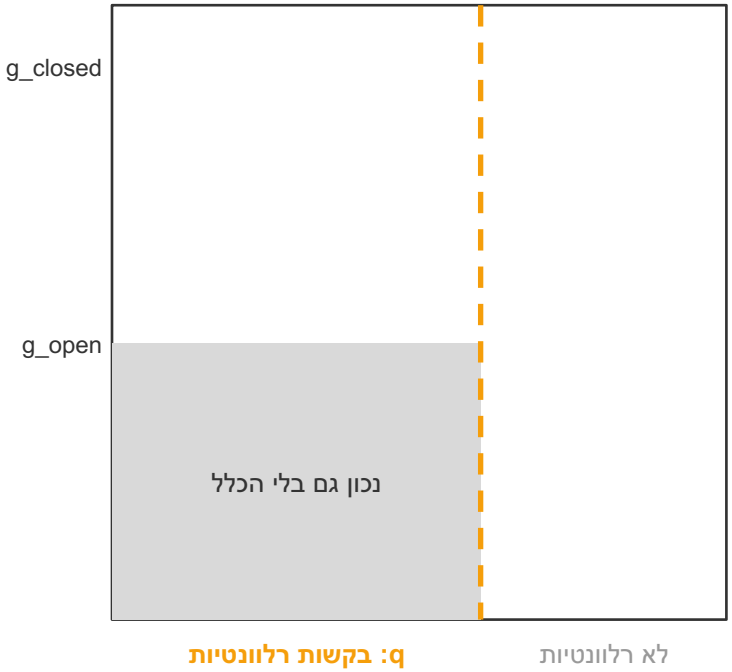
ניסוח פורמלי - הרווח הצפוי מכלל:

<u>הגדרות:</u>

כלל הוא שורה בקובץ ההנחיה שסוגרת החלטה ℓ. ההחלטה רלוונטית רק לחלק מהבקשות: נסמן ב-q_ℓ את שבר הבקשות שבהן היא רלוונטית (למשל, "argparse לשורת הפקודה" רלוונטי רק לבקשות שנוגעות בשורת הפקודה).

עבור בקשה רלוונטית: מספר החריגות הצפוי בלי הכלל ועם הכלל הוא התוחלת מחלק ב׳, עם בקשה אחת. ההפרש ביניהם הוא מה שהכלל חוסך:

$$(1 - g_\ell^{\text{open}}) - (1 - g_\ell^{\text{closed}}) = g_\ell^{\text{closed}} - g_\ell^{\text{open}}$$



Vibe Coding

שיעור 4 - חלק ד׳ - ניסוח פורמלי: הרווח הצפוי מכלל

ניסוח פורמלי - הרווח הצפוי מכלל:

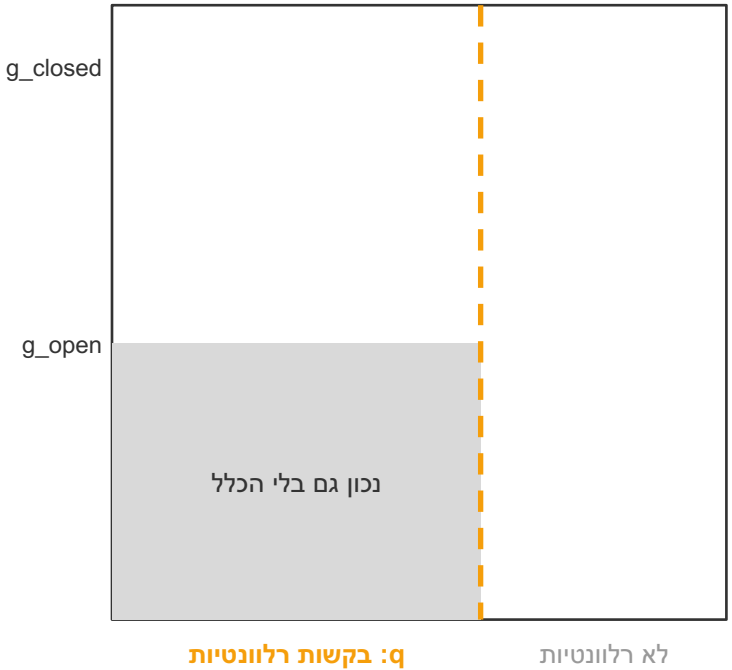
<u>הגדרות:</u>

כלל הוא שורה בקובץ ההנחיה שסוגרת החלטה ℓ. ההחלטה רלוונטית רק לחלק מהבקשות: נסמן ב-q_ℓ את שבר הבקשות שבהן היא רלוונטית (למשל, "argparse לשורת הפקודה" רלוונטי רק לבקשות שנוגעות בשורת הפקודה).

עבור בקשה רלוונטית: מספר החריגות הצפוי בלי הכלל ועם הכלל הוא התוחלת מחלק ב׳, עם בקשה אחת. ההפרש ביניהם הוא מה שהכלל חוסך:

$$(1 - g_\ell^{\text{open}}) - (1 - g_\ell^{\text{closed}}) = g_\ell^{\text{closed}} - g_\ell^{\text{open}}$$

עבור בקשה לא רלוונטית: הכלל לא משנה דבר, והוא חוסך 0.



Vibe Coding

שיעור 4 - חלק ד׳ - ניסוח פורמלי: הרווח הצפוי מכלל

ניסוח פורמלי - הרווח הצפוי מכלל:

<u>הגדרות:</u>

כלל הוא שורה בקובץ ההנחיה שסוגרת החלטה ℓ. ההחלטה רלוונטית רק לחלק מהבקשות: נסמן ב-q_ℓ את שבר הבקשות שבהן היא רלוונטית (למשל, "argparse לשורת הפקודה" רלוונטי רק לבקשות שנוגעות בשורת הפקודה).

עבור בקשה רלוונטית: מספר החריגות הצפוי בלי הכלל ועם הכלל הוא התוחלת מחלק ב׳, עם בקשה אחת. ההפרש ביניהם הוא מה שהכלל חוסך:

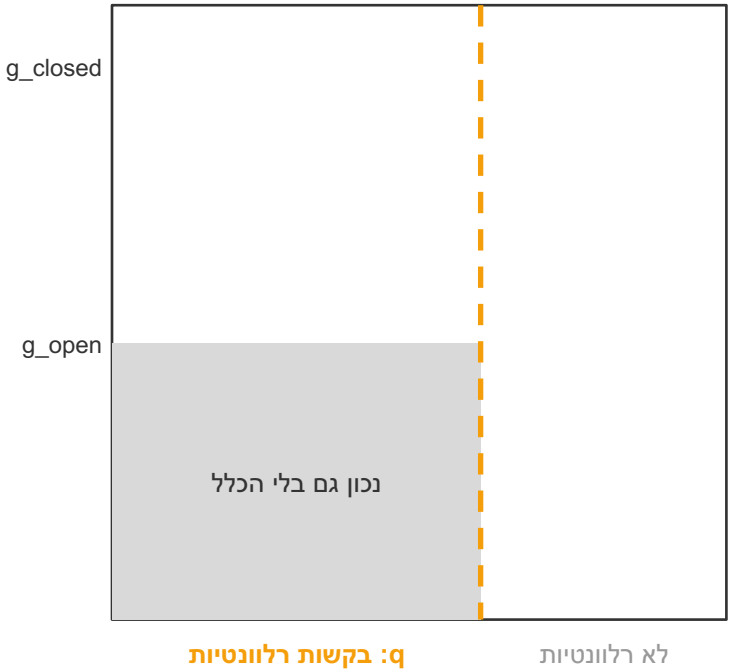
$$(1 - g_\ell^{\text{open}}) - (1 - g_\ell^{\text{closed}}) = g_\ell^{\text{closed}} - g_\ell^{\text{open}}$$

עבור בקשה לא רלוונטית: הכלל לא משנה דבר, והוא חוסך 0.

<u>נוסחה:</u>

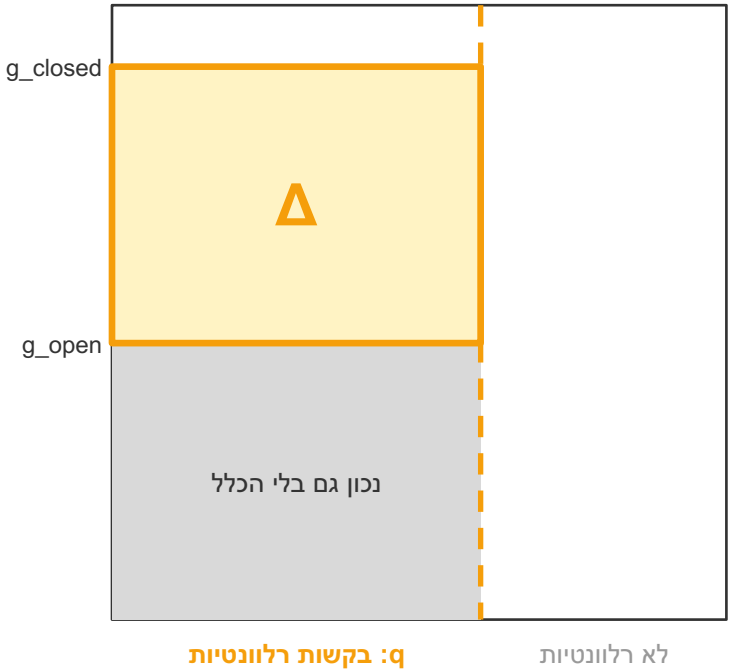
בקשה היא רלוונטית בסיכוי q_ℓ. לכן **הרווח הצפוי מהכלל**, מספר החריגות שהוא חוסך בממוצע לבקשה, הוא:

$$\Delta_\ell = q_\ell \left(g_\ell^{\text{closed}} - g_\ell^{\text{open}} \right)$$



Vibe Coding

שיעור 4 - חלק ד׳ - ניסוח פורמלי: הרווח הצפוי מכלל



ניסוח פורמלי - הרווח הצפוי מכלל:

<u>הגדרות:</u>

כלל הוא שורה בקובץ ההנחיה שסוגרת החלטה ℓ. ההחלטה רלוונטית רק לחלק מהבקשות: נסמן ב-q_ℓ את שבר הבקשות שבהן היא רלוונטית (למשל, "argparse לשורת הפקודה" רלוונטי רק לבקשות שנוגעות בשורת הפקודה).

עבור בקשה רלוונטית: מספר החריגות הצפוי בלי הכלל ועם הכלל הוא התוחלת מחלק ב׳, עם בקשה אחת. ההפרש ביניהם הוא מה שהכלל חוסך:

$$(1 - g_\ell^{\text{open}}) - (1 - g_\ell^{\text{closed}}) = g_\ell^{\text{closed}} - g_\ell^{\text{open}}$$

עבור בקשה לא רלוונטית: הכלל לא משנה דבר, והוא חוסך 0.

<u>נוסחה:</u>

בקשה היא רלוונטית בסיכוי q_ℓ. לכן **הרווח הצפוי מהכלל**, מספר החריגות שהוא חוסך בממוצע לבקשה, הוא:

$$\Delta_\ell = q_\ell \left(g_\ell^{\text{closed}} - g_\ell^{\text{open}} \right)$$

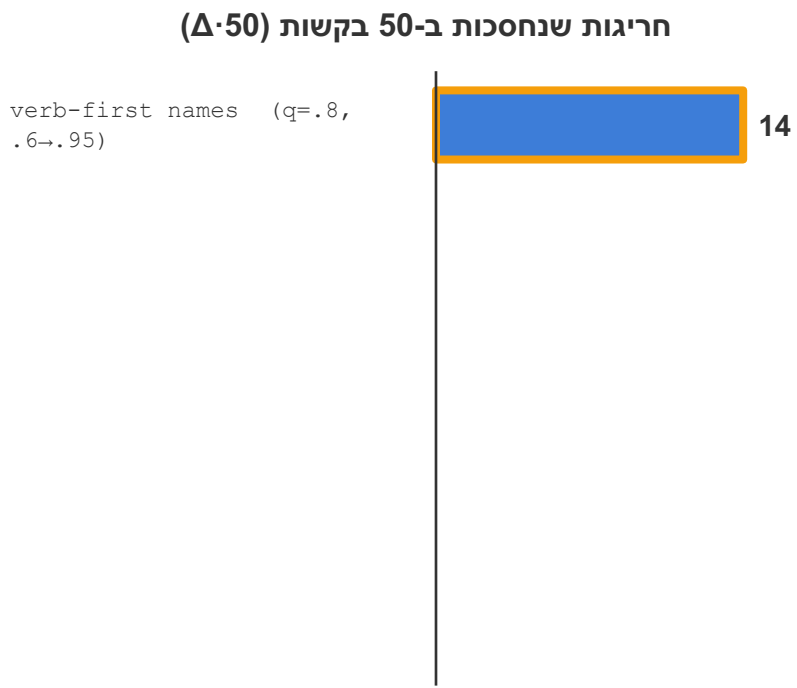
באיור: הרוחב הוא q_ℓ, הגובה הוא הפער בין $g^{\text{open}}{}_\ell$ ל-$g^{\text{closed}}{}_\ell$, ו-Δ_ℓ הוא השטח המודגש. לאורך Q בקשות נחסכות $Q \cdot \Delta_\ell$ חריגות.

Vibe Coding

שיעור 4 - חלק ד׳ - דוגמה מחושבת

<u>דוגמה מחושבת, 50 בקשות (q ו-g להמחשה; החישוב אמיתי):</u>

חמישה כללים מועמדים לקובץ ההנחיה של פרויקט ה-Todo. ליד כל כלל באיור מופיעים ה-q שלו, וה-g בלי הכלל ועם הכלל. אורך העמודה הוא מספר החריגות שהכלל חוסך ב-50 בקשות, כלומר $\Delta \cdot 50$.



Vibe Coding

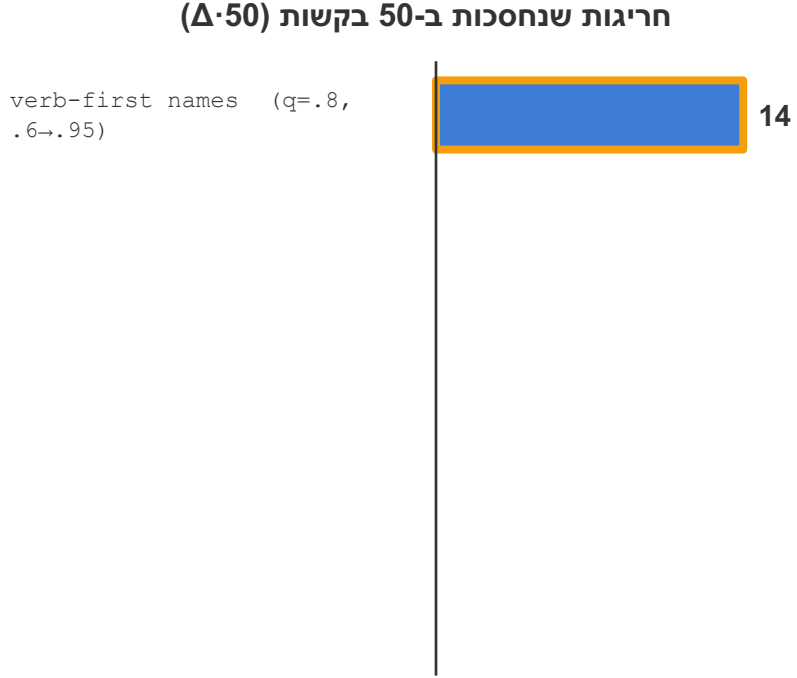
שיעור 4 - חלק ד׳ - דוגמה מחושבת

<u>דוגמה מחושבת, 50 בקשות (q ו-g להמחשה; החישוב אמיתי):</u>

חמישה כללים מועמדים לקובץ ההנחיה של פרויקט ה-Todo. ליד כל כלל באיור מופיעים ה-q שלו, וה-g בלי הכלל ועם הכלל. אורך העמודה הוא מספר החריגות שהכלל חוסך ב-50 בקשות, כלומר $50 \cdot \Delta$.

שמות בסגנון פועל-בהתחלה (add_task): רלוונטי לרוב הבקשות, $q = 0.8$, ובלעדיו המודל בוחר בסגנון הזה בסיכוי של 60% בלבד:

$$\Delta = 0.8 \cdot (0.95 - 0.6) = 0.28, \qquad 50\,\Delta = 14$$



Vibe Coding

שיעור 4 - חלק ד׳ - דוגמה מחושבת

<u>דוגמה מחושבת, 50 בקשות (q ו-g להמחשה; החישוב אמיתי):</u>

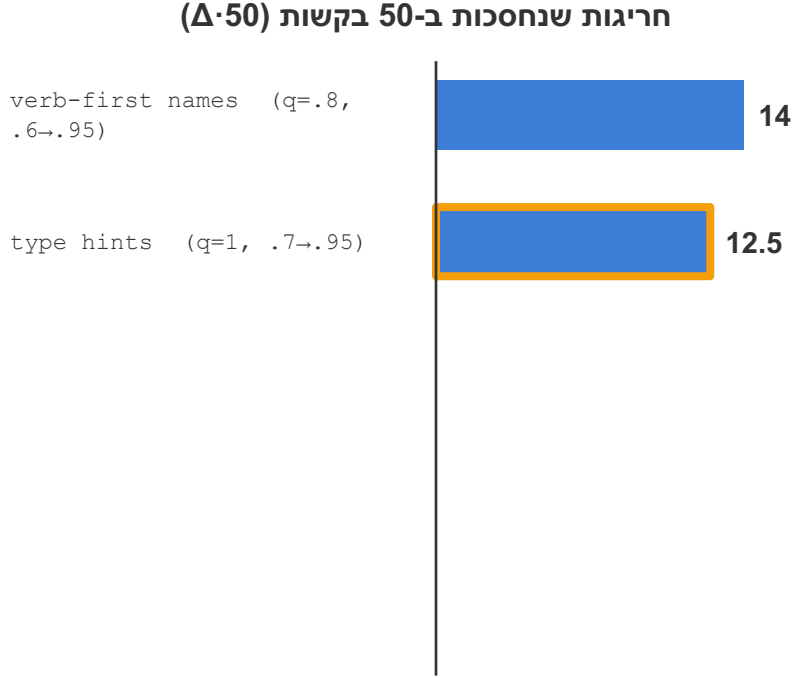
חמישה כללים מועמדים לקובץ ההנחיה של פרויקט ה-Todo. ליד כל כלל באיור מופיעים ה-q שלו, וה-g בלי הכלל ועם הכלל. אורך העמודה הוא מספר החריגות שהכלל חוסך ב-50 בקשות, כלומר $50 \cdot \Delta$.

שמות בסגנון פועל-בהתחלה (add_task): רלוונטי לרוב הבקשות, $q = 0.8$, ובלעדיו המודל בוחר בסגנון הזה בסיכוי של 60% בלבד:

$$\Delta = 0.8 \cdot (0.95 - 0.6) = 0.28, \qquad 50\,\Delta = 14$$

type hints בכל פונקציה: רלוונטי לכל בקשה ($q = 1$), ובלעדיו המודל מוסיף אותם בסיכוי של 70%, כמו בחלק ב׳:

$$\Delta = 1 \cdot (0.95 - 0.7) = 0.25, \qquad 50\,\Delta = 12.5$$



Vibe Coding

שיעור 4 - חלק ד׳ - דוגמה מחושבת

<u>דוגמה מחושבת, 50 בקשות (q ו-g להמחשה; החישוב אמיתי):</u>

חמישה כללים מועמדים לקובץ ההנחיה של פרויקט ה-Todo. ליד כל כלל באיור מופיעים ה-q שלו, וה-g בלי הכלל ועם הכלל. אורך העמודה הוא מספר החריגות שהכלל חוסך ב-50 בקשות, כלומר $50 \cdot \Delta$.

שמות בסגנון פועל-בהתחלה (add_task): רלוונטי לרוב הבקשות, $q = 0.8$, ובלעדיו המודל בוחר בסגנון הזה בסיכוי של 60% בלבד:

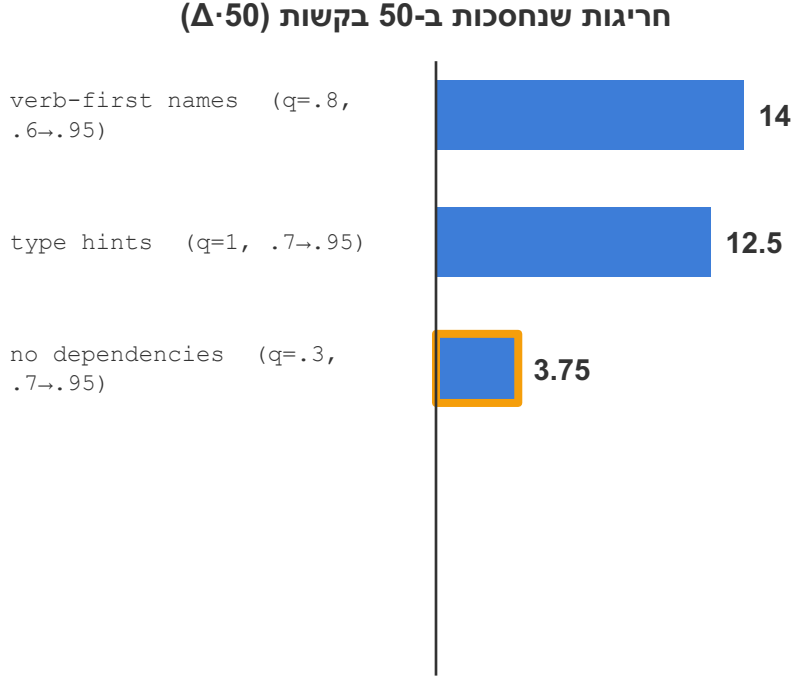
$$\Delta = 0.8 \cdot (0.95 - 0.6) = 0.28, \qquad 50\,\Delta = 14$$

type hints בכל פונקציה: רלוונטי לכל בקשה ($q = 1$), ובלעדיו המודל מוסיף אותם בסיכוי של 70%, כמו בחלק ב׳:

$$\Delta = 1 \cdot (0.95 - 0.7) = 0.25, \qquad 50\,\Delta = 12.5$$

בלי תלויות חיצוניות: רלוונטי רק לחלק קטן מהבקשות ($q = 0.3$), והמודל ממילא נמנע מהן לרוב:

$$\Delta = 0.3 \cdot (0.95 - 0.7) = 0.075, \qquad 50\,\Delta = 3.75$$



Vibe Coding

שיעור 4 - חלק ד׳ - דוגמה מחושבת

<u>דוגמה מחושבת, 50 בקשות (q ו-g להמחשה; החישוב אמיתי):</u>

חמישה כללים מועמדים לקובץ ההנחיה של פרויקט ה-Todo. ליד כל כלל באיור מופיעים ה-q שלו, וה-g בלי הכלל ועם הכלל. אורך העמודה הוא מספר החריגות שהכלל חוסך ב-50 בקשות, כלומר $50 \cdot \Delta$.

שמות בסגנון פועל-בהתחלה (add_task): רלוונטי לרוב הבקשות, $q = 0.8$, ובלעדיו המודל בוחר בסגנון הזה בסיכוי של 60% בלבד:

$$\Delta = 0.8 \cdot (0.95 - 0.6) = 0.28, \qquad 50\,\Delta = 14$$

type hints בכל פונקציה: רלוונטי לכל בקשה ($q = 1$), ובלעדיו המודל מוסיף אותם בסיכוי של 70%, כמו בחלק ב׳:

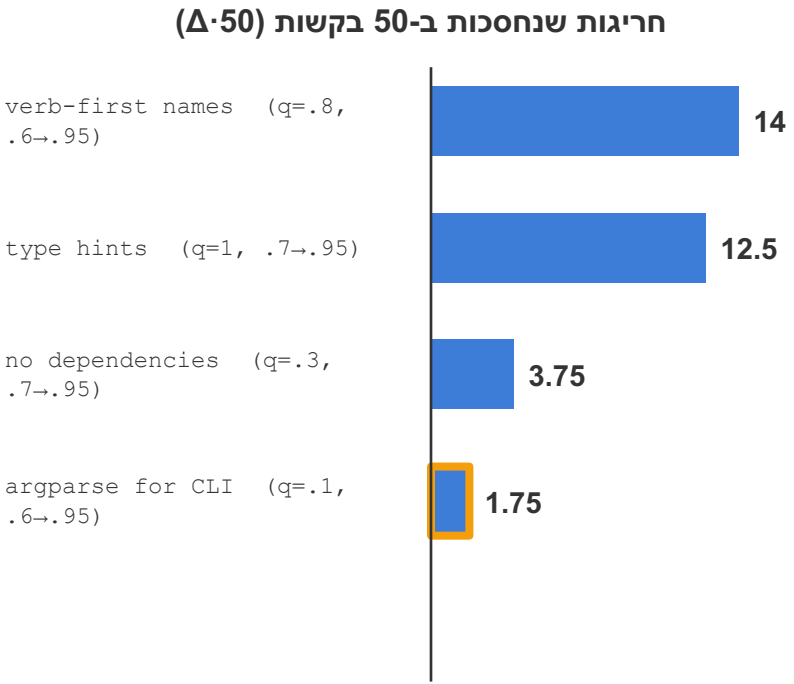
$$\Delta = 1 \cdot (0.95 - 0.7) = 0.25, \qquad 50\,\Delta = 12.5$$

בלי תלויות חיצוניות: רלוונטי רק לחלק קטן מהבקשות ($q = 0.3$), והמודל ממילא נמנע מהן לרוב:

$$\Delta = 0.3 \cdot (0.95 - 0.7) = 0.075, \qquad 50\,\Delta = 3.75$$

argparse לשורת הפקודה: מתוקן היטב, אבל רלוונטי רק לעשירית מהבקשות:

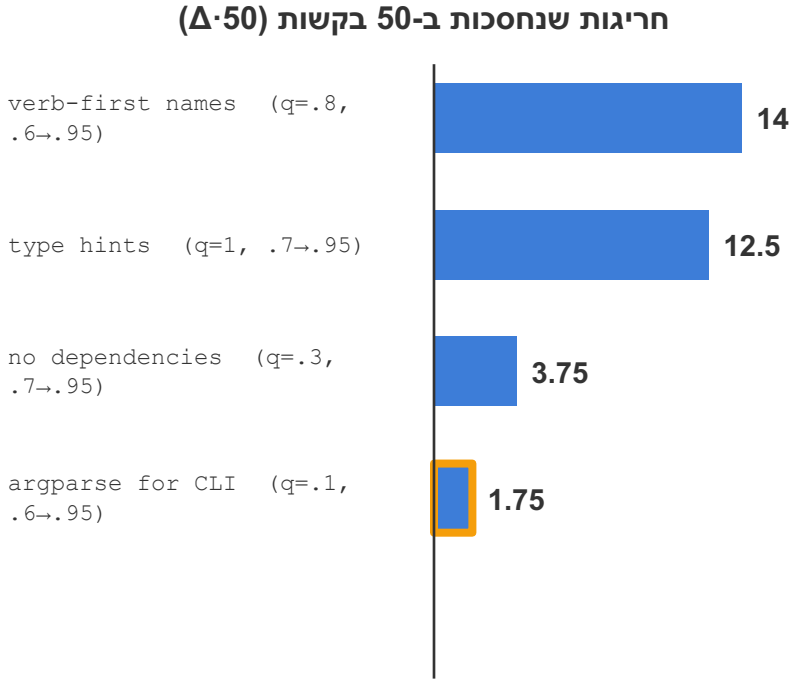
$$\Delta = 0.1 \cdot (0.95 - 0.6) = 0.035, \qquad 50\,\Delta = 1.75$$



Vibe Coding

שיעור 4 - חלק ד׳ - דוגמה מחושבת: מה למדנו

שאלה: ומה עם הכלל **"כתוב ב-Python"**? הוא רלוונטי לכל בקשה $(q = 1)$.

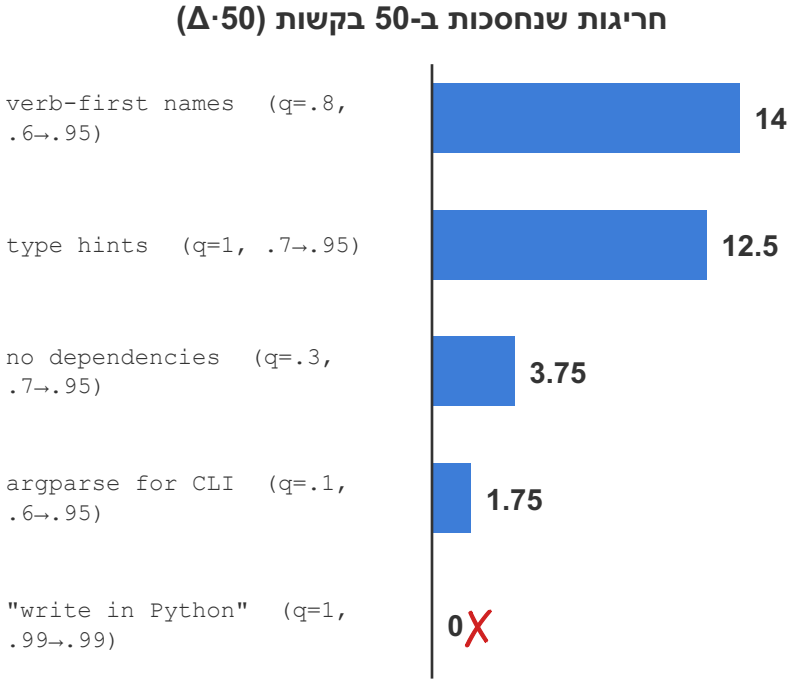


Vibe Coding

שיעור 4 - חלק ד׳ - דוגמה מחושבת: מה למדנו

שאלה: ומה עם הכלל **"כתוב ב-Python"**? הוא רלוונטי לכל בקשה $(q = 1)$.

תשובה: אפס. המודל כותב Python ממילא: בהסתברות 0.99 עם הכלל, ובאותה הסתברות בלעדיו. הפער אפס, ולכן $\Delta = 0$.



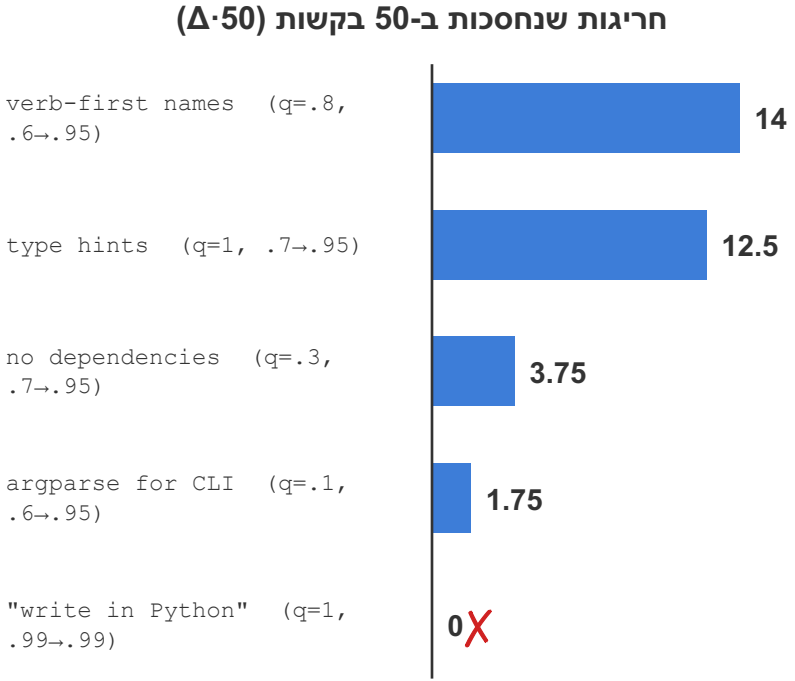
Vibe Coding

שיעור 4 - חלק ד׳ - דוגמה מחושבת: מה למדנו

שאלה: ומה עם הכלל **"כתוב ב-Python"**? הוא רלוונטי לכל בקשה $(q = 1)$.

תשובה: אפס. המודל כותב Python ממילא: בהסתברות 0.99 עם הכלל, ובאותה הסתברות בלעדיו. הפער אפס, ולכן $\Delta = 0$.

ושימו לב לשני הכללים הראשונים: כלל השמות רלוונטי לפחות בקשות מכלל ה-type hints, **ובכל זאת חוסך יותר**, כי הפער שלו גדול יותר.



Vibe Coding

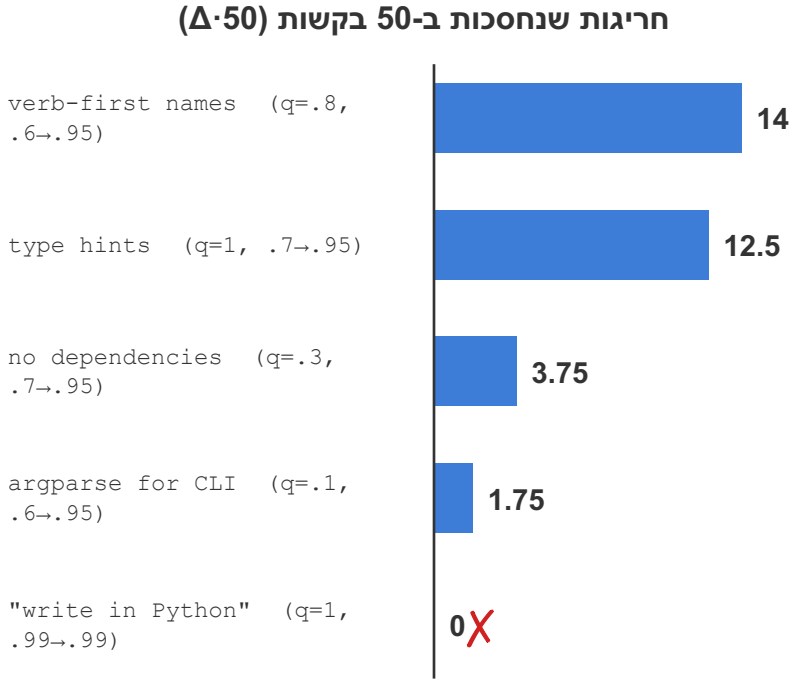
שיעור 4 - חלק ד׳ - דוגמה מחושבת: מה למדנו

שאלה: ומה עם הכלל **"כתוב ב-Python"**? הוא רלוונטי לכל בקשה $(q = 1)$.

תשובה: אפס. המודל כותב Python ממילא: בהסתברות 0.99 עם הכלל, ובאותה הסתברות בלעדיו. הפער אפס, ולכן $\Delta = 0$.

ושימו לב לשני הכללים הראשונים: כלל השמות רלוונטי לפחות בקשות מכלל ה-type hints, **ובכל זאת חוסך יותר**, כי הפער שלו גדול יותר.

המסקנה: כלל שווה מקום רק כשההחלטה **חוזרת בהרבה בקשות** $(q_\ell$ גבוה), **וגם** המודל **טועה בה בלי הכלל** (פער גדול בין $g^{\text{open}}{}_\ell$ ל-$g^{\text{closed}}{}_\ell$).



Vibe Coding

שיעור 4 - חלק ד׳ - העלות בטוקנים

ניסוח פורמלי - העלות בטוקנים:

הרווח מכלל הוא Δ; המחיר שלו הוא המקום שהוא תופס. קובץ ההנחיה נכנס לכל בקשה, ולכן לאורך Q בקשות הוא עולה $Q \cdot n_{\text{inst}}$ טוקנים. לפי תקציב החלון מחלק א׳, מה שנשאר לשאר החלקים בכל בקשה:

$$n_{\text{history}} + n_{\text{files}} + n_{\text{prompt}} + n_{\text{answer}} \;\le\; L - n_{\text{tool}} - n_{\text{inst}}$$

חלון ההקשר

L = 128,000

Vibe Coding

שיעור 4 - חלק ד׳ - העלות בטוקנים

ניסוח פורמלי - העלות בטוקנים:

הרווח מכלל הוא Δ; המחיר שלו הוא המקום שהוא תופס. קובץ ההנחיה נכנס לכל בקשה, ולכן לאורך Q בקשות הוא עולה $Q \cdot n_{\text{inst}}$ טוקנים. לפי תקציב החלון מחלק א׳, מה שנשאר לשאר החלקים בכל בקשה:

$$n_{\text{history}} + n_{\text{files}} + n_{\text{prompt}} + n_{\text{answer}} \leq L - n_{\text{tool}} - n_{\text{inst}}$$

<u>דוגמה (נמדד; החלון L = 128,000 להמחשה):</u>

קובץ ה-Todo מחלק ג׳, n_{inst} = 226, תופס כ-0.18% מהחלון: זניח. לאורך 50 בקשות הוא עולה 11,300 טוקנים באנגלית, מול 17,400 בעברית. בגישה דרך API, טוקנים הם כסף.



Vibe Coding

שיעור 4 - חלק ד׳ - העלות בטוקנים

ניסוח פורמלי - העלות בטוקנים:

הרווח מכלל הוא Δ; המחיר שלו הוא המקום שהוא תופס. קובץ ההנחיה נכנס לכל בקשה, ולכן לאורך Q בקשות הוא עולה $Q \cdot n_{\text{inst}}$ טוקנים. לפי תקציב החלון מחלק א׳, מה שנשאר לשאר החלקים בכל בקשה:

$$n_{\text{history}} + n_{\text{files}} + n_{\text{prompt}} + n_{\text{answer}} \leq L - n_{\text{tool}} - n_{\text{inst}}$$

<u>דוגמה (נמדד; החלון L = 128,000 להמחשה):</u>

קובץ ה-Todo מחלק ג׳, n_{inst} = 226, תופס כ-0.18% מהחלון: זניח. לאורך 50 בקשות הוא עולה 11,300 טוקנים באנגלית, מול 17,400 בעברית. בגישה דרך API, טוקנים הם כסף.

קובץ "מנופח" של 2,000 מילים הוא כ-2,000 עד 3,000 טוקנים: **בערך 2% מהחלון בכל בקשה**, ועשרות אלפי טוקנים לאורך פרויקט.



Vibe Coding

שיעור 4 - חלק ד׳ - העלות בטוקנים

ניסוח פורמלי - העלות בטוקנים:

הרווח מכלל הוא Δ; המחיר שלו הוא המקום שהוא תופס. קובץ ההנחיה נכנס לכל בקשה, ולכן לאורך Q בקשות הוא עולה $Q \cdot n_{\text{inst}}$ טוקנים. לפי תקציב החלון מחלק א׳, מה שנשאר לשאר החלקים בכל בקשה:

$$n_{\text{history}} + n_{\text{files}} + n_{\text{prompt}} + n_{\text{answer}} \leq L - n_{\text{tool}} - n_{\text{inst}}$$

<u>דוגמה (נמדד; החלון L = 128,000 להמחשה):</u>

קובץ ה-Todo מחלק ג׳, n_{inst} = 226, תופס כ-0.18% מהחלון: זניח. לאורך 50 בקשות הוא עולה 11,300 טוקנים באנגלית, מול 17,400 בעברית. בגישה דרך API, טוקנים הם כסף.

קובץ "מנופח" של 2,000 מילים הוא כ-2,000 עד 3,000 טוקנים: **בערך 2% מהחלון בכל בקשה**, ועשרות אלפי טוקנים לאורך פרויקט.

אז למה בכל זאת לשמור על קובץ קצר? כי כלל שהמודל מקיים גם בלעדיו ($\Delta \approx 0$) לא חוסך אף חריגה, אבל עדיין צריך לעדכן אותו כשהפרויקט משתנה, והוא עלול לסתור כלל אחר.



Vibe Coding

שיעור 4 - חלק ד׳ - מה לא נכנס לקובץ

<u>מה לא נכנס לקובץ (משמאל, ארבעה מועמדים שנמחקים):</u>

- **דברים שהמודל עושה ממילא** ($\Delta \approx 0$), כמו "Write all code in Python": הכלל לא חוסך אף חריגה.

```
# CLAUDE.md (candidates)

"Write all code in Python."                    Δ = 0

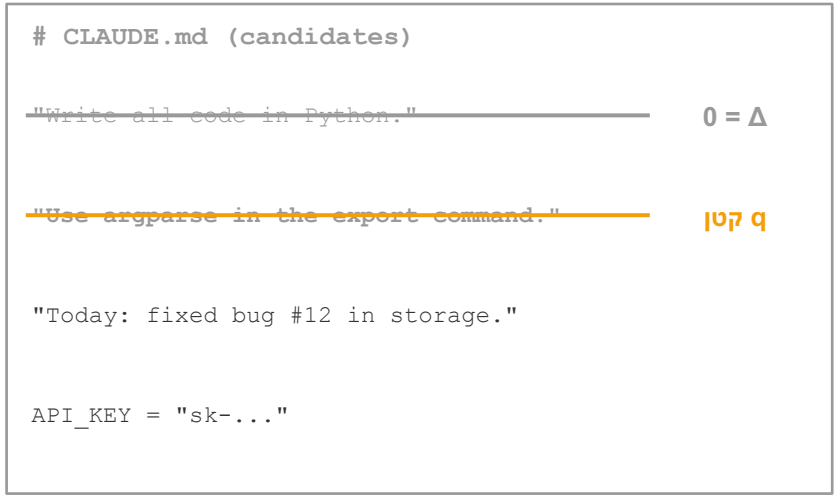
"Use argparse in the export command."

"Today: fixed bug #12 in storage."

API_KEY = "sk-..."
```

Vibe Coding

שיעור 4 - חלק ד׳ - מה לא נכנס לקובץ

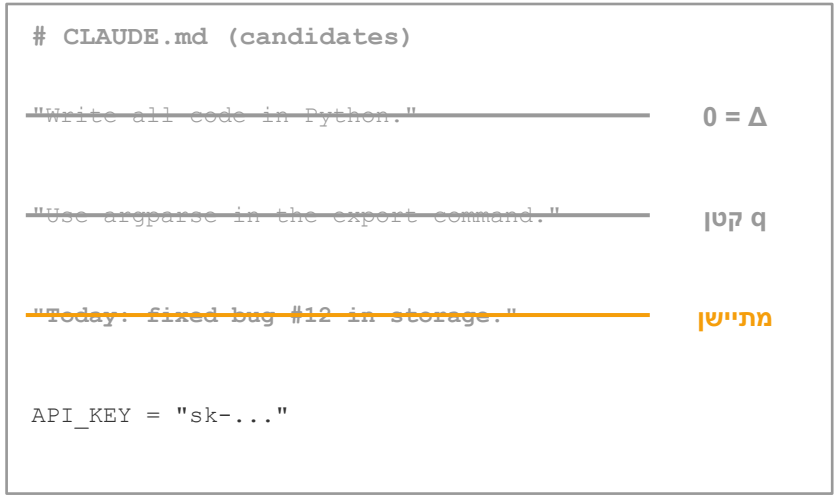


<u>מה לא נכנס לקובץ (משמאל, ארבעה מועמדים שנמחקים):</u>

- **דברים שהמודל עושה ממילא** ($\Delta \approx 0$), כמו "Write all code in Python": הכלל לא חוסך אף חריגה.
- **החלטות של בקשה אחת** (q קטן), כמו "argparse בפקודת הייצוא": מקומן בפרומפט של אותה בקשה.

Vibe Coding

שיעור 4 - חלק ד׳ - מה לא נכנס לקובץ



<u>מה לא נכנס לקובץ (משמאל, ארבעה מועמדים שנמחקים):</u>

- **דברים שהמודל עושה ממילא** ($\Delta \approx 0$), כמו "Write all code in Python": הכלל לא חוסך אף חריגה.
- **החלטות של בקשה אחת** (q קטן), כמו "argparse בפקודת הייצוא": מקומן בפרומפט של אותה בקשה.
- **מידע שמשתנה כל יום** ("Today: fixed bug #12"): יתיישן מהר (חלק ה׳). מקומו בסיכום מעבר (חלק ז׳).

Vibe Coding

שיעור 4 - חלק ד׳ - מה לא נכנס לקובץ



<u>מה לא נכנס לקובץ (משמאל, ארבעה מועמדים שנמחקים):</u>

- **דברים שהמודל עושה ממילא** ($\Delta \approx 0$), כמו "Write all code in Python": הכלל לא חוסך אף חריגה.
- **החלטות של בקשה אחת** (q קטן), כמו "argparse בפקודת הייצוא": מקומן בפרומפט של אותה בקשה.
- **מידע שמשתנה כל יום** ("Today: fixed bug #12"): יתיישן מהר (חלק ה׳). מקומו בסיכום מעבר (חלק ז׳).
- **סודות** (מפתחות API, סיסמאות, כתובות של שרתים פנימיים): הקובץ נשמר ב-git **ונשלח לספק בכל בקשה** (מפגש 10).

Vibe Coding

שיעור 4 - חלק ד׳ - מה לא נכנס לקובץ



חוזר? | **טועה בלעדיו?** | **בדיק?**

<u>מה לא נכנס לקובץ (משמאל, ארבעה מועמדים שנמחקים):</u>

- **דברים שהמודל עושה ממילא** ($\Delta \approx 0$), כמו "Write all code in Python": הכלל לא חוסך אף חריגה.
- **החלטות של בקשה אחת** (q קטן), כמו "argparse בפקודת הייצוא": מקומן בפרומפט של אותה בקשה.
- **מידע שמשתנה כל יום** ("Today: fixed bug #12"): יתיישן מהר (חלק ה׳). מקומו בסיכום מעבר (חלק ז׳).
- **סודות** (מפתחות API, סיסמאות, כתובות של שרתים פנימיים): הקובץ נשמר ב-git **ונשלח לספק בכל בקשה** (מפגש 10).

Best practice:

לכל כלל שאלו שלוש שאלות: **האם ההחלטה חוזרת** בהרבה בקשות? **האם המודל טועה** בה בלי הכלל? **האם אפשר לבדוק** תשובה מול הכלל? כלל שנכשל באחת מהן עובר לפרומפט, או נמחק.

Vibe Coding

שיעור 4 - חלק ד׳ - מכאן לחלק הבא

<u>מכאן לחלק הבא:</u>

עכשיו יש לנו קובץ קצר, שכל כלל בו מרוויח את מקומו. אבל הפרויקט לא עומד במקום: ספריות מתחלפות והמבנה משתנה.
בחלק ה׳ נראה **מה קורה לקובץ שלא עודכן**, ולמה הוא מזיק יותר מאשר אין קובץ בכלל.



ד״ר ולד לנדא

☕ הפסקה - 10 דקות

Vibe Coding

שיעור 4 - חלק ה׳ - ניסוח פורמלי: כלל מיושן

ניסוח פורמלי - למה כלל מיושן גרוע מאין כלל:

<u>הגדרות:</u>

כלל מיושן הוא כלל שהקוד כבר לא מקיים. בפרויקט ה-Todo, השמירה עברה מ-JSON ל-SQLite, אבל בקובץ ההנחיה עדיין כתוב " Storage: JSON".

```
# CLAUDE.md

Storage: JSON
```

```
# storage.py

sqlite3.connect()
```

הקובץ אומר דבר אחד, הקוד דבר אחר

Vibe Coding

שיעור 4 - חלק ה׳ - ניסוח פורמלי: כלל מיושן

ניסוח פורמלי - למה כלל מיושן גרוע מאין כלל:

<u>הגדרות:</u>

כלל מיושן הוא כלל שהקוד כבר לא מקיים. בפרויקט ה-Todo, השמירה עברה מ-JSON ל-SQLite, אבל בקובץ ההנחיה עדיין כתוב " Storage: JSON".

המודל מציית לקובץ בהסתברות g^{closed}_{ℓ}, כלומר כותב לפי **הכלל הישן**. כמו בחלק ד׳, q_{ℓ} הוא שבר הבקשות שההחלטה רלוונטית להן (כאן: בקשות שנוגעות בשמירה).

```
# CLAUDE.md

Storage: JSON
```

```
# storage.py

sqlite3.connect()
```

הקובץ אומר דבר אחד, הקוד דבר אחר

המודל מציית לקובץ
← כותב JSON

Vibe Coding

שיעור 4 - חלק ה׳ - ניסוח פורמלי: כלל מיושן

ניסוח פורמלי - למה כלל מיושן גרוע מאין כלל:

<u>הגדרות:</u>

כלל מיושן הוא כלל שהקוד כבר לא מקיים. בפרויקט ה-Todo, השמירה עברה מ-JSON ל-SQLite, אבל בקובץ ההנחיה עדיין כתוב " Storage: JSON".

המודל מציית לקובץ בהסתברות g^{closed}_{ℓ}, כלומר כותב לפי **הכלל הישן**. כמו בחלק ד׳, q_{ℓ} הוא שבר הבקשות שההחלטה רלוונטית להן (כאן: בקשות שנוגעות בשמירה).

שאלה: כלל מיושן הוא "פחות טוב" מהיעדר כלל, או **גרוע** ממנו?

```
# CLAUDE.md

Storage: JSON
```

```
# storage.py

sqlite3.connect()
```

הקובץ אומר דבר אחד, הקוד דבר אחר

המודל מציית לקובץ
← כותב JSON

Vibe Coding

שיעור 4 - חלק ה׳ - ניסוח פורמלי: כלל מיושן

ניסוח פורמלי - למה כלל מיושן גרוע מאין כלל:

<u>הגדרות:</u>

כלל מיושן הוא כלל שהקוד כבר לא מקיים. בפרויקט ה-Todo, השמירה עברה מ-JSON ל-SQLite, אבל בקובץ ההנחיה עדיין כתוב " Storage: JSON".

המודל מציית לקובץ בהסתברות g^{closed}_{ℓ}, כלומר כותב לפי **הכלל הישן**. כמו בחלק ד׳, q_ℓ הוא שבר הבקשות שההחלטה רלוונטית להן (כאן: בקשות שנוגעות בשמירה).

שאלה: כלל מיושן הוא "פחות טוב" מהיעדר כלל, או **גרוע** ממנו?

<u>נוסחה:</u>

תשובה: נשווה למצב שבו אין כלל בכלל, ובו התשובה מתאימה למוסכמה הנוכחית בהסתברות g^{open}_{ℓ}. עם כלל מיושן, התשובה מתאימה רק כשהמודל **לא** מציית לו, בהסתברות $1 - g^{\text{closed}}_{\ell}$ (בהנחה המפשטת שאז הוא בוחר נכון). הרווח מהכלל המיושן, לעומת היעדר כלל:

$$\Delta^{\text{stale}}_{\ell} = q_\ell \left((1 - g^{\text{closed}}_{\ell}) - g^{\text{open}}_{\ell} \right)$$

```
# CLAUDE.md

Storage: JSON
```

```
# storage.py

sqlite3.connect()
```

הקובץ אומר דבר אחד, הקוד דבר אחר

המודל מציית לקובץ
← כותב JSON

Vibe Coding

שיעור 4 - חלק ה׳ - כלל מיושן: דוגמה ומסקנה

<u>דוגמה (ערכים להמחשה):</u>

כל בקשה בדוגמה נוגעת בשמירה ($q = 1$), $g^{\text{open}} = 0.3$ ו-$g^{\text{closed}} = 0.95$.
כלל **מעודכן** ("Storage: SQLite"), לפי הנוסחה מחלק ד׳:

$$\Delta_\ell = q_\ell \left(g_\ell^{\text{closed}} - g_\ell^{\text{open}} \right)$$
$$= 1 \cdot (0.95 - 0.3) = 0.65$$

```
# CLAUDE.md

Storage: JSON
```

```
# storage.py

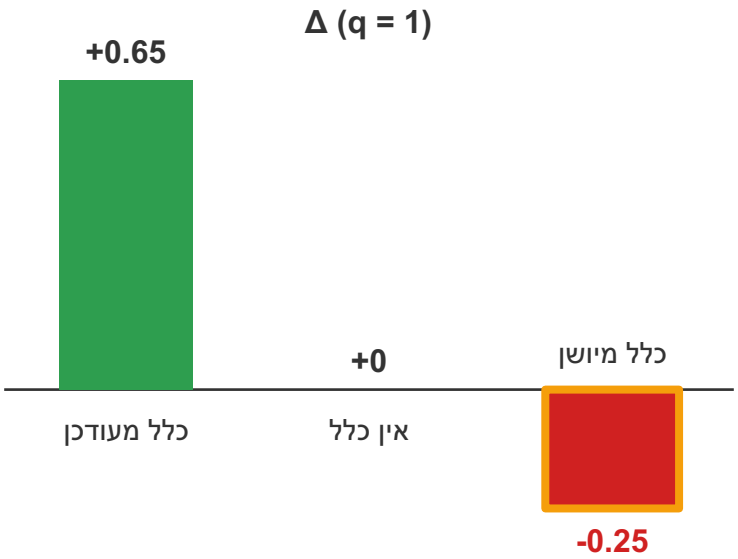
sqlite3.connect()
```

הקובץ אומר דבר אחד, הקוד דבר אחר

המודל מציית לקובץ
כותב JSON ←

Vibe Coding

שיעור 4 - חלק ה׳ - כלל מיושן: דוגמה ומסקנה



<u>דוגמה (ערכים להמחשה):</u>

כל בקשה בדוגמה נוגעת בשמירה ($q = 1$), $g^{\text{open}} = 0.3$ ו-$g^{\text{closed}} = 0.95$.

כלל **מעודכן** ("Storage: SQLite"), לפי הנוסחה מחלק ד׳:

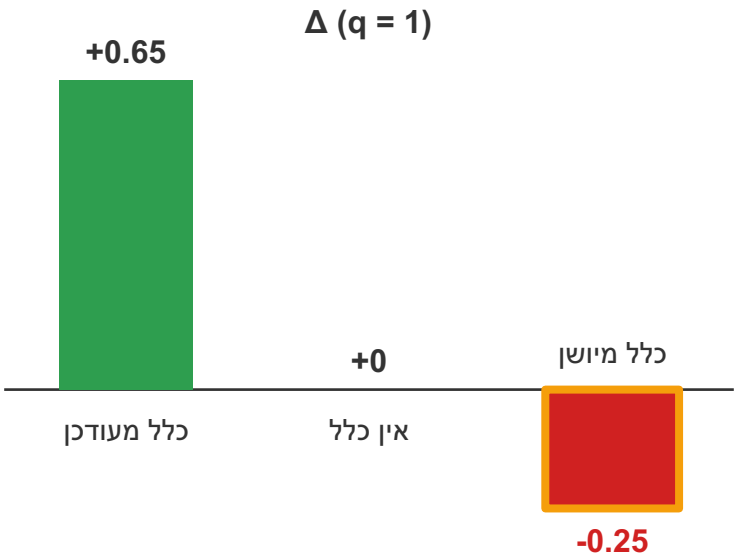
$$\Delta_\ell = q_\ell \left(g_\ell^{\text{closed}} - g_\ell^{\text{open}}\right) = 1 \cdot (0.95 - 0.3) = 0.65$$

כלל **מיושן** ("Storage: JSON"), לפי הנוסחה מהשקופית הקודמת:

$$\Delta_\ell^{\text{stale}} = q_\ell \left((1 - g_\ell^{\text{closed}}) - g_\ell^{\text{open}}\right) = 1 \cdot ((1 - 0.95) - 0.3) = -0.25$$

Vibe Coding

שיעור 4 - חלק ה׳ - כלל מיושן: דוגמה ומסקנה



<u>דוגמה (ערכים להמחשה):</u>

כל בקשה בדוגמה נוגעת בשמירה ($q = 1$), $g^{\text{open}} = 0.3$ ו-$g^{\text{closed}} = 0.95$.

כלל **מעודכן** ("Storage: SQLite"), לפי הנוסחה מחלק ד׳:

$$\Delta_\ell = q_\ell \left(g_\ell^{\text{closed}} - g_\ell^{\text{open}}\right) = 1 \cdot (0.95 - 0.3) = 0.65$$

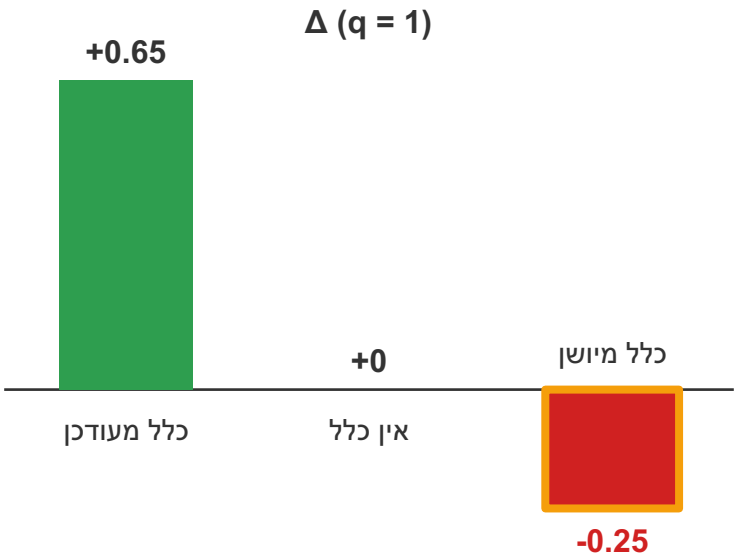
כלל **מיושן** ("Storage: JSON"), לפי הנוסחה מהשקופית הקודמת:

$$\Delta_\ell^{\text{stale}} = q_\ell \left((1 - g_\ell^{\text{closed}}) - g_\ell^{\text{open}}\right) = 1 \cdot ((1 - 0.95) - 0.3) = -0.25$$

כלומר, הכלל המיושן לא רק מפסיד את 0.65 החריגות שהיה חוסך: הוא **מוסיף** רבע חריגה לכל בקשה, לעומת מצב שבו אין כלל.

Vibe Coding

שיעור 4 - חלק ה׳ - כלל מיושן: דוגמה ומסקנה



דוגמה (ערכים להמחשה):

כל בקשה בדוגמה נוגעת בשמירה ($q = 1$), $g^{\text{open}} = 0.3$ ו-$g^{\text{closed}} = 0.95$.
כלל **מעודכן** ("Storage: SQLite"), לפי הנוסחה מחלק ד׳:

$$\Delta_\ell = q_\ell \left(g_\ell^{\text{closed}} - g_\ell^{\text{open}}\right) = 1 \cdot (0.95 - 0.3) = 0.65$$

כלל **מיושן** ("Storage: JSON"), לפי הנוסחה מהשקופית הקודמת:

$$\Delta_\ell^{\text{stale}} = q_\ell \left((1 - g_\ell^{\text{closed}}) - g_\ell^{\text{open}}\right) = 1 \cdot ((1 - 0.95) - 0.3) = -0.25$$

כלומר, הכלל המיושן לא רק מפסיד את 0.65 החריגות שהיה חוסך: הוא **מוסיף** רבע חריגה לכל בקשה, לעומת מצב שבו אין כלל.

מסקנה:

כלל מיושן לא "פחות טוב"; הוא **הופך את הכיוון**. ככל שהמודל מציית טוב יותר, הוא מזיק יותר. לכן מעדכנים את הקובץ **באותו commit** של השינוי בקוד.

Vibe Coding

שיעור 4 - חלק ה׳ - מכאן לחלק הבא

<u>מכאן לחלק הבא:</u>

חלקים ג׳-ה׳ עסקו בחלק **הקבוע** של הקלט, קובץ ההנחיה, שנכנס לכל בקשה. בחלק ו׳ נעבור לחלק **שמשתנה מבקשה לבקשה:** הקבצים מהפרויקט שמצרפים כדי שהמודל יוכל לבצע משימה מסוימת.



Vibe Coding

שיעור 4 - חלק ו׳ - הקשר ממוקד: מה מצרפים לבקשה

<u>למה זה חשוב:</u>

קובץ ההנחיה נכנס לכל בקשה; **הקבצים** נכנסים רק לבקשה שצריכה אותם, והם החלק היקר בקלט. יש שתי טעויות הפוכות:

`storage.py`	`tasks.py`
`main.py`	`dates.py`
`config.py`	`models.py`
`cli_help.py`	`export.py`
`tests/test_storage.py`	`tests/test_tasks.py`
`README.md`	`tests/test_dates.py`

Vibe Coding

שיעור 4 - חלק ו׳ - הקשר ממוקד: מה מצרפים לבקשה

<u>למה זה חשוב:</u>

קובץ ההנחיה נכנס לכל בקשה; **הקבצים** נכנסים רק לבקשה שצריכה אותם, והם החלק היקר בקלט. יש שתי טעויות הפוכות:

- **מצרפים מעט מדי:** המודל לא רואה פונקציה שקיימת, וממציא אחת, בשם אחר או עם פרמטרים אחרים (הזיות ממפגש 2).

`storage.py`	`tasks.py`
`main.py`	`dates.py`
`config.py`	`models.py`
`cli_help.py`	`export.py`
`tests/test_storage.py`	`tests/test_tasks.py`
`README.md`	`tests/test_dates.py`

Vibe Coding

שיעור 4 - חלק ו׳ - הקשר ממוקד: מה מצרפים לבקשה

<u>למה זה חשוב:</u>

קובץ ההנחיה נכנס לכל בקשה; **הקבצים** נכנסים רק לבקשה שצריכה אותם, והם החלק היקר בקלט. יש שתי טעויות הפוכות:

- **מצרפים מעט מדי:** המודל לא רואה פונקציה שקיימת, וממציא אחת, בשם אחר או עם פרמטרים אחרים (הזיות ממפגש 2).
- **מצרפים הכול:** החלון מתמלא, וההקשר העמוס מקטין את חדות ההחלטות. וכל בקשה בהמשך השיחה נושאת את כל זה בהיסטוריה.

`storage.py`	`tasks.py`
`main.py`	`dates.py`
`config.py`	`models.py`
`cli_help.py`	`export.py`
`tests/test_storage.py`	`tests/test_tasks.py`
`README.md`	`tests/test_dates.py`

Vibe Coding

שיעור 4 - חלק ו׳ - הקשר ממוקד: מה מצרפים לבקשה

למה זה חשוב:

קובץ ההנחיה נכנס לכל בקשה; **הקבצים** נכנסים רק לבקשה שצריכה אותם, והם החלק היקר בקלט. יש שתי טעויות הפוכות:

- **מצרפים מעט מדי:** המודל לא רואה פונקציה שקיימת, וממציא אחת, בשם אחר או עם פרמטרים אחרים (הזיות ממפגש 2).
- **מצרפים הכול:** החלון מתמלא, וההקשר העמוס מקטין את חדות ההחלטות. וכל בקשה בהמשך השיחה נושאת את כל זה בהיסטוריה.

דוגמה מלווה:

בפרויקט ה-Todo יש 12 קבצים (הרשת משמאל), כ-2,000 שורות. המשימה: להוסיף ל-tasks.py פונקציה filter_by_date שמסננת מטלות לפי תאריך, עם טסט.

storage.py	tasks.py
main.py	dates.py
config.py	models.py
cli_help.py	export.py
tests/test_storage.py	tests/test_tasks.py
README.md	tests/test_dates.py

Vibe Coding

שיעור 4 - חלק ו׳ - הקשר ממוקד: מה מצרפים לבקשה

למה זה חשוב:

קובץ ההנחיה נכנס לכל בקשה; **הקבצים** נכנסים רק לבקשה שצריכה אותם, והם החלק היקר בקלט. יש שתי טעויות הפוכות:

- **מצרפים מעט מדי:** המודל לא רואה פונקציה שקיימת, וממציא אחת, בשם אחר או עם פרמטרים אחרים (הזיות ממפגש 2).
- **מצרפים הכול:** החלון מתמלא, וההקשר העמוס מקטין את חדות ההחלטות. וכל בקשה בהמשך השיחה נושאת את כל זה בהיסטוריה.

דוגמה מלווה:

בפרויקט ה-Todo יש 12 קבצים (הרשת משמאל), כ-2,000 שורות. המשימה: להוסיף ל-tasks.py פונקציה filter_by_date שמסננת מטלות לפי תאריך, עם טסט.

שאלה: אילו קבצים המשימה **צריכה**, וכמה מכל אחד מהם?

storage.py	tasks.py
main.py	dates.py
config.py	models.py
cli_help.py	export.py
tests/test_storage.py	tests/test_tasks.py
README.md	tests/test_dates.py

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ו׳ - הקשר ממוקד: מה מצרפים לבקשה

<u>למה זה חשוב:</u>

קובץ ההנחיה נכנס לכל בקשה; **הקבצים** נכנסים רק לבקשה שצריכה אותם, והם החלק היקר בקלט. יש שתי טעויות הפוכות:

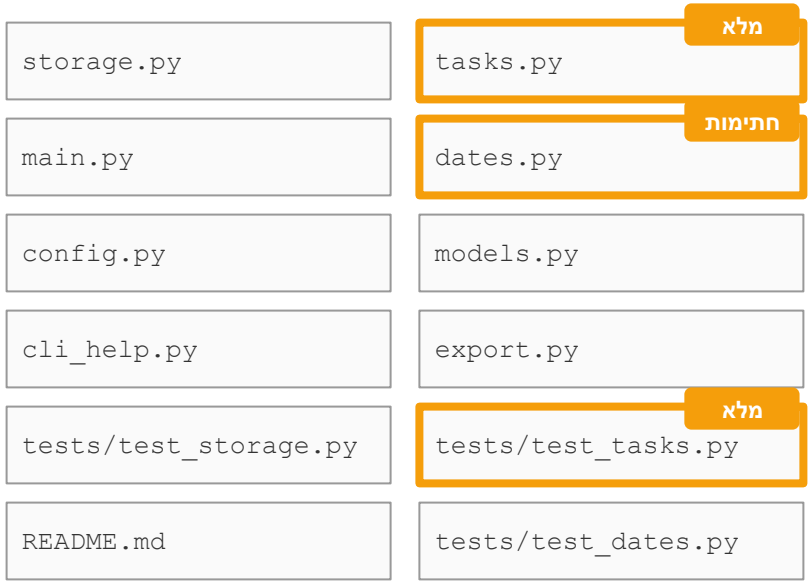
- **מצרפים מעט מדי:** המודל לא רואה פונקציה שקיימת, וממציא אחת, בשם אחר או עם פרמטרים אחרים (הזיות ממפגש 2).
- **מצרפים הכול:** החלון מתמלא, וההקשר העמוס מקטין את חדות ההחלטות. וכל בקשה בהמשך השיחה נושאת את כל זה בהיסטוריה.

<u>דוגמה מלווה:</u>

בפרויקט ה-Todo יש 12 קבצים (הרשת משמאל), כ-2,000 שורות. המשימה: להוסיף ל-tasks.py פונקציה filter_by_date שמסננת מטלות לפי תאריך, עם טסט.

שאלה: אילו קבצים המשימה **צריכה**, וכמה מכל אחד מהם?

תשובה: שלושה **צרכים** (התגיות באיור). את tasks.py ואת test_tasks.py המשימה **משנה**, ולכן המודל צריך את **התוכן המלא** שלהם. ב-dates.py היא רק **משתמשת** בפונקציה parse_date, שכבר קיימת שם.



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ו׳ - הקשר ממוקד: מה מצרפים לבקשה

<u>למה זה חשוב:</u>

קובץ ההנחיה נכנס לכל בקשה; **הקבצים** נכנסים רק לבקשה שצריכה אותם, והם החלק היקר בקלט. יש שתי טעויות הפוכות:

- **מצרפים מעט מדי:** המודל לא רואה פונקציה שקיימת, וממציא אחת, בשם אחר או עם פרמטרים אחרים (הזיות ממפגש 2).
- **מצרפים הכול:** החלון מתמלא, וההקשר העמוס מקטין את חדות ההחלטות. וכל בקשה בהמשך השיחה נושאת את כל זה בהיסטוריה.

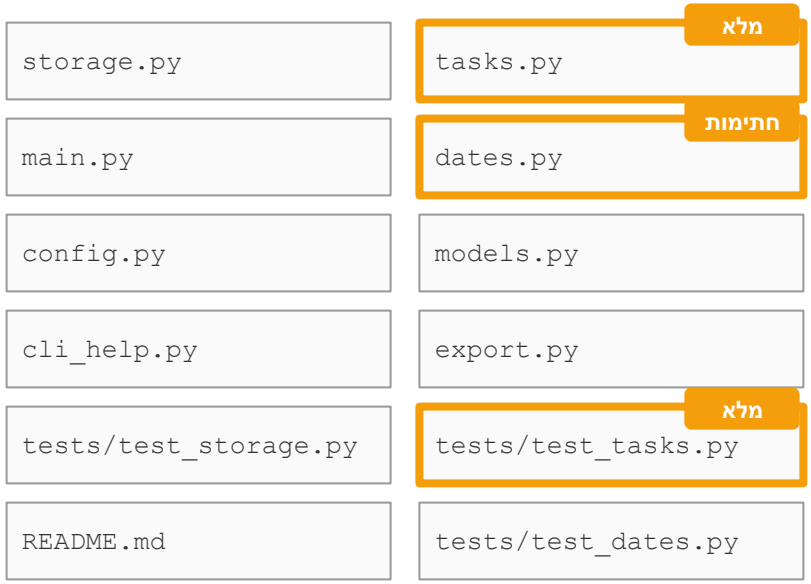
<u>דוגמה מלווה:</u>

בפרויקט ה-Todo יש 12 קבצים (הרשת משמאל), כ-2,000 שורות. המשימה: להוסיף ל-tasks.py פונקציה filter_by_date שמסננת מטלות לפי תאריך, עם טסט.

שאלה: אילו קבצים המשימה **צריכה**, וכמה מכל אחד מהם?

תשובה: שלושה **צרכים** (התגיות באיור). את tasks.py ואת test_tasks.py המשימה **משנה**, ולכן המודל צריך את **התוכן המלא** שלהם. ב-dates.py היא רק **משתמשת** בפונקציה parse_date, שכבר קיימת שם.

כשרק משתמשים בפונקציה מקובץ, מספיקות **החתימות** שלו: השורה הראשונה של כל פונקציה (שם, פרמטרים וטיפוסים) יחד עם ה-docstring, בלי הקוד שבתוכה.



Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: שתי שאלות

ניסוח פורמלי - כיסוי, דיוק וטוקנים:

<u>הרעיון:</u>

לפני ששולחים בקשה, בודקים את הקבצים שצירפתם בעזרת שתי שאלות:

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: שתי שאלות

ניסוח פורמלי - כיסוי, דיוק וטוקנים:

<u>הרעיון:</u>

לפני ששולחים בקשה, בודקים את הקבצים שצירפתם בעזרת שתי שאלות:

1. **האם צירפתי את כל מה שהמשימה צריכה?** אם חסר משהו, המודל ינחש.

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: שתי שאלות

ניסוח פורמלי - כיסוי, דיוק וטוקנים:

<u>הרעיון:</u>

לפני ששולחים בקשה, בודקים את הקבצים שצירפתם בעזרת שתי שאלות:

1. **האם צירפתי את כל מה שהמשימה צריכה?** אם חסר משהו, המודל ינחש.
2. **האם כל מה שצירפתי באמת נחוץ?** אם לא, שילמתי בטוקנים על קבצים מיותרים.

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: שתי שאלות

ניסוח פורמלי - כיסוי, דיוק וטוקנים:

<u>הרעיון:</u>

לפני ששולחים בקשה, בודקים את הקבצים שצירפתם בעזרת שתי שאלות:

1. **האם צירפתי את כל מה שהמשימה צריכה?** אם חסר משהו, המודל ינחש.
2. **האם כל מה שצירפתי באמת נחוץ?** אם לא, שילמתי בטוקנים על קבצים מיותרים.

כדי לענות על השאלות במספרים, כותבים **שתי רשימות** ומשווים ביניהן: מה המשימה צריכה, ומה צירפתם בפועל.

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה המשימה צריכה

<u>רשימה 1 - מה המשימה צריכה:</u>

הקבצים שהמודל חייב לראות כדי לענות נכון, ולכל קובץ **כמה** ממנו: **תוכן מלא** אם המשימה משנה אותו, או **חתימות בלבד** אם היא רק משתמשת בפונקציה שלו. כל שורה ברשימה נקראת **צורך**, ואת כל הרשימה נסמן באות $\mathcal{N}$.

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה המשימה צריכה

<u>רשימה 1 - מה המשימה צריכה:</u>

הקבצים שהמודל חייב לראות כדי לענות נכון, ולכל קובץ **כמה** ממנו: **תוכן מלא** אם המשימה משנה אותו, או **חתימות בלבד** אם היא רק משתמשת בפונקציה שלו. כל שורה ברשימה נקראת **צורך**, ואת כל הרשימה נסמן באות $\mathcal{N}$.

במשימה שלנו (filter_by_date) יש שלושה צרכים:

קובץ	מה המשימה עושה איתו	מה המודל צריך לראות
tasks.py	מוסיפה לו את הפונקציה החדשה	תוכן מלא

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה המשימה צריכה

<u>רשימה 1 - מה המשימה צריכה:</u>

הקבצים שהמודל חייב לראות כדי לענות נכון, ולכל קובץ **כמה** ממנו: **תוכן מלא** אם המשימה משנה אותו, או **חתימות בלבד** אם היא רק משתמשת בפונקציה שלו. כל שורה ברשימה נקראת **צורך**, ואת כל הרשימה נסמן באות $\mathcal{N}$.

במשימה שלנו (filter_by_date) יש שלושה צרכים:

קובץ	מה המשימה עושה איתו	מה המודל צריך לראות
tasks.py	מוסיפה לו את הפונקציה החדשה	תוכן מלא
tests/test_tasks.py	מוסיפה לו טסט	תוכן מלא

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה המשימה צריכה

<u>רשימה 1 - מה המשימה צריכה:</u>

הקבצים שהמודל חייב לראות כדי לענות נכון, ולכל קובץ **כמה** ממנו: **תוכן מלא** אם המשימה משנה אותו, או **חתימות בלבד** אם היא רק משתמשת בפונקציה שלו. כל שורה ברשימה נקראת **צורך**, ואת כל הרשימה נסמן באות $\mathcal{N}$.

במשימה שלנו (filter_by_date) יש שלושה צרכים:

קובץ	מה המשימה עושה איתו	מה המודל צריך לראות
tasks.py	מוסיפה לו את הפונקציה החדשה	תוכן מלא
tests/test_tasks.py	מוסיפה לו טסט	תוכן מלא
dates.py	משתמשת בפונקציה parse_date שבו	חתימות בלבד

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה המשימה צריכה

<u>רשימה 1 - מה המשימה צריכה:</u>

הקבצים שהמודל חייב לראות כדי לענות נכון, ולכל קובץ **כמה** ממנו: **תוכן מלא** אם המשימה משנה אותו, או **חתימות בלבד** אם היא רק משתמשת בפונקציה שלו. כל שורה ברשימה נקראת **צורך**, ואת כל הרשימה נסמן באות $\mathcal{N}$.

במשימה שלנו (filter_by_date) יש שלושה צרכים:

קובץ	מה המשימה עושה איתו	מה המודל צריך לראות
tasks.py	מוסיפה לו את הפונקציה החדשה	תוכן מלא
tests/test_tasks.py	מוסיפה לו טסט	תוכן מלא
dates.py	משתמשת בפונקציה parse_date שבו	חתימות בלבד
9 הקבצים האחרים	לא נוגעת בהם	כלום (הם לא צורך)

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה צירפתם, ומתי צורך מכוסה

<u>רשימה 2 - מה צירפתם:</u>

הקבצים שצירפתם בפועל לבקשה, ולכל אחד: צירפתם את **כולו**, או רק את **החתימות** שלו. את הרשימה הזו נסמן באות $\mathcal{C}$.

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה צירפתם, ומתי צורך מכוסה

<u>רשימה 2 - מה צירפתם:</u>

הקבצים שצירפתם בפועל לבקשה, ולכל אחד: צירפתם את **כולו**, או רק את **החתימות** שלו. את הרשימה הזו נסמן באות $\mathcal{C}$.

<u>מתי צורך מכוסה:</u>

צורך **מכוסה** אם הקובץ שלו נמצא ברשימה 2, והוא צורף **לפחות** בכמות שהמשימה צריכה:

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה צירפתם, ומתי צורך מכוסה

<u>רשימה 2 - מה צירפתם:</u>

הקבצים שצירפתם בפועל לבקשה, ולכל אחד: צירפתם את **כולו**, או רק את **החתימות** שלו. את הרשימה הזו נסמן באות $\mathcal{C}$.

<u>מתי צורך מכוסה:</u>

צורך **מכוסה** אם הקובץ שלו נמצא ברשימה 2, והוא צורף **לפחות** בכמות שהמשימה צריכה:

- קובץ שצורף **במלואו** מכסה כל צורך באותו קובץ.

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה צירפתם, ומתי צורך מכוסה

<u>רשימה 2 - מה צירפתם:</u>

הקבצים שצירפתם בפועל לבקשה, ולכל אחד: צירפתם את **כולו**, או רק את **החתימות** שלו. את הרשימה הזו נסמן באות $\mathcal{C}$.

<u>מתי צורך מכוסה:</u>

צורך **מכוסה** אם הקובץ שלו נמצא ברשימה 2, והוא צורף **לפחות** בכמות שהמשימה צריכה:

- קובץ שצורף **במלואו** מכסה כל צורך באותו קובץ.
- קובץ שצורפו ממנו **רק חתימות** מכסה רק צורך של חתימות. אם המשימה צריכה לשנות את הקובץ, החתימות לא מספיקות.

ד״ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: מה צירפתם, ומתי צורך מכוסה

<u>רשימה 2 - מה צירפתם:</u>

הקבצים שצירפתם בפועל לבקשה, ולכל אחד: צירפתם את **כולו**, או רק את **החתימות** שלו. את הרשימה הזו נסמן באות $\mathcal{C}$.

<u>מתי צורך מכוסה:</u>

צורך **מכוסה** אם הקובץ שלו נמצא ברשימה 2, והוא צורף **לפחות** בכמות שהמשימה צריכה:

- קובץ שצורף **במלואו** מכסה כל צורך באותו קובץ.
- קובץ שצורפו ממנו **רק חתימות** מכסה רק צורך של חתימות. אם המשימה צריכה לשנות את הקובץ, החתימות לא מספיקות.

<u>טוקנים:</u>

המספר n_{F} הוא כמה טוקנים קובץ F תופס, כפי שצירפתם אותו: הרבה אם צירפתם את כולו, ומעט אם רק את החתימות.

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: נוסחאות

<u>נוסחאות:</u>

עכשיו שתי השאלות הופכות לשני שברים. בנוסחאות, קווים סביב רשימה מסמנים את מספר הפריטים בה.

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: נוסחאות

נוסחאות:

עכשיו שתי השאלות הופכות לשני שברים. בנוסחאות, קווים סביב רשימה מסמנים את מספר הפריטים בה.

כיסוי (recall) עונה על שאלה 1: כמה מהצרכים מכוסים, מתוך כל הצרכים. כיסוי 1 אומר שלא חסר כלום; פחות מזה, והמודל ינחש את מה שחסר.

$$\text{rec}(\mathcal{C}) = \frac{\#\{\text{needs covered by } \mathcal{C}\}}{|\mathcal{N}|}$$

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: נוסחאות

נוסחאות:

עכשיו שתי השאלות הופכות לשני שברים. בנוסחאות, קווים סביב רשימה מסמנים את מספר הפריטים בה.

כיסוי (recall) עונה על שאלה 1: כמה מהצרכים מכוסים, מתוך כל הצרכים. כיסוי 1 אומר שלא חסר כלום; פחות מזה, והמודל ינחש את מה שחסר.

$$\text{rec}(\mathcal{C}) = \frac{\#\{\text{needs covered by } \mathcal{C}\}}{|\mathcal{N}|}$$

דיוק (precision) עונה על שאלה 2: כמה מהקבצים שצירפתם מכסים צורך, מתוך כל הקבצים שצירפתם. דיוק 1 אומר שאין קובץ מיותר.

$$\text{prec}(\mathcal{C}) = \frac{\#\{\text{items in } \mathcal{C} \text{ that cover a need}\}}{|\mathcal{C}|}$$

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: נוסחאות

<u>נוסחאות:</u>

עכשיו שתי השאלות הופכות לשני שברים. בנוסחאות, קווים סביב רשימה מסמנים את מספר הפריטים בה.

כיסוי (recall) עונה על שאלה 1: כמה מהצרכים מכוסים, מתוך כל הצרכים. כיסוי 1 אומר שלא חסר כלום; פחות מזה, והמודל ינחש את מה שחסר.

$$\mathrm{rec}(\mathcal{C}) = \frac{\#\{\text{needs covered by } \mathcal{C}\}}{|\mathcal{N}|}$$

דיוק (precision) עונה על שאלה 2: כמה מהקבצים שצירפתם מכסים צורך, מתוך כל הקבצים שצירפתם. דיוק 1 אומר שאין קובץ מיותר.

$$\mathrm{prec}(\mathcal{C}) = \frac{\#\{\text{items in } \mathcal{C} \text{ that cover a need}\}}{|\mathcal{C}|}$$

המחיר הוא סכום הטוקנים של כל מה שצירפתם, כלומר האיבר n_{files} בתקציב החלון מחלק א׳:

$$n_{\text{files}} = \sum_{F \in \mathcal{C}} n_F$$

Vibe Coding

שיעור 4 - חלק ו׳ - ניסוח פורמלי: נוסחאות

<u>נוסחאות:</u>

עכשיו שתי השאלות הופכות לשני שברים. בנוסחאות, קווים סביב רשימה מסמנים את מספר הפריטים בה.

כיסוי (recall) עונה על שאלה 1: כמה מהצרכים מכוסים, מתוך כל הצרכים. כיסוי 1 אומר שלא חסר כלום; פחות מזה, והמודל ינחש את מה שחסר.

$$\mathrm{rec}(\mathcal{C}) = \frac{\#\{\text{needs covered by } \mathcal{C}\}}{|\mathcal{N}|}$$

דיוק (precision) עונה על שאלה 2: כמה מהקבצים שצירפתם מכסים צורך, מתוך כל הקבצים שצירפתם. דיוק 1 אומר שאין קובץ מיותר.

$$\mathrm{prec}(\mathcal{C}) = \frac{\#\{\text{items in } \mathcal{C} \text{ that cover a need}\}}{|\mathcal{C}|}$$

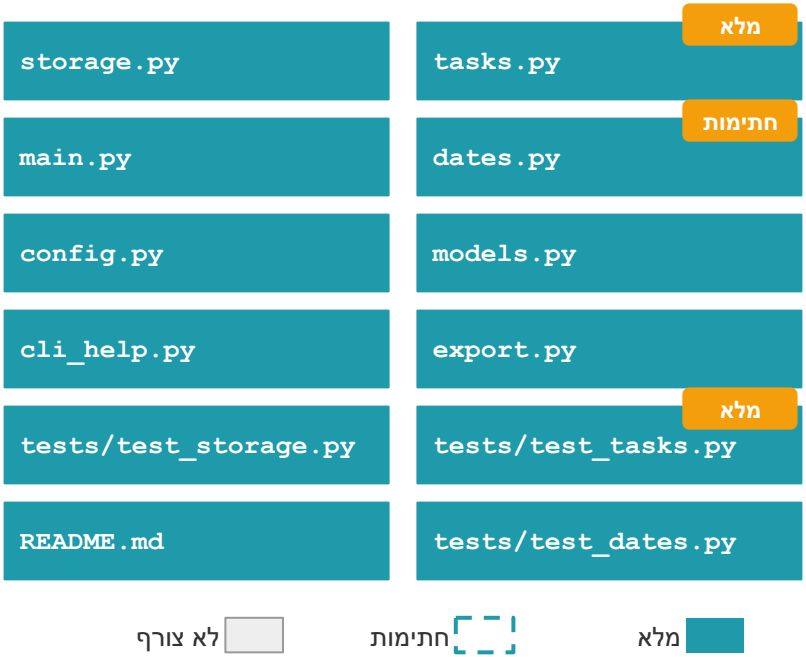
המחיר הוא סכום הטוקנים של כל מה שצירפתם, כלומר האיבר n_{files} בתקציב החלון מחלק א׳:

$$n_{\text{files}} = \sum_{F \in \mathcal{C}} n_F$$

אלה אותם מדדים שבודקים בהם מנועי חיפוש: מתוך מה שהיה צריך למצוא, כמה נמצא; ומתוך מה שנמצא, כמה רלוונטי.

Vibe Coding

שיעור 4 - חלק ו׳ - דוגמה מחושבת: שלוש אפשרויות



דוגמה מחושבת:

המשימה מהשקופית הקודמת, וגודלים להמחשה: tasks.py 180 שורות, dates.py 60, test_tasks.py 200, וכל הפרויקט כ-2,000, בכ-10 טוקנים לשורה. החתימות של dates.py: כ-60 טוקנים.

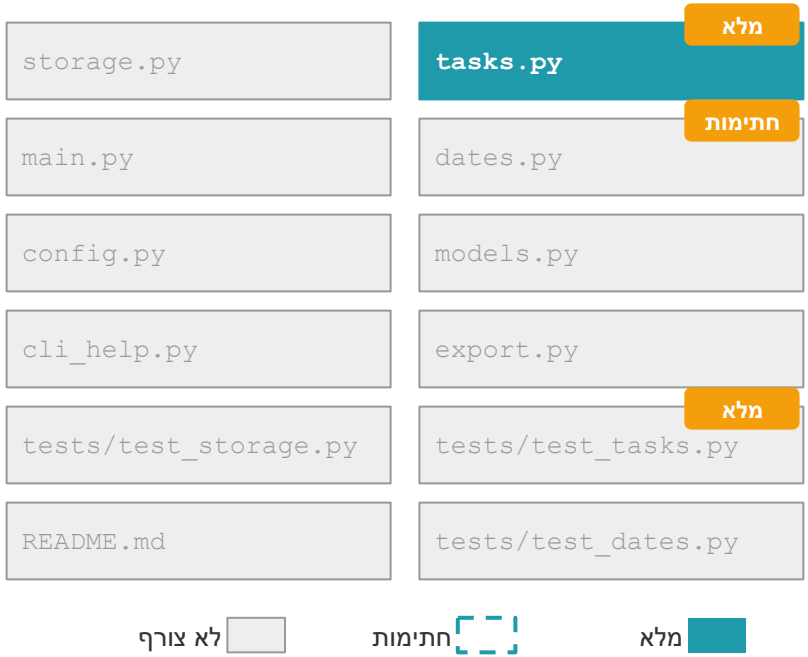
בטבלה: מה צירפתם, הכיסוי, הדיוק ו-n_{files}. ברשת משמאל: מלא (טורקיז), חתימות (מקווקו), לא צורף (אפור).

מה צירפתם	rec	prec	טוקנים
א. כל 12 הקבצים, מלאים	3/3	3/12	20,000

אפשרות א׳ מכסה הכול, אבל 75% ממה שצורף לא נחוץ, ועלות הטוקנים פי 5 מאפשרות ג׳.

Vibe Coding

שיעור 4 - חלק ו׳ - דוגמה מחושבת: שלוש אפשרויות



<u>דוגמה מחושבת:</u>

המשימה מהשקופית הקודמת, וגודלים להמחשה: tasks.py 180 שורות, dates.py 60, test_tasks.py 200, וכל הפרויקט כ-2,000, בכ-10 טוקנים לשורה. החתימות של dates.py: כ-60 טוקנים.

בטבלה: מה צירפתם, הכיסוי, הדיוק ו-n_{files}. ברשת משמאל: מלא (טורקיז), חתימות (מקווקו), לא צורף (אפור).

מה צירפתם	rec	prec	טוקנים
א. כל 12 הקבצים, מלאים	3/3	3/12	20,000
ב. רק tasks.py	1/3	1/1	1,800

אפשרות ב׳ זולה, אבל המודל לא יודע ש-parse_date קיימת: הוא יכתוב פענוח תאריכים משלו, או ינחש שם. גם את סגנון הטסטים הוא לא ראה.

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ו׳ - דוגמה מחושבת: שלוש אפשרויות

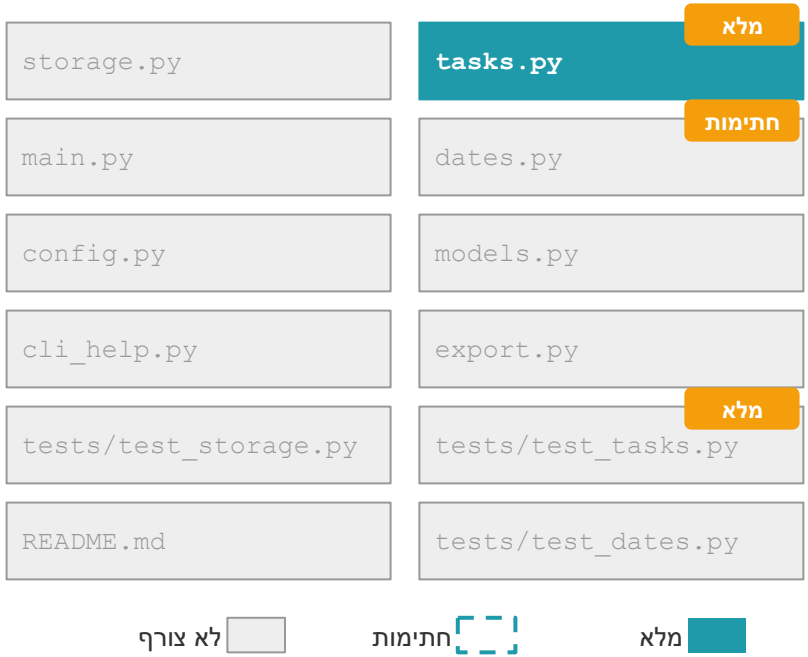
<u>דוגמה מחושבת:</u>

המשימה מהשקופית הקודמת, וגודלים להמחשה: tasks.py 180 שורות, dates.py 60, test_tasks.py 200, וכל הפרויקט כ-2,000, בכ-10 טוקנים לשורה. החתימות של dates.py: כ-60 טוקנים.

בטבלה: מה צירפתם, הכיסוי, הדיוק ו-n_{files}. ברשת משמאל: מלא (טורקיז), חתימות (מקווקו), לא צורף (אפור).

מה צירפתם	rec	prec	טוקנים
א. כל 12 הקבצים, מלאים	3/3	3/12	20,000
ב. רק tasks.py	1/3	1/1	1,800

שאלה: איך מכסים את כל הצרכים, בלי לשלם על כל הפרויקט?



ד"ר ולד לנדא

Vibe Coding

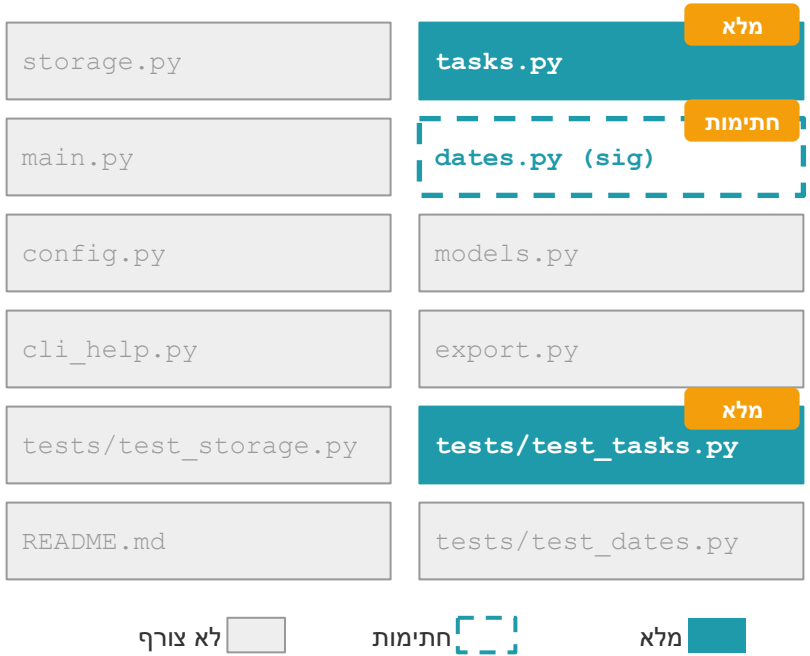
שיעור 4 - חלק ו׳ - דוגמה מחושבת: שלוש אפשרויות

<u>דוגמה מחושבת:</u>

המשימה מהשקופית הקודמת, וגודלים להמחשה: tasks.py 180 שורות, test_tasks.py 200, dates.py 60, וכל הפרויקט כ-2,000, בכ-10 טוקנים לשורה. החתימות של dates.py: כ-60 טוקנים.

בטבלה: מה צירפתם, הכיסוי, הדיוק ו-n_{files}. ברשת משמאל: מלא (טורקיז), חתימות (מקווקו), לא צורף (אפור).

מה צירפתם	rec	prec	טוקנים
א. כל 12 הקבצים, מלאים	3/3	3/12	20,000
ב. רק tasks.py	1/3	1/1	1,800
ג. 2 מלאים + חתימות	3/3	3/3	3,860



שאלה: איך מכסים את כל הצרכים, בלי לשלם על כל הפרויקט?

תשובה: אפשרות ג׳: שני הקבצים שהמשימה משנה, במלואם, ורק החתימות של dates.py. כיסוי מלא, בפחות מחמישית מהטוקנים של א׳.

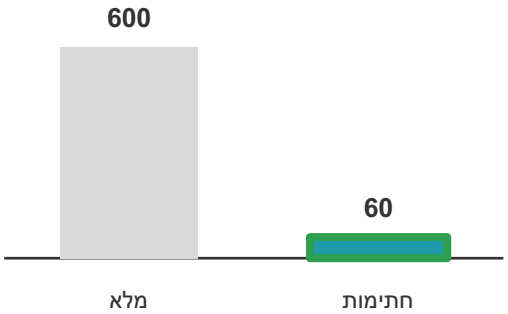
Vibe Coding

שיעור 4 - חלק ו׳ - מסקנות

<u>מסקנות:</u>

1. **קודם כיסוי, אחר כך חיסכון.** צורך שלא כוסה הופך להחלטה פתוחה, והמודל ינחש. רק כשכל הצרכים מכוסים מצמצמים טוקנים.

```python
# dates.py (signatures only)
def parse_date(text: str) -> date:
    """Parse 'DD/MM/YYYY' into a date.
    Raises ValueError on invalid input."""
def format_date(d: date) -> str:
    """Format a date as 'DD/MM/YYYY'."""
```



ד"ר ולד לנדא

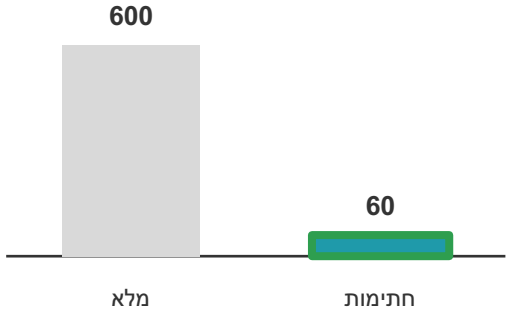
Vibe Coding

שיעור 4 - חלק ו׳ - מסקנות

<u>מסקנות:</u>

1. **קודם כיסוי, אחר כך חיסכון.** צורך שלא כוסה הופך להחלטה פתוחה, והמודל ינחש. רק כשכל הצרכים מכוסים מצמצמים טוקנים.
2. **מקובץ שרק משתמשים בפונקציה שלו מצרפים חתימות, לא תוכן מלא.** באפשרות ג׳ החתימות של dates.py עולות כ-60 טוקנים במקום 600 של הקובץ המלא, ומכסות את אותו צורך.

```python
# dates.py (signatures only)
def parse_date(text: str) -> date:
    """Parse 'DD/MM/YYYY' into a date.
    Raises ValueError on invalid input."""
def format_date(d: date) -> str:
    """Format a date as 'DD/MM/YYYY'."""
```



Vibe Coding

שיעור 4 - חלק ו׳ - הפניה לדפוס קיים

<u>הפניה לדפוס קיים:</u>

כשמוסיפים קוד שדומה לקוד קיים, הדרך הקצרה לסגור את כל החלטות הסגנון היא **להפנות אליו**. משמאל למעלה: הפונקציה הקיימת add_task.

קיים: tasks.py

```python
def add_task(tasks: list[dict], title: str) -> None:
    """Add a new task with the given title."""
    if not title.strip():
        raise ValueError("title must not be empty")
    tasks.append({"title": title, "done": False})
```

Vibe Coding

שיעור 4 - חלק ו׳ - הפניה לדפוס קיים

<u>הפניה לדפוס קיים:</u>

כשמוסיפים קוד שדומה לקוד קיים, הדרך הקצרה לסגור את כל החלטות הסגנון היא **להפנות אליו**. משמאל למעלה: הפונקציה הקיימת add_task.

למשל: **"ראה את add_task ב-tasks.py. כתוב edit_task באותו סגנון בדיוק: type hints, docstring, טיפול בקלט לא תקין."** משמאל למטה: התוצאה (פלט מייצג).

קיים: tasks.py

```python
def add_task(tasks: list[dict], title: str) -> None:
    """Add a new task with the given title."""
    if not title.strip():
        raise ValueError("title must not be empty")
    tasks.append({"title": title, "done": False})
```



חדש, באותו סגנון (פלט מייצג)

```python
def edit_task(tasks: list[dict], index: int,
              new_title: str) -> None:
    """Edit the title of the task at index."""
    if not 0 <= index < len(tasks):
        raise IndexError(f"no task at index {index}")
    tasks[index]["title"] = new_title
```

Vibe Coding

שיעור 4 - חלק ו׳ - הפניה לדפוס קיים

<u>הפניה לדפוס קיים:</u>

כשמוסיפים קוד שדומה לקוד קיים, הדרך הקצרה לסגור את כל החלטות הסגנון היא **להפנות אליו**. משמאל למעלה: הפונקציה הקיימת add_task.

למשל: **"ראה את add_task ב-tasks.py. כתוב edit_task באותו סגנון בדיוק: type hints, docstring, טיפול בקלט לא תקין."** משמאל למטה: התוצאה (פלט מייצג).

זה few-shot ממפגש 3, עם דוגמה **מהקוד שלכם**: פונקציה קיימת אחת סוגרת בבת אחת את כל החלטות הסגנון שהיא מדגימה.

קיים: tasks.py

```python
def add_task(tasks: list[dict], title: str) -> None:
    """Add a new task with the given title."""
    if not title.strip():
        raise ValueError("title must not be empty")
    tasks.append({"title": title, "done": False})
```



חדש, באותו סגנון (פלט מייצג)

```python
def edit_task(tasks: list[dict], index: int,
              new_title: str) -> None:
    """Edit the title of the task at index."""
    if not 0 <= index < len(tasks):
        raise IndexError(f"no task at index {index}")
    tasks[index]["title"] = new_title
```

Vibe Coding

שיעור 4 - חלק ו׳ - הפניה לדפוס קיים

<u>הפניה לדפוס קיים:</u>

כשמוסיפים קוד שדומה לקוד קיים, הדרך הקצרה לסגור את כל החלטות הסגנון היא **להפנות אליו**. משמאל למעלה: הפונקציה הקיימת add_task.

למשל: **"ראה את add_task ב-tasks.py. כתוב edit_task באותו סגנון בדיוק: type hints, docstring, טיפול בקלט לא תקין."** משמאל למטה: התוצאה (פלט מייצג).

זה few-shot ממפגש 3, עם דוגמה **מהקוד שלכם**: פונקציה קיימת אחת סוגרת בבת אחת את כל החלטות הסגנון שהיא מדגימה.

שאלה: מספיק לצרף כאן רק את החתימה של add_task?

קיים: tasks.py

```python
def add_task(tasks: list[dict], title: str) -> None:
    """Add a new task with the given title."""
    if not title.strip():
        raise ValueError("title must not be empty")
    tasks.append({"title": title, "done": False})
```



חדש, באותו סגנון (פלט מייצג)

```python
def edit_task(tasks: list[dict], index: int,
              new_title: str) -> None:
    """Edit the title of the task at index."""
    if not 0 <= index < len(tasks):
        raise IndexError(f"no task at index {index}")
    tasks[index]["title"] = new_title
```

Vibe Coding

שיעור 4 - חלק ו׳ - הפניה לדפוס קיים

<u>הפניה לדפוס קיים:</u>

כשמוסיפים קוד שדומה לקוד קיים, הדרך הקצרה לסגור את כל החלטות הסגנון היא **להפנות אליו**. משמאל למעלה: הפונקציה הקיימת add_task.

למשל: **"ראה את add_task ב-tasks.py. כתוב edit_task באותו סגנון בדיוק: type hints, docstring, טיפול בקלט לא תקין."** משמאל למטה: התוצאה (פלט מייצג).

זה few-shot ממפגש 3, עם דוגמה **מהקוד שלכם**: פונקציה קיימת אחת סוגרת בבת אחת את כל החלטות הסגנון שהיא מדגימה.

שאלה: מספיק לצרף כאן רק את החתימה של add_task?

תשובה: לא. **הסגנון נמצא בגוף.** כשמעתיקים סגנון, הקובץ עם הדוגמה הוא צורך של **תוכן מלא**, גם אם לא משנים אותו.

קיים: tasks.py

```python
def add_task(tasks: list[dict], title: str) -> None:
    """Add a new task with the given title."""
    if not title.strip():
        raise ValueError("title must not be empty")
    tasks.append({"title": title, "done": False})
```

חדש, באותו סגנון (פלט מייצג)

```python
def edit_task(tasks: list[dict], index: int,
              new_title: str) -> None:
    """Edit the title of the task at index."""
    if not 0 <= index < len(tasks):
        raise IndexError(f"no task at index {index}")
    tasks[index]["title"] = new_title
```

Vibe Coding

שיעור 4 - חלק ו׳ - מכאן לחלק הבא

<u>מכאן לחלק הבא:</u>

קובץ ההנחיה והקבצים המצורפים הם שניים מהחלקים שבשליטתכם. החלק השלישי הוא **ההיסטוריה.** כל מה שצירפתם, וכל תשובה של המודל, נשארים בה ונשלחים שוב בכל בקשה.
בחלק ז׳ נראה מה זה אומר כששיחה מתארכת.

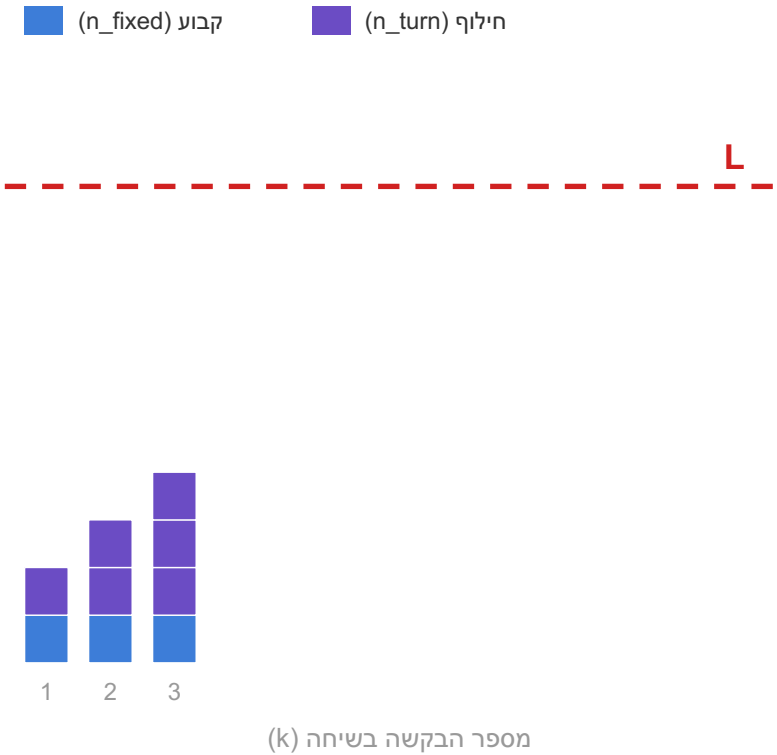


Vibe Coding

שיעור 4 - חלק ז׳ - שיחות ארוכות

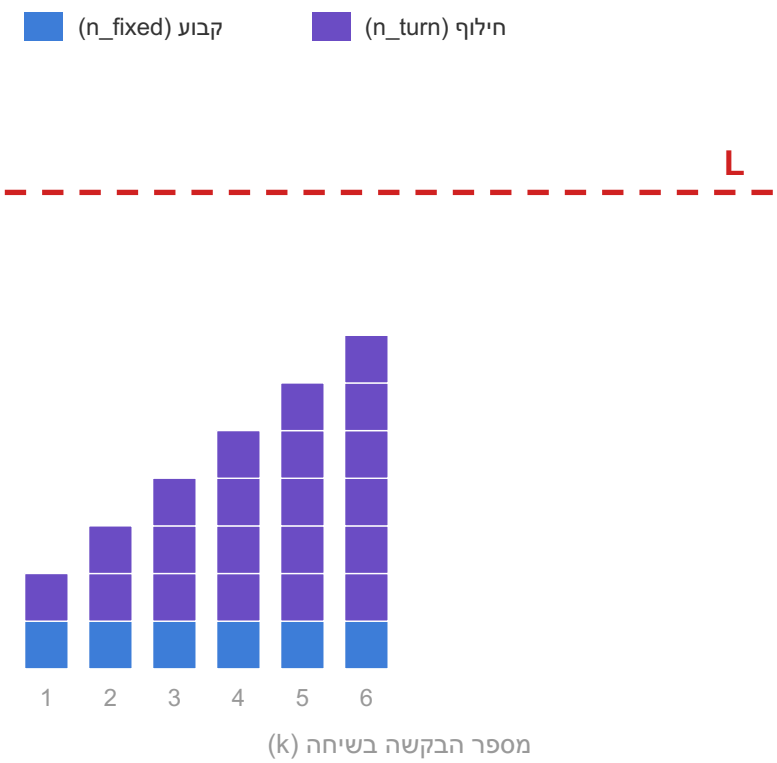
<u>למה זה חשוב:</u>

שיחה ארוכה מרגישה כמו יתרון ("הוא כבר מכיר את כל מה שעשינו"), אבל ההיסטוריה גדלה בכל בקשה **ונשלחת מחדש בכל בקשה** (מפגש 2).



Vibe Coding

שיעור 4 - חלק ז׳ - שיחות ארוכות



<u>למה זה חשוב:</u>

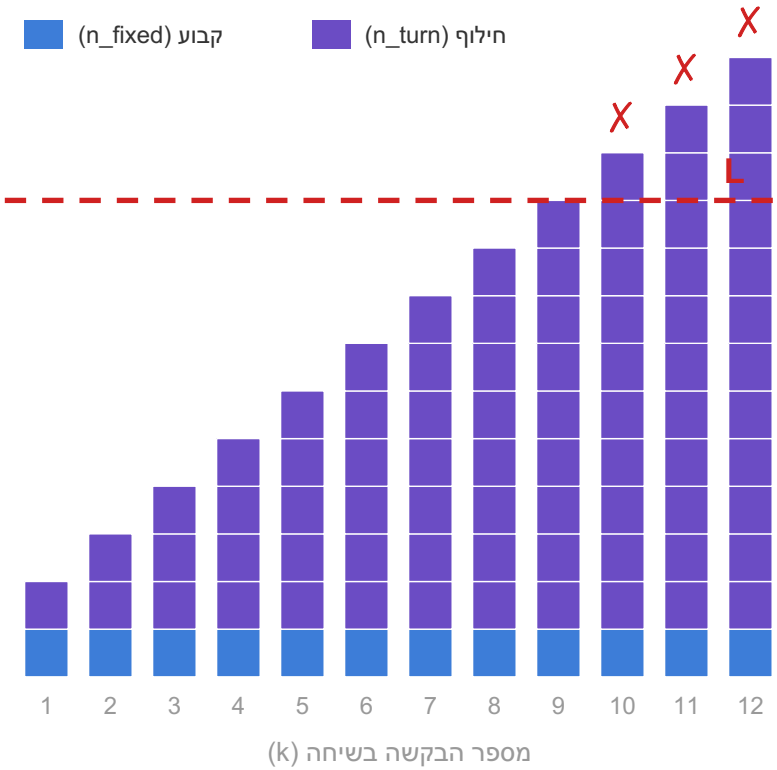
שיחה ארוכה מרגישה כמו יתרון ("הוא כבר מכיר את כל מה שעשינו"), אבל ההיסטוריה גדלה בכל בקשה **ונשלחת מחדש בכל בקשה** (מפגש 2).

<u>ההסבר:</u>

כל חילוף (הודעה שלכם + תשובה של המודל) מתווסף להיסטוריה, ולכן כל בקשה ארוכה מקודמתה. באיור: עמודה לכל בקשה; כחול הוא החלק הקבוע, וסגול הוא החילופים הקודמים.

Vibe Coding

שיעור 4 - חלק ז׳ - שיחות ארוכות



<u>למה זה חשוב:</u>

שיחה ארוכה מרגישה כמו יתרון ("הוא כבר מכיר את כל מה שעשינו"), אבל ההיסטוריה גדלה בכל בקשה **ונשלחת מחדש בכל בקשה** (מפגש 2).

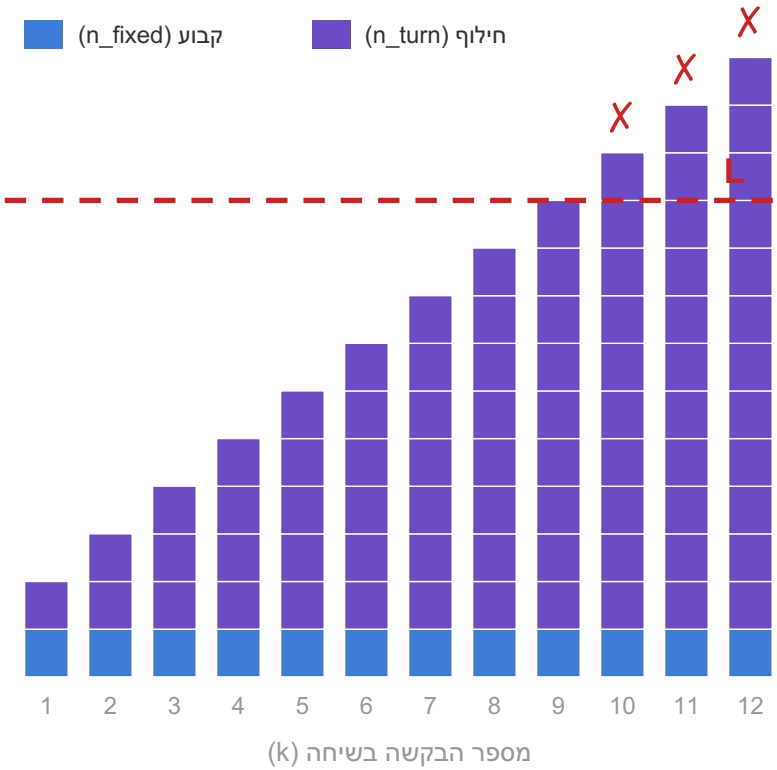
<u>ההסבר:</u>

כל חילוף (הודעה שלכם + תשובה של המודל) מתווסף להיסטוריה, ולכן כל בקשה ארוכה מקודמתה. באיור: עמודה לכל בקשה; כחול הוא החלק הקבוע, וסגול הוא החילופים הקודמים.

אחרי מספיק בקשות, הבקשה כבר ארוכה מחלון ההקשר L (הקו האדום המקווקו).

Vibe Coding

שיעור 4 - חלק ז׳ - שיחות ארוכות



<u>למה זה חשוב:</u>

שיחה ארוכה מרגישה כמו יתרון ("הוא כבר מכיר את כל מה שעשינו"), אבל ההיסטוריה גדלה בכל בקשה **ונשלחת מחדש בכל בקשה** (מפגש 2).

<u>ההסבר:</u>

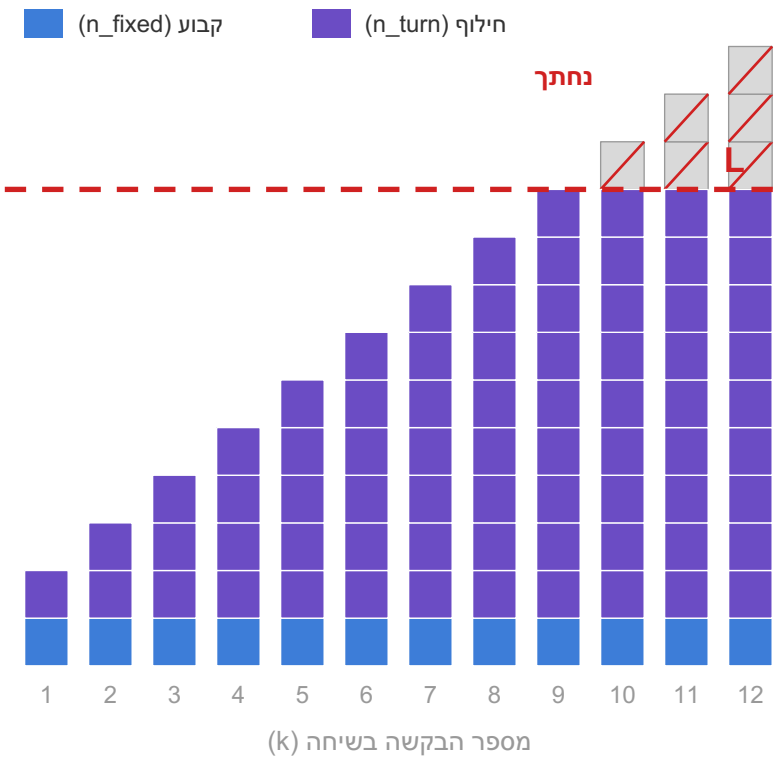
כל חילוף (הודעה שלכם + תשובה של המודל) מתווסף להיסטוריה, ולכן כל בקשה ארוכה מקודמתה. באיור: עמודה לכל בקשה; כחול הוא החלק הקבוע, וסגול הוא החילופים הקודמים.

אחרי מספיק בקשות, הבקשה כבר ארוכה מחלון ההקשר L (הקו האדום המקווקו).

שאלה: מה הכלי עושה כשהבקשה כבר לא נכנסת לחלון?

Vibe Coding

שיעור 4 - חלק ז׳ - שיחות ארוכות



<u>למה זה חשוב:</u>

שיחה ארוכה מרגישה כמו יתרון ("הוא כבר מכיר את כל מה שעשינו"), אבל ההיסטוריה גדלה בכל בקשה **ונשלחת מחדש בכל בקשה** (מפגש 2).

<u>ההסבר:</u>

כל חילוף (הודעה שלכם + תשובה של המודל) מתווסף להיסטוריה, ולכן כל בקשה ארוכה מקודמתה. באיור: עמודה לכל בקשה; כחול הוא החלק הקבוע, וסגול הוא החילופים הקודמים.

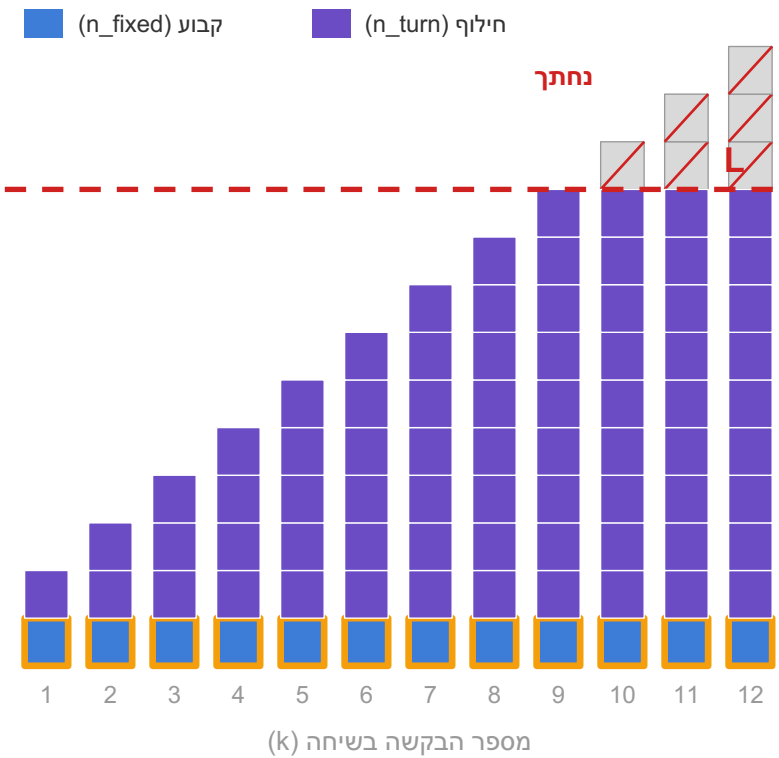
אחרי מספיק בקשות, הבקשה כבר ארוכה מחלון ההקשר L (הקו האדום המקווקו).

שאלה: מה הכלי עושה כשהבקשה כבר לא נכנסת לחלון?

תשובה: חותך או מתמצת את **החלק הישן**: ההודעות הראשונות, שבהן בדרך כלל כתבתם את הכללים.

Vibe Coding

שיעור 4 - חלק ז׳ - שיחות ארוכות



<u>למה זה חשוב:</u>

שיחה ארוכה מרגישה כמו יתרון ("הוא כבר מכיר את כל מה שעשינו"), אבל ההיסטוריה גדלה בכל בקשה **ונשלחת מחדש בכל בקשה** (מפגש 2).

<u>ההסבר:</u>

כל חילוף (הודעה שלכם + תשובה של המודל) מתווסף להיסטוריה, ולכן כל בקשה ארוכה מקודמתה. באיור: עמודה לכל בקשה; כחול הוא החלק הקבוע, וסגול הוא החילופים הקודמים.

אחרי מספיק בקשות, הבקשה כבר ארוכה מחלון ההקשר L (הקו האדום המקווקו).

שאלה: מה הכלי עושה כשהבקשה כבר לא נכנסת לחלון?

תשובה: חותך או מתמצת את **החלק הישן**: ההודעות הראשונות, שבהן בדרך כלל כתבתם את הכללים.

הוראות הכלי וקובץ ההנחיה, לעומת זאת, נשלחים מחדש בראש כל בקשה **ולא נחתכים** (החלק המודגש).

Vibe Coding

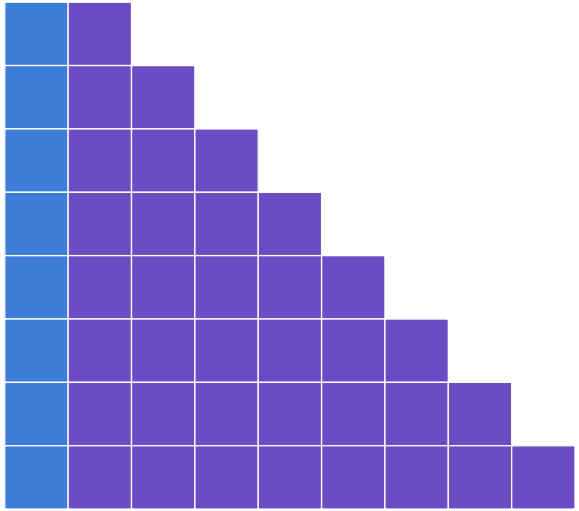
שיעור 4 - חלק ז' - ניסוח פורמלי

ניסוח פורמלי - כמה עולה שיחה של K בקשות:

<u>הגדרות (מודל מפשט):</u>

- K: מספר הבקשות בשיחה אחת; $k = 1, 2, ..., K$ הוא מספר הבקשה.

כל שורה = בקשה. סך הטוקנים = השטח



Vibe Coding

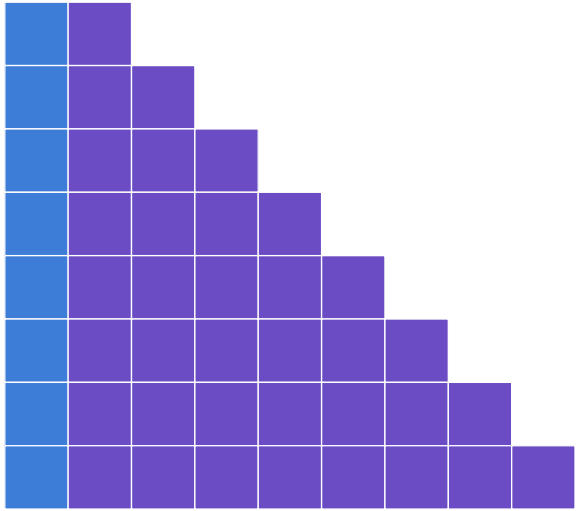
שיעור 4 - חלק ז׳ - ניסוח פורמלי

ניסוח פורמלי - כמה עולה שיחה של K בקשות:

<u>הגדרות (מודל מפשט):</u>

- K: מספר הבקשות בשיחה אחת; $k = 1, 2, ..., K$ הוא מספר הבקשה.
- n_{fixed}: הטוקנים שנשלחים בכל בקשה בלי קשר להיסטוריה (הוראות, קובץ הנחיה, קבצים קבועים).

כל שורה = בקשה. סך הטוקנים = השטח



Vibe Coding

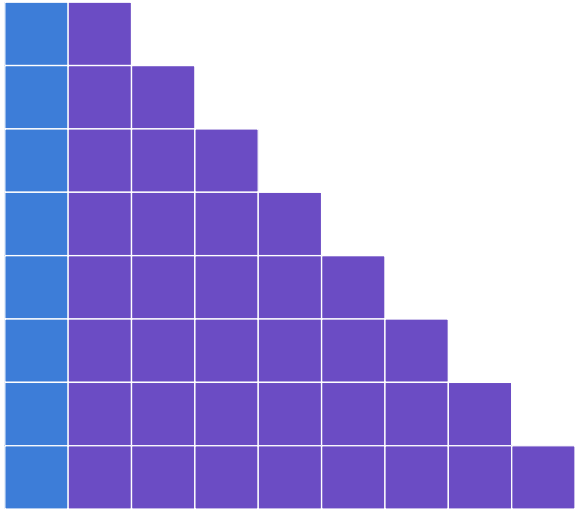
שיעור 4 - חלק ז׳ - ניסוח פורמלי

ניסוח פורמלי - כמה עולה שיחה של K בקשות:

<u>הגדרות (מודל מפשט):</u>

- K: מספר הבקשות בשיחה אחת; $k = 1, 2, \ldots, K$ הוא מספר הבקשה.
- n_{fixed}: הטוקנים שנשלחים בכל בקשה בלי קשר להיסטוריה (הוראות, קובץ הנחיה, קבצים קבועים).
- n_{turn}: הטוקנים שכל חילוף (הודעה + תשובה) מוסיף להיסטוריה, בממוצע.

כל שורה = בקשה. סך הטוקנים = השטח



Vibe Coding

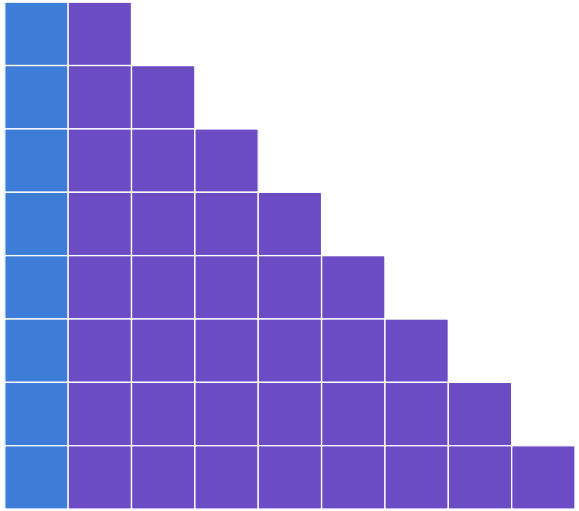
שיעור 4 - חלק ז׳ - ניסוח פורמלי

ניסוח פורמלי - כמה עולה שיחה של K בקשות:

<u>הגדרות (מודל מפשט):</u>

- K: מספר הבקשות בשיחה אחת; $k = 1, 2, ..., K$ הוא מספר הבקשה.
- n_{fixed}: הטוקנים שנשלחים בכל בקשה בלי קשר להיסטוריה (הוראות, קובץ הנחיה, קבצים קבועים).
- n_{turn}: הטוקנים שכל חילוף (הודעה + תשובה) מוסיף להיסטוריה, בממוצע.
- T_{k} : הטוקנים שהמודל מעבד בבקשה מספר k: החלק הקבוע, ועוד k חילופים בהיסטוריה ($k-1$ קודמים, ועוד הנוכחי).

כל שורה = בקשה. סך הטוקנים = השטח



Vibe Coding

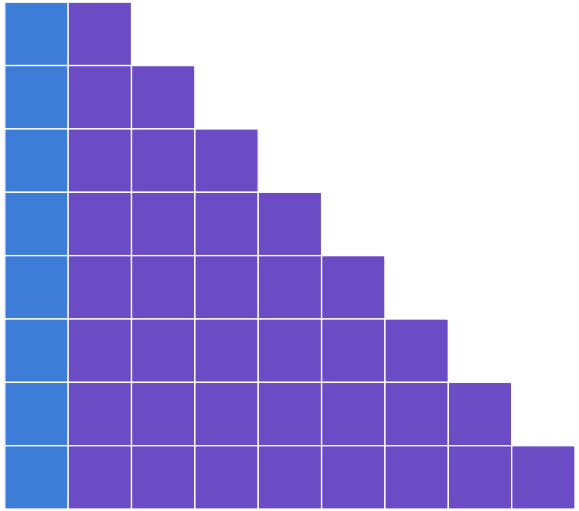
שיעור 4 - חלק ז׳ - ניסוח פורמלי

ניסוח פורמלי - כמה עולה שיחה של K בקשות:

<u>הגדרות (מודל מפשט):</u>

- K: מספר הבקשות בשיחה אחת; $k = 1, 2, ..., K$ הוא מספר הבקשה.
- n_{fixed}: הטוקנים שנשלחים בכל בקשה בלי קשר להיסטוריה (הוראות, קובץ הנחיה, קבצים קבועים).
- n_{turn}: הטוקנים שכל חילוף (הודעה + תשובה) מוסיף להיסטוריה, בממוצע.
- T_{k} : הטוקנים שהמודל מעבד בבקשה מספר k: החלק הקבוע, ועוד k חילופים בהיסטוריה ($k-1$ קודמים, ועוד הנוכחי).
- $K_{\max}$: מספר הבקשות המרבי לפני שהחלון L מתמלא, ואחריו הכלי מתחיל לחתוך או לתמצת.

כל שורה = בקשה. סך הטוקנים = השטח



Vibe Coding

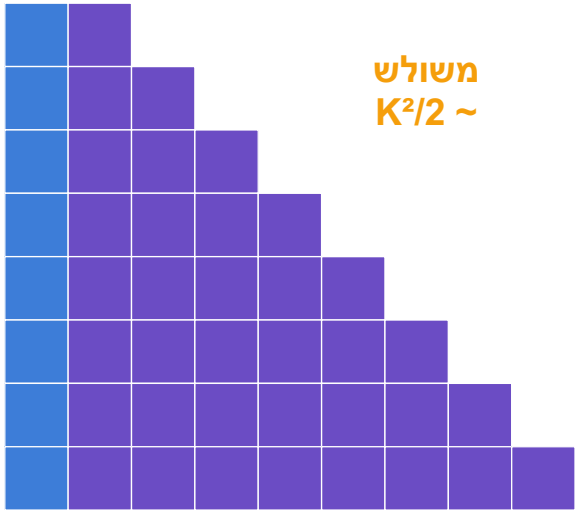
שיעור 4 - חלק ז׳ - ניסוח פורמלי: נוסחאות

<u>נוסחאות:</u>

הטוקנים בבקשה מספר k, לפי ההגדרה:

$$T_k = n_{\text{fixed}} + k \cdot n_{\text{turn}}$$

כל שורה = בקשה. סך הטוקנים = השטח



Vibe Coding

שיעור 4 - חלק ז׳ - ניסוח פורמלי: נוסחאות

נוסחאות:

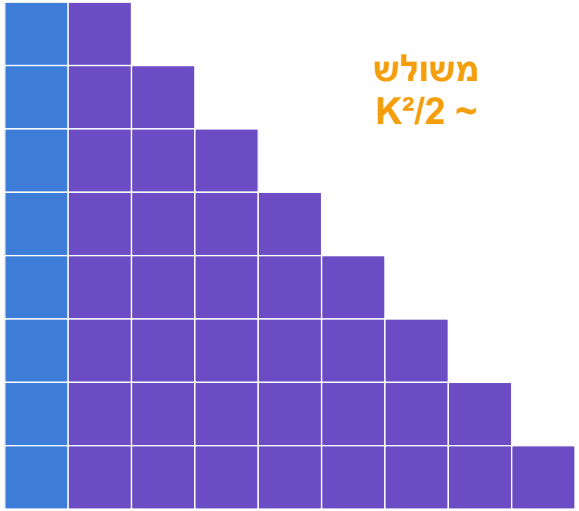
הטוקנים בבקשה מספר k, לפי ההגדרה:

$$T_k = n_{\text{fixed}} + k \cdot n_{\text{turn}}$$

הבקשה האחרונה שנכנסת לחלון בשלמותה ($\lfloor \cdot \rfloor$ הוא עיגול כלפי מטה):

$$K_{\max} = \left\lfloor \frac{L - n_{\text{fixed}}}{n_{\text{turn}}} \right\rfloor$$

כל שורה = בקשה. סך הטוקנים = השטח



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ז׳ - ניסוח פורמלי: נוסחאות

נוסחאות:

הטוקנים בבקשה מספר k, לפי ההגדרה:

$$T_k = n_{\text{fixed}} + k \cdot n_{\text{turn}}$$

הבקשה האחרונה שנכנסת לחלון בשלמותה ($\lfloor \cdot \rfloor$ הוא עיגול כלפי מטה):

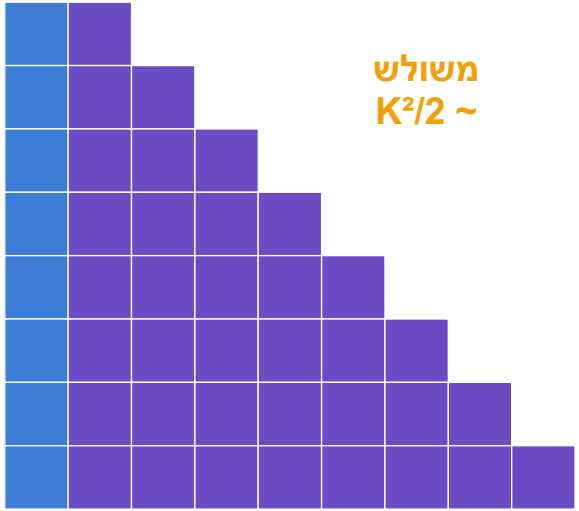
$$K_{\max} = \left\lfloor \frac{L - n_{\text{fixed}}}{n_{\text{turn}}} \right\rfloor$$

סך הטוקנים בשיחה, $T(K)$, הוא סכום T_k על כל הבקשות. בעזרת סכום הסדרה החשבונית:

$$1 + 2 + \cdots + K = \frac{K(K+1)}{2}$$

$$T(K) = \sum_{k=1}^{K} T_k = K \cdot n_{\text{fixed}} + n_{\text{turn}} \cdot \frac{K(K+1)}{2}$$

כל שורה = בקשה. סך הטוקנים = השטח



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ז׳ - ניסוח פורמלי: נוסחאות

<u>נוסחאות:</u>

הטוקנים בבקשה מספר k, לפי ההגדרה:

$$T_k = n_{\text{fixed}} + k \cdot n_{\text{turn}}$$

הבקשה האחרונה שנכנסת לחלון בשלמותה ($\lfloor \cdot \rfloor$ הוא עיגול כלפי מטה):

$$K_{\max} = \left\lfloor \frac{L - n_{\text{fixed}}}{n_{\text{turn}}} \right\rfloor$$

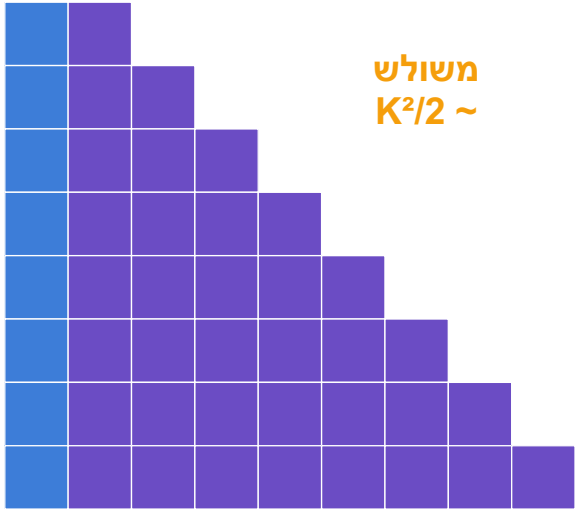
סך הטוקנים בשיחה, $T(K)$, הוא סכום T_k על כל הבקשות. בעזרת סכום הסדרה החשבונית:

$$1 + 2 + \cdots + K = \frac{K(K+1)}{2}$$

$$T(K) = \sum_{k=1}^{K} T_k = K \cdot n_{\text{fixed}} + n_{\text{turn}} \cdot \frac{K(K+1)}{2}$$

האיבר השני **ריבועי** ב-K: שיחה ארוכה פי 2 עולה כמעט פי 4 באיבר הזה (באיור: השטח של המשולש).

כל שורה = בקשה. סך הטוקנים = השטח

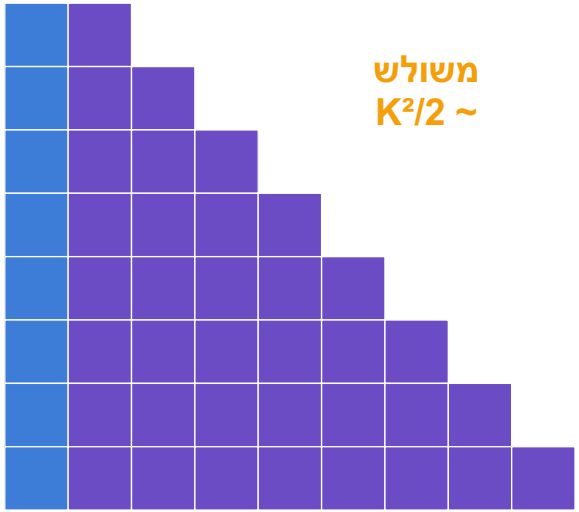


Vibe Coding

שיעור 4 - חלק ז׳ - דוגמה מחושבת: פתרון

אותם ערכים: n_{fixed} = 2,000, n_{turn} = 1,500. שיחה אחת של 40 בקשות,
מול שתיים של 20 עם סיכום מעבר של 1,000 טוקנים.

כל שורה = בקשה. סך הטוקנים = השטח



Vibe Coding

שיעור 4 - חלק ז׳ - דוגמה מחושבת: פתרון

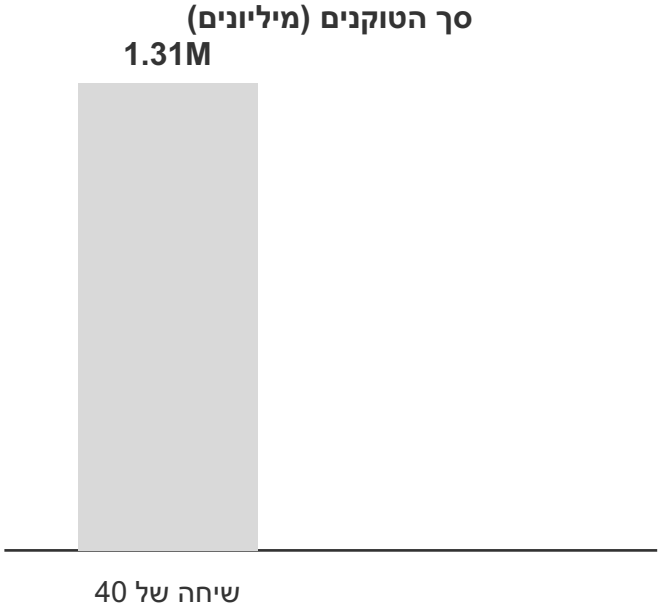
אותם ערכים: n_{fixed} = 2,000, n_{turn} = 1,500. שיחה אחת של 40 בקשות, מול שתיים של 20 עם סיכום מעבר של 1,000 טוקנים.

תשובה: שיחה אחת: מציבים $K = 40$ בנוסחת $T(K)$:

$$T(K) = K \cdot n_{\text{fixed}} + n_{\text{turn}} \cdot \frac{K(K+1)}{2}$$

$$T(40) = 40 \cdot 2{,}000 + 1{,}500 \cdot \frac{40 \cdot 41}{2}$$

$$= 1{,}310{,}000$$



Vibe Coding

שיעור 4 - חלק ז׳ - דוגמה מחושבת: פתרון

אותם ערכים: n_{fixed} = 2,000, n_{turn} = 1,500. שיחה אחת של 40 בקשות, מול שתיים של 20 עם סיכום מעבר של 1,000 טוקנים.

תשובה: שיחה אחת: מציבים $K = 40$ בנוסחת $T(K)$:

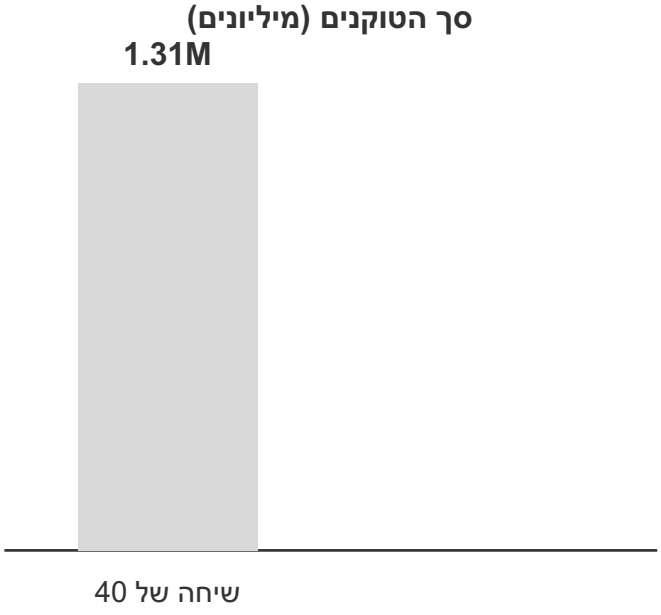
$$T(K) = K \cdot n_{\text{fixed}} + n_{\text{turn}} \cdot \frac{K(K+1)}{2}$$

$$T(40) = 40 \cdot 2{,}000 + 1{,}500 \cdot \frac{40 \cdot 41}{2}$$

$$= 1{,}310{,}000$$

שתי שיחות: בכל אחת $K = 20$. השיחה הראשונה:

$$T_{\text{1st}}(20) = 20 \cdot 2{,}000 + 1{,}500 \cdot \frac{20 \cdot 21}{2} = 355{,}000$$



Vibe Coding

שיעור 4 - חלק ז׳ - דוגמה מחושבת: פתרון

אותם ערכים: n_{fixed} = 2,000, n_{turn} = 1,500. שיחה אחת של 40 בקשות, מול שתיים של 20 עם סיכום מעבר של 1,000 טוקנים.

תשובה: שיחה אחת: מציבים $K = 40$ בנוסחת $T(K)$:

$$T(K) = K \cdot n_{\text{fixed}} + n_{\text{turn}} \cdot \frac{K(K+1)}{2}$$

$$T(40) = 40 \cdot 2{,}000 + 1{,}500 \cdot \frac{40 \cdot 41}{2}$$

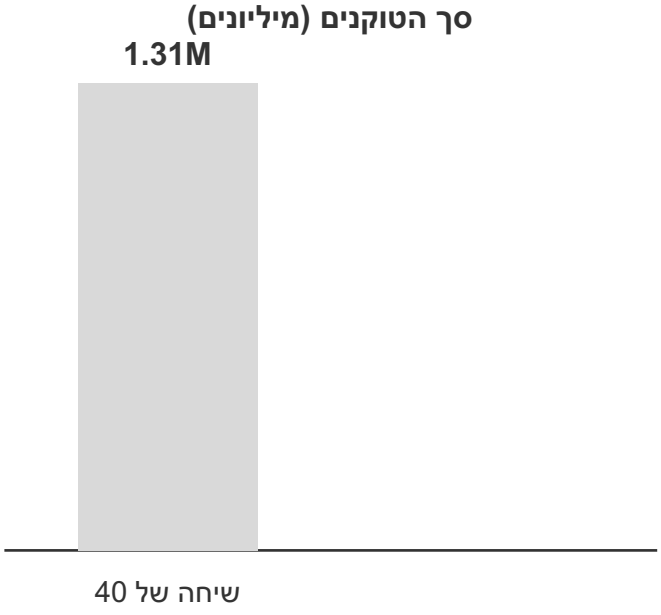
$$= 1{,}310{,}000$$

שתי שיחות: בכל אחת $K = 20$. השיחה הראשונה:

$$T_{\text{1st}}(20) = 20 \cdot 2{,}000 + 1{,}500 \cdot \frac{20 \cdot 21}{2} = 355{,}000$$

בשיחה השנייה הסיכום נשלח בכל בקשה, ולכן בה n_{fixed} = 3,000:

$$T_{\text{2nd}}(20) = 20 \cdot 3{,}000 + 1{,}500 \cdot \frac{20 \cdot 21}{2} = 375{,}000$$



Vibe Coding

שיעור 4 - חלק ז׳ - דוגמה מחושבת: פתרון

אותם ערכים: n_{fixed} = 2,000, n_{turn} = 1,500. שיחה אחת של 40 בקשות, מול שתיים של 20 עם סיכום מעבר של 1,000 טוקנים.

תשובה: שיחה אחת: מציבים $K = 40$ בנוסחת $T(K)$:

$$T(K) = K \cdot n_{\text{fixed}} + n_{\text{turn}} \cdot \frac{K(K+1)}{2}$$

$$T(40) = 40 \cdot 2{,}000 + 1{,}500 \cdot \frac{40 \cdot 41}{2}$$

$$= 1{,}310{,}000$$

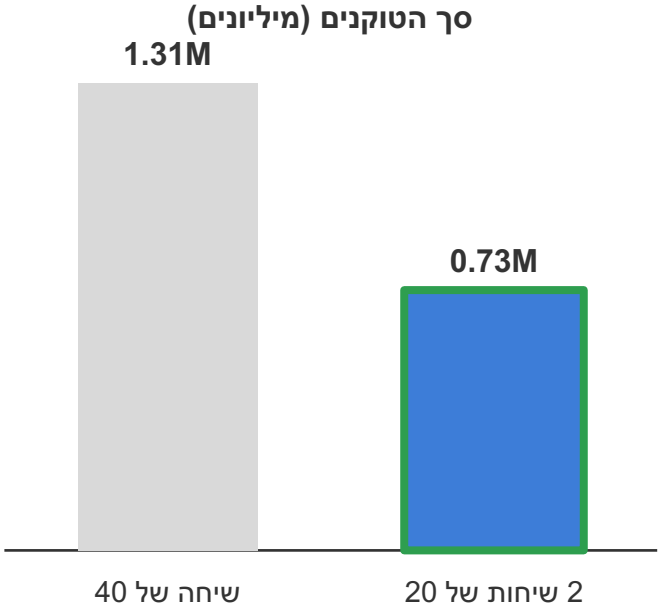
שתי שיחות: בכל אחת $K = 20$. השיחה הראשונה:

$$T_{\text{1st}}(20) = 20 \cdot 2{,}000 + 1{,}500 \cdot \frac{20 \cdot 21}{2} = 355{,}000$$

בשיחה השנייה הסיכום נשלח בכל בקשה, ולכן בה n_{fixed} = 3,000:

$$T_{\text{2nd}}(20) = 20 \cdot 3{,}000 + 1{,}500 \cdot \frac{20 \cdot 21}{2} = 375{,}000$$

ביחד 730,000 טוקנים: שתי השיחות עולות כ-56% מהשיחה האחת.



Vibe Coding

שיעור 4 - חלק ז׳ - סיכום מעבר

<u>תבנית לסיכום מעבר:</u>

סיכום מעבר הוא הדרך לפצל עבודה לכמה שיחות בלי לאבד את ההחלטות.

```
## Goal
What we are building (one sentence).
## Decisions
Decisions made, and why (one line each).
## Current state
Files changed, what works, what was tested.
## Open issues
Known bugs or open questions.
## Next step
The single next thing to do.
```

Vibe Coding

שיעור 4 - חלק ז׳ - סיכום מעבר

<u>תבנית לסיכום מעבר:</u>

סיכום מעבר הוא הדרך לפצל עבודה לכמה שיחות בלי לאבד את ההחלטות.

כשפותחים שיחה חדשה באמצע עבודה, מבקשים מהמודל (או כותבים בעצמכם) סיכום **בתבנית קבועה**, כמו התבנית משמאל, ומדביקים אותו כהודעה הראשונה בשיחה החדשה.

```
## Goal
What we are building (one sentence).
## Decisions
Decisions made, and why (one line each).
## Current state
Files changed, what works, what was tested.
## Open issues
Known bugs or open questions.
## Next step
The single next thing to do.
```

Vibe Coding

שיעור 4 - חלק ז׳ - סיכום מעבר

<u>תבנית לסיכום מעבר:</u>

סיכום מעבר הוא הדרך לפצל עבודה לכמה שיחות בלי לאבד את ההחלטות.

כשפותחים שיחה חדשה באמצע עבודה, מבקשים מהמודל (או כותבים בעצמכם) סיכום **בתבנית קבועה**, כמו התבנית משמאל, ומדביקים אותו כהודעה הראשונה בשיחה החדשה.

```
## Goal
What we are building (one sentence).
## Decisions
Decisions made, and why (one line each).
## Current state
Files changed, what works, what was tested.
## Open issues
Known bugs or open questions.
## Next step
The single next thing to do.
```

סיכום של 1,000 טוקנים במקום היסטוריה של 30,000: **שומר את ההחלטות ומוותר על הפרטים.**



Vibe Coding

שיעור 4 - חלק ז׳ - סיכום מעבר

<u>תבנית לסיכום מעבר:</u>

סיכום מעבר הוא הדרך לפצל עבודה לכמה שיחות בלי לאבד את ההחלטות.

כשפותחים שיחה חדשה באמצע עבודה, מבקשים מהמודל (או כותבים בעצמכם) סיכום **בתבנית קבועה**, כמו התבנית משמאל, ומדביקים אותו כהודעה הראשונה בשיחה החדשה.

```
## Goal
What we are building (one sentence).
## Decisions
Decisions made, and why (one line each).
## Current state
Files changed, what works, what was tested.
## Open issues
Known bugs or open questions.
## Next step
The single next thing to do.
```

סיכום של 1,000 טוקנים במקום היסטוריה של 30,000: **שומר את ההחלטות ומוותר על הפרטים.**

שאלה: מה **לא** נכנס לסיכום?



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ז׳ - סיכום מעבר

<u>תבנית לסיכום מעבר:</u>

סיכום מעבר הוא הדרך לפצל עבודה לכמה שיחות בלי לאבד את ההחלטות.

כשפותחים שיחה חדשה באמצע עבודה, מבקשים מהמודל (או כותבים בעצמכם) סיכום **בתבנית קבועה**, כמו התבנית משמאל, ומדביקים אותו כהודעה הראשונה בשיחה החדשה.

```
## Goal
What we are building (one sentence).
## Decisions
Decisions made, and why (one line each).
## Current state
Files changed, what works, what was tested.
## Open issues
Known bugs or open questions.
## Next step
The single next thing to do.
```

סיכום של 1,000 טוקנים במקום היסטוריה של 30,000: **שומר את ההחלטות ומוותר על הפרטים.**

שאלה: מה **לא** נכנס לסיכום?

תשובה: גרסאות שגויות של הקוד, שגיאות שכבר טופלו, ודיונים שהסתיימו: בדיוק מה שהסיכום נועד להשאיר מאחור.



Vibe Coding

שיעור 4 - חלק ז׳ - סיכום מעבר

<u>תבנית לסיכום מעבר:</u>

סיכום מעבר הוא הדרך לפצל עבודה לכמה שיחות בלי לאבד את ההחלטות.

כשפותחים שיחה חדשה באמצע עבודה, מבקשים מהמודל (או כותבים בעצמכם) סיכום **בתבנית קבועה**, כמו התבנית משמאל, ומדביקים אותו כהודעה הראשונה בשיחה החדשה.

```
## Goal
What we are building (one sentence).
## Decisions
Decisions made, and why (one line each).
## Current state
Files changed, what works, what was tested.
## Open issues
Known bugs or open questions.
## Next step
The single next thing to do.
```

סיכום של 1,000 טוקנים במקום היסטוריה של 30,000: **שומר את ההחלטות ומוותר על הפרטים.**

שאלה: מה **לא** נכנס לסיכום?

תשובה: גרסאות שגויות של הקוד, שגיאות שכבר טופלו, ודיונים שהסתיימו: בדיוק מה שהסיכום נועד להשאיר מאחור.



ומוסכמה קבועה של הפרויקט? **לא לסיכום, אלא לקובץ ההנחיה.**

Vibe Coding

שיעור 4 - חלק ז׳ - מכאן לחלק הבא

<u>מכאן לחלק הבא:</u>

עד עכשיו **אתם** בחרתם מה נכנס להקשר: כתבתם את קובץ ההנחיה, צירפתם קבצים, והחלטתם מתי לפתוח שיחה חדשה.
בחלק האחרון נחזור לפרויקט עצמו, ונראה איך **מבנה תיקיות ושמות** עוזרים לבחור הקשר טוב, גם כשהכלי בוחר אותו בעצמו.



Vibe Coding

שיעור 4 - חלק ח׳ - מבנה תיקיות, שמות ותיעוד

<u>למה זה חשוב:</u>

כלים שבוחרים קבצים בעצמם (סוכנים, מפגש 6) מחפשים לפי שמות, מבנה ותוכן. משמאל שני מבנים של אותו פרויקט Todo: לפני ואחרי.

לפני

```
stuff.py
```

```
stuff2.py
```

```
helper.py
```

```
main.py
```

```
test1.py
```

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ח׳ - מבנה תיקיות, שמות ותיעוד

<u>למה זה חשוב:</u>

כלים שבוחרים קבצים בעצמם (סוכנים, מפגש 6) מחפשים לפי שמות, מבנה ותוכן. משמאל שני מבנים של אותו פרויקט Todo: לפני ואחרי.

שאלה: הבקשה: **"הוסף פונקציה לשמירת מטלות".** לאיזה קובץ היא נכנסת?

לפני

```
stuff.py
```

```
stuff2.py
```

```
helper.py
```

```
main.py
```

```
test1.py
```

Vibe Coding

שיעור 4 - חלק ח׳ - מבנה תיקיות, שמות ותיעוד

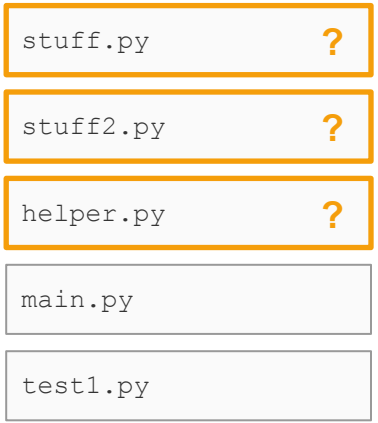
<u>למה זה חשוב:</u>

כלים שבוחרים קבצים בעצמם (סוכנים, מפגש 6) מחפשים לפי שמות, מבנה ותוכן. משמאל שני מבנים של אותו פרויקט Todo: לפני ואחרי.

שאלה: הבקשה: **"הוסף פונקציה לשמירת מטלות".** לאיזה קובץ היא נכנסת?

תשובה: בגרסה "לפני" אין לדעת: helper.py ?stuff.py? קובץ חדש?

לפני



Vibe Coding

שיעור 4 - חלק ח׳ - מבנה תיקיות, שמות ותיעוד

<u>למה זה חשוב:</u>

כלים שבוחרים קבצים בעצמם (סוכנים, מפגש 6) מחפשים לפי שמות, מבנה ותוכן. משמאל שני מבנים של אותו פרויקט Todo: לפני ואחרי.

שאלה: הבקשה: **"הוסף פונקציה לשמירת מטלות".** לאיזה קובץ היא נכנסת?

תשובה: בגרסה "לפני" אין לדעת: helper.py ?stuff.py? קובץ חדש?

בגרסה "אחרי", **storage.py סוגר את ההחלטה**, בלי מילה בקובץ ההנחיה.

לפני	אחרי
stuff.py ?	README.md
stuff2.py ?	tasks.py
helper.py ?	**storage.py** ✓
main.py	dates.py
test1.py	main.py
	tests/test_tasks.py
	tests/test_storage.py

ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ח׳ - מבנה תיקיות, שמות ותיעוד

<u>למה זה חשוב:</u>

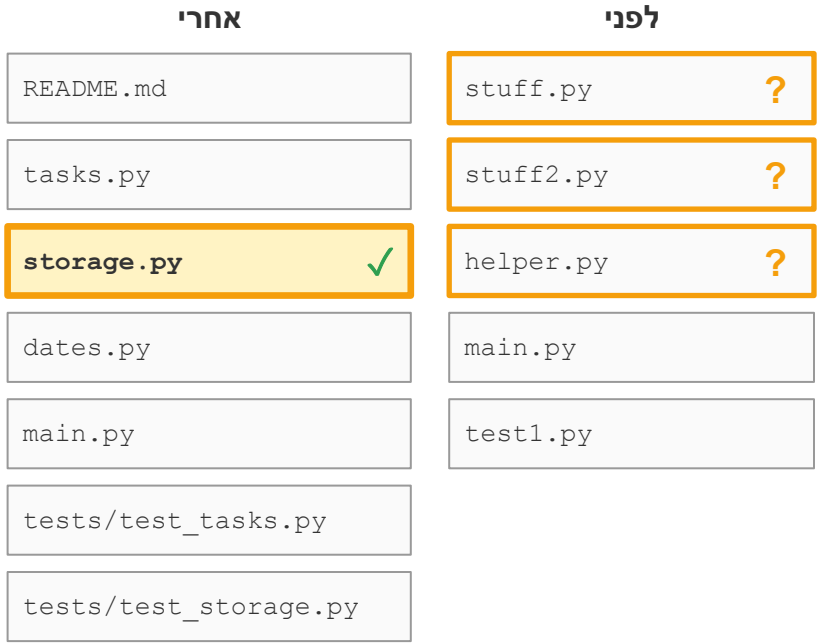
כלים שבוחרים קבצים בעצמם (סוכנים, מפגש 6) מחפשים לפי שמות, מבנה ותוכן. משמאל שני מבנים של אותו פרויקט Todo: לפני ואחרי.

שאלה: הבקשה: **"הוסף פונקציה לשמירת מטלות".** לאיזה קובץ היא נכנסת?

תשובה: בגרסה "לפני" אין לדעת: helper.py ?stuff.py? קובץ חדש?

בגרסה "אחרי", **storage.py סוגר את ההחלטה**, בלי מילה בקובץ ההנחיה.

וכלי שמחפש לבד ימצא את storage.py, כי השם **קרוב במשמעות** ל"שמירת מטלות" (embeddings, מפגש 2). את stuff2.py הוא לא ימצא.



ד"ר ולד לנדא

Vibe Coding

שיעור 4 - חלק ח׳ - מבנה תיקיות, שמות ותיעוד

<u>למה זה חשוב:</u>

כלים שבוחרים קבצים בעצמם (סוכנים, מפגש 6) מחפשים לפי שמות, מבנה ותוכן. משמאל שני מבנים של אותו פרויקט Todo: לפני ואחרי.

שאלה: הבקשה: **"הוסף פונקציה לשמירת מטלות".** לאיזה קובץ היא נכנסת?

תשובה: בגרסה "לפני" אין לדעת: helper.py ?stuff.py? קובץ חדש?

בגרסה "אחרי", **storage.py סוגר את ההחלטה**, בלי מילה בקובץ ההנחיה.

וכלי שמחפש לבד ימצא את storage.py, כי השם **קרוב במשמעות** ל"שמירת מטלות" (embeddings, מפגש 2). את stuff2.py הוא לא ימצא.

כלומר, **מבנה טוב מעלה את הכיסוי** כשהכלי בוחר את ההקשר בעצמו.

לפני	אחרי
stuff.py ?	README.md
stuff2.py ?	tasks.py
helper.py ?	**storage.py** ✓
main.py	dates.py
test1.py	main.py
	tests/test_tasks.py
	tests/test_storage.py

Vibe Coding

שיעור 4 - דמו חי: בונים ובודקים קובץ הנחיה

עברנו על כל החלקים של ההקשר שבשליטתכם. עכשיו נחבר אותם: נבנה קובץ הנחיה קצר, **ונבדוק אם הוא באמת משנה את התשובה.** הפרויקט: מחשבון עם פונקציה אחת, calculate(a, op, b).

שלב 1 - בלי קובץ הנחיה, בשיחה חדשה: "תוסיף למחשבון אפשרות לחשב חזקה". תוצאה טיפוסית: הקוד עובד, אבל בלי type hints ובלי טיפול בחלוקה באפס או בקלט לא תקין.

שלב 2 - בונים יחד קובץ של שלוש שורות:

```
Stack: Python 3.11, type hints required.
Do: handle every edge case (division by zero, invalid op).
Don't: add external libraries.
```

שלב 3 - אותו פרומפט, בשיחה חדשה, עם הקובץ. משווים: האם יש type hints? האם יש טיפול במקרי קצה?

שלב 4 - שאלה לכיתה: כמה החלטות סגר הקובץ? מה עוד נשאר פתוח, והאם הוא שייך לקובץ או לפרומפט?

ד״ר ולד לנדא

Vibe Coding

שיעור 4 - סיכום

מסגרת	מסקנה
הרכב הקלט (א׳)	כל מה שהמודל יודע על הפרויקט נמצא בקלט; ההודעה היא חלק אחד מחמישה
עקביות לאורך Q בקשות (ב׳)	עבור בקשה אחת המוסכמה נשמרת לרוב, אבל לאורך פרויקט כמעט בטוח שתהיה חריגה. הקובץ מבטל את הסיכוי לשכוח, לא את הצורך לבדוק
ערך של כלל (ד׳)	לקובץ נכנס כלל שחוזר, מתקן טעות של המודל, ואפשר לבדוק אותו
כלל מיושן (ה׳)	כלל מיושן מזיק יותר מאין כלל; מעדכנים אותו באותו commit
הקשר ממוקד (ו׳)	קודם כיסוי מלא, אחר כך מינימום טוקנים. מקובץ שרק משתמשים בפונקציה שלו, מספיקות חתימות
שיחה ארוכה (ז׳)	עלות ריבועית, וההתחלה נחתכת ראשונה. שיחה אחת למשימה, וסיכום מעבר בין שיחות
מבנה ושמות (ח׳)	מבנה, שמות ותיעוד מאפשרים לכלי שבוחר הקשר לבד להגיע לכיסוי מלא

Vibe Coding

שיעור 4 - מה הלאה

תרגול (HM_04, אפשר בבית): האם קובץ ההנחיה באמת עוזר? במקום לנחש, מודדים:

- בונים בודק מוסכמות אוטומטי (עם המודול ast של Python): type hints, docstrings, שמות, תלויות, טיפול ברשימה ריקה.
- שולחים חמש בקשות **בלי** קובץ הנחיה ו**עם** קובץ, כל אחת בשיחה חדשה.
- אומדים לכל מוסכמה את g^{open}, את g^{closed} ואת Δ, ומחליטים איזה כלל אפשר למחוק.

- שיעורי בית: מממשים את $T(K)$ ואת $K_{\max}$, ובודקים כמה חוסך פיצול של שיחה בת 60 בקשות.

מפגש 5: אם הכלים כל כך דומים ברעיון, **איך בוחרים ביניהם?** אחד הקריטריונים יהיה בדיוק מה שלמדנו היום: איך הכלי בונה את ההקשר, ומה הוא מאפשר לכם לשלוט בו.